POLS 603 Notes*

Casey Crisman-Cox

Last updated: 19 February 2026

*These notes are for personal use only and not for distribution. They are rife with errors, typos, and are not referenced. This is not my own work, but simply notes for me to lecture from. I do not claim any original work other than in adaptating from other works. Do not cite or disseminate these notes.

Contents

1 MLE setup and basics

Before we start getting this semester, let's refresh some of our memory about what empirical models are, what they're good for, and what are actual goals are in learning this stuff.

In the kind of social science we do there are three main uses for empirical models:

1. **Predict** values of y for new values of X that maybe we haven't observed yet (e.g., forecasting election outcomes or predicting whether some individual enters a job training program)
2. **Describe** how do changes in y match changes in X (on average voters of color tend to vote at lower rates than white votes)
3. **Identify the treatment/marginal (causal) effect** that a change in x_i will have on values of y_i (i.e., How many more votes can a candidate get by spending another dollar, on average?)

We will often have one or more of these goals in mind when conducting research. Overwhelmingly, our goal is to identify a marginal effect or treatment effect. Often, sadly, we won't be in a position to credibly label it a casual effect, although we will talk about when it makes sense to think of it that way. In other cases, we'll be somewhere in the overlap of 2 and 3, where we acknowledge that we're describing associations, but taking care to get as close to 3 as possible.

Recall that the major obstacle in moving from 2 to 3 is **endogeneity**. Most often, we think about this as some kind of ommited variable bias, but it is broader than that. We can imagine breaking down estimation strategies into several approaches based on how they handle endogeneity concerns (i.e., how to get from 2 to 3):

1. Conditioning on the observables. This is probably the most common, but also the most suspect. Here we are just relying on having all the correct control variables in a regression or matching setting. This is why we often find ourselves somewhere between 2 & 3 and talking about association.
2. Instrumental variables. If a valid and strong instrument is available it can avoid many endogeneity problems.
3. Fixed effects. If we have panel data, we can control for unobserved heterogeneity among the units in our study. Under additional assumptions we can get to difference-in-differences designs
4. Structural approaches. Modeling the endogeneity directly (e.g., selection into the sample or strategic interactions).

This semester we will largely be exploring what are known as generalized linear models (GLMs). To preview there are pros and cons to consider when deciding if you want to use a GLM versus an ordinary linear model like you learned about last semester.

Pro GLMs take the types of values y can take on very seriously. For example, if y is Bernoulli then $\Pr(y = 1) \in [0, 1]$, but you probably saw last semester that the linear probability model (LPM) can produce nonsense estimates of this probability.

Con They often require additional assumptions on y, these assumptions are often not based on anything substantive or defensible.

Con In many cases, GLM estimates do not have closed-form solutions. This means we need to solve for them using numerical methods rather than having a go-to formula like $(X'X)^{-1}X'y$.

GLMs can be fit in any number of ways, but we will focus on the method of maximum likelihood. This method is very a powerful tool for fitting empirical models. At its core, a maximum likelihood estimator contains three parts:

1. A model of the data. This can be as simple or as complicated as needed, but without a model of where the data come from we cannot move on. Some common choices include a common named distribution or a theoretical model. You'll often see this model called the *data generating process (DGP)*. This model has parameters $\theta \in \Theta$ that we wish to estimate.
2. The observed data y_i, $i = 1, \ldots, N$. These are realizations from the random variable y. We will fit the model to these data to estimate θ
3. A likelihood function $\mathcal{L} : \Theta \to \mathbb{R}$. In words, we input a guess at θ and $\mathcal{L}$ returns a number that describes how how "likely" is it that the observed data y_i were generated from the assumed DGP/model if its parameters were in fact the guessed θ.

Intuitively, we want to find the values of θ that maximize the likelihood function. That is, what values of θ are the most likely for this model given the data.

You were probably been exposed to some applications of maximum likelihood estimation last semester, but let start at the very beginning with a review some of important terminology.

1.1 Random variables and distributions

A random variable is a function $y : S \to \mathbb{R}$, mapping from the **sample space** (the set of all possible outcomes from whatever it is we're doing) S into some kind of numeric outcome. For example, imagine that flip a coin N times and count the number of heads. The sample

space is then $S = \{H, T\}^N$, which is incredibly unwieldy. The random variable y maps these sequences into the numbers 1, 2, ..., N.

While random variables are, and always will be, functions, it is often useful to think about them in terms of outcomes of an observed experiment. The intuition here is that we have an experiment in mind and the random variable will record the outcome when we run in the future. In the present we are simply thinking about all the things that might happen when we run the experiment and how likely those different outcomes are.

When discussing random variables, we will frequently describe them in terms of the distribution over outcomes. These can be the probability of a specific outcome $\Pr(y = y_i)$ or that the outcome is within an interval $\Pr(a \leq y_i \leq b)$. Typically, we'll want to distingush between discrete and continuous random variables.

A random variable y is discrete if its outcomes are a countable set (finite or not). The **probability mass function** (PMF) f describes the probability of each event such that

$$\begin{aligned} f(y_i) &= \Pr(y = y_i) \\ \Pr(y = A) &= \sum_{i:y_i \in A} f(y_i) \\ 1 &= \sum_i f(y_i), \forall y_i \in \text{supp}(y). \end{aligned}$$

Where $\text{supp}(y)$ is the **support** of y, which just means all values that y can take on with positive probability. Some examples include

- Bernoulli(p), $\text{supp}(y) = \{0, 1\}$
- Binomial(N, p), $\text{supp}(y) = \{0, 1, \ldots, N\}$. The sum of N iid Bernoulli(p) random variables
- Poisson(λ), $\text{supp}(y) = \{0, 1, \ldots\}$, which is the set of all non-negative integers $\mathbb{Z}_+$.

Consider tossing a coin once. This is a realization from a Bernoulli distribution, where with probability p, we observe a head. The PMF for the Bernoulli is a fairly simple

$$f(y_i|p) = p^{y_i}(1-p)^{1-y_i}.$$

If our experiment is do determine if the coin is fair, we may toss it N times. Each set of N tosses produces an observation y_i, which is the total number of heads. A few questions to make sure you're paying attention:

1. What kind of distribution is this? Binomial.
2. What is the probability that we get a specific sequence of y_i heads and $N - y_i$ tails? For example, all y_i heads in a row and then all tails? Well each toss is independent, so it is simply the product

$$\Pr(y_1 = 1) \dots \Pr(y_i = 1)(1 - \Pr(y_{i+1} = 1)) \dots (1 - \Pr(y_N = 1)) = p^{y_i}(1-p)^{N-y_i}.$$

3. Ok now what is the probability of observing any sequence containing y_i heads? This is the probability of a specific sequence times the number of specific sequences that exist. So how many different ways are there to get y_i heads in N tosses? We'll save some and note that this is a solved problem, we can use the binomial or "choose" function:

$$\binom{N}{y_i} = \frac{N!}{(N-y_i)!y_i!}.$$

So the PMF of the binomial is

$$f(y_i|N,p) = \Pr(y = y_i) = \binom{N}{y_i} p^{y_i}(1-p)^{N-y_i}.$$

When $N = 1$, this simplifies to the Bernoulli distribution.

A random variable y has a continuous distribution if there is a weakly positive function f such that for all a and b

$$\Pr(a < y \leq b) = \int_a^b f(y_i)dy_i$$

and

$$\int_{-\infty}^{\infty} f(y_i)dy_i = 1.$$

Now f is called a **probability density function** (PDF), which is similar to, but not the same as a PMF. For a continuous variable y, $\Pr(y = y_i) = 0$ for all y_i. Instead, the PDF $f(y_i)$,returns a density which can be thought of as reflecting how likely are to see something "near" y_i. Some examples include

- Uniform $U[a, b]$ where every value in the interval is equally likely. This PDF is then $f(y_i) = 1/(b-a)$, $\text{supp}(y) = [a, b]$
- Normal $N(\mu, \sigma^2)$, you've surely seen this before and will see it extensively, $\text{supp}(y) = \mathbb{R}$
- Exponential(λ), where $f(y_i|\lambda) = \lambda \exp(-\lambda y_i)$ for $\lambda > 0$ and $y > 0$. $\text{supp}(y) = \mathbb{R}_+$
- t, χ^2, F, logistic, and many others

As in the binomial example, we can say that if we have a sequence of $y_1, \dots, y_N$ random

variables than their joint density is given as

$$f(y_1, y_2, \ldots, y_N) = \prod_{i=1}^{N} f(y_i).$$

Finally, a **cumulative distribution function** (CDF) is as a function $F : \mathbb{R} \to [0, 1]$ such that $F(y_i) = \Pr(y \leq y_i)$. A few interesting facts about CDFs that follow from the axioms of probability are

1. F is weakly increasing $F(y_i) \leq F(y_j)$ for all $i \leq j$.
2. F does not have to be continuous, but it is continuous from the right, i.e.

$$\lim_{h \to 0^+} F(y_i + h) = F(y_i).$$

   The notation $h \to 0^+$ means as h approaches 0 from the right ($h > 0$, decreasing to 0).
3. $\lim_{y_i \to \infty} F(y_i) = 1$ and
4. $\lim_{y_i \to -\infty} F(y_i) = 0$
5. $0 \leq F(y_i) \leq 1$

For discrete random variables, the CDF is defined as

$$F(y_i) = \sum_{y \leq y_i} y,$$

while for continuous random variables it is

$$F(y_i) = \int_{-\infty}^{y_i} f(y) dy.$$

Note, that this means that $f(y_i) = D_y F(y_i)$.

1.2 Likelihood function

Suppose that we believe the data come from a normal distribution with $\theta = (\mu, \sigma^2)$. Our goal is to produce the maximum likelihood estimate of θ. So what do we need? We have

1. A model, the normal distribution
2. Data, y_i

What's left? A likelihood function! Before we get to the likelihood in practice, let's think about what we what it to be in theory. We want to find the values of θ that are *most likely*

given the observed data so the problem we're looking at should be something like

$$\hat{\theta} = \underset{\theta \in \Theta}{\operatorname{argmax}}\, g(\theta|y),$$

where Θ is the **parameter space,** which is the set of all possible values that θ could be.

What is g though? Well maybe this is case where Bayes' rule can help us out,

$$g(\theta|y) = \frac{f(y|\theta)g(\theta)}{f(y)}.$$

Ok here we have:

1. $f(y|\theta)$ is the probability density/mass function of y from our model.
2. $g(\theta)$ is an unconditional distribution that maps from the parameter space $\Theta \to [0, 1]$. This is what's known as a **prior** distribution.
3. $f(y)$ is the unconditional probability of the observed data. Using the law of total probability we know that
$$f(y) = \int_{\Theta} f(y|\theta)g(\theta)d\theta.$$

We don't know $g(\theta)$ or $f(y)$, but we have some options. Regarding the latter, we can treat the observed data as fixed then $f(y)$ is constant. In this case it doesn't matter for the maximization problem and we can change the original problem to maximizing

$$h(\theta|y) \propto f(y|\theta)g(\theta)$$

That's progress! What do we do with the $g(\theta)$? We have two real options here

1. Take a guess at $g(\theta)$ using our best substantive knowledge (Bayesian approach)
2. Ignore $g(\theta)$ and only rely on the joint PDF/PMF of the data (maximum likelihood)

The pros and cons of working with a prior distribution are debated *ad nauseum*, and we will not take the time to hash those out, but we will point out that the second approach is in fact a special case of the first where we assume that all values $\theta \in \Theta$ are equally probable absent the model. This is a very specific kind of uninformative prior. In many cases, informative priors are not easy to justify. So, we'll admit our ignorance with respect to prior knowledge and cast aside g. This means we will only focus on the joint distribution of the sample $f(y|\theta)$.

This joint PDF is exactly how we're going to define the likelihood function. Specifically, we'll refer to the joint PDF when we want to discuss the probability of observing values of y given parameters θ, but we will switch the conditionality around to discuss how likely is a specific

parameter value given the data, this gives us

$$\mathcal{L}(\theta|y) = f(y|\theta)$$

.

Ok, we're making progress. Now we just need to be able to describe the joint distribution of the data. To do this let's formalize our first main assumption to make our life a little easier

Assumption A1 (DGP) *The data y_i, $i = 1, \ldots, N$ are independent and identically distributed (iid) realizations from a distribution $f(y|\theta)$*

So with N iid realizations, the joint PDF is…?

$$\mathcal{L}(\theta|y) = \prod_{i=1}^{N} f(y_i|\theta).$$

Suppose that f is the normal PDF with parameters $\theta = (\mu, \sigma^2)$. The maximum likelihood estimates (MLEs) are then defined as

$$\begin{aligned}\hat{\theta} &= \underset{\theta \in \Theta}{\operatorname{argmax}}\, \mathcal{L}(\theta|y) \\ &= \underset{\theta \in \Theta}{\operatorname{argmax}} \prod_{i=1}^{N} \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(\frac{-(y_i - \mu)^2}{2\sigma^2}\right).\end{aligned}$$

We could try to directly maximize this, but more often then not easier for us to work with

the log-likelihood function. To review, the log function has the following properties

$$\begin{aligned}
\log(xy) &= \log(x) + \log(y) \\
\log \prod_{i=1}^{N} x_i &= \sum_{i=1}^{N} \log(x_i) \\
\log(e^x) &= x \\
e^{\log(x)} &= x \\
\log(y^x) &= x \log(y) \\
\log(x/y) &= \log(x) - \log(y) \\
\log(x) &\leq x - 1 \\
\log(x) &= \text{undefined}, x < 0 \\
\log(0) &= -\infty \\
\log(x) &< 0, \ x \in (0, 1) \\
\log(1) &= 0 \\
\log(x) &> 0, \ x > 1 \\
D_x \log(x) &= 1/x > 0 \text{ Strictly increasing for } x > 0 \\
D_x^2 \log(x) &= -1/x^2 < 0 \text{ Concave function}
\end{aligned}$$

Why can we do this? 2 reasons:

1. The likelihood is the product of probabilities/densities, so long as there are no zero-probability events in the data, then the log will be well defined.
2. When looking for maxima, the first order conditions of the log-likelihood are

$$D_\theta \log(\mathcal{L}(\theta|y)) = \frac{1}{\mathcal{L}(\theta|y)} D_\theta \mathcal{L}(\theta|y),$$

by the chain rule. Note that this can only equal 0 when $D_\theta \mathcal{L}(\theta|y) = 0$, so the FOCs are preserved across this transformation. For notational ease, define the FOC as a function

$$s(y|\theta) = \sum_{i=1}^{N} D_\theta \log f(y_i|\theta),$$

where s is called the score function or the gradient function.

So we once again rewrite the problem:

$$\begin{aligned}
\hat{\theta} &= \underset{\theta \in \Theta}{\operatorname{argmax}} \log(\mathcal{L}(\theta|y)) \\
&= \underset{\theta \in \Theta}{\operatorname{argmax}} L(\theta|y) \\
&= \underset{\theta \in \Theta}{\operatorname{argmax}} \sum_{i=1}^{N} -\frac{1}{2}\log(2\pi\sigma^2) - \left(\frac{(y_i-\mu)^2}{2\sigma^2}\right) \\
&= \underset{\theta \in \Theta}{\operatorname{argmax}} \sum_{i=1}^{N} -\frac{1}{2}\log(2\pi) - \frac{1}{2}\log(\sigma^2) - \left(\frac{(y_i-\mu)^2}{2\sigma^2}\right) \\
&= \underset{\theta \in \Theta}{\operatorname{argmax}} -\frac{N}{2}\log(2\pi) - \frac{N}{2}\log(\sigma^2) - \frac{1}{2\sigma^2}\sum_{i=1}^{N}(y_i-\mu)^2.
\end{aligned}$$

Now we just need to find the values μ and σ^2 that solve $s(y|\theta) = 0$.

1.3 Aside: Computer algebra systems

We could do this by hand, but let's take a little time to learn something else useful: computer algebra! Computer algebra comes in many different forms and packages. Mathematica is perhaps the most popular. (Un)fortunately for you, I couldn't afford it when I was younger, so I learned to use sympy instead which is a very nice python package for this sort of thing.

I recommend using Spyder for python work, it's an R Studio-like environment, available from https://www.spyder-ide.org/download. Python syntax is similar to R in many ways, but with a few notable differences, so let's take this slowly

There are two ways to load a package in python. First, you can write `import` [PACKAGE] `as` [X], where [X] acts as prefix to denote functions from that package

```
## Load the package
import sympy as sy


##optional but makes the printed math nicer
sy.init_printing()
```

Alternatively, you can import specific functions using `from` [PACKAGE] `import` and then either list specific functions you want or use `*` to import everything. In this case, we could do

```
## Load the package
from sympy import *
```

```
##optional but makes the printed math nicer
init_printing()
```

In working with sympy, we start by declaring the symbols we're working with. These are any parameter and variables we use. We can declare certain traits about the symbols in this step too.

We will also declare the data y as an `IndexedBase`, meaning that we want to think of y as a vector with indexed elements. We can now translate our likelihood into symbols

```
## We first need to declare what our variables are
m = symbols("mu", real=True)
s = symbols("sigma^2", positive=True)
i, N = symbols("i, N", positive=True, integer=True)


## This will be used to sum over
y = IndexedBase('y')

## Now we can define the likelihood
L = -N*log(2*pi)/2 -  N*log(s)/2 - Sum((y[i]-m)**2, (i,1,N))/ (2*s)


## Take a look
L
```

$$-\frac{N\log\left(\sigma^2\right)}{2} - \frac{N\log\left(2\pi\right)}{2} - \frac{\sum_{i=1}^{N}\left(-\mu + y_i\right)^2}{2\sigma^2}$$

We now want to know the first derivatives. The function `diff` takes a derivative, we can also append it to an existing expression.

```
## First derivatives
FOC_mu = L.diff(m)
FOC_s = L.diff(s)

## Take a look
FOC_mu
```

```
FOC_s
```

$$-\frac{\sum_{i=1}^{N}\left(2\mu-2y_i\right)}{2\sigma^2}$$
$$-\frac{N}{2\sigma^2}+\frac{\sum_{i=1}^{N}\left(-\mu+y_i\right)^2}{2\left(\sigma^2\right)^2}$$

Most of the time, you'd stop here to convert these to code to let R solve these conditions numerically. However, in this case we know there's an analytic solution, so we'll keep going. Unfortunately, the symbolic equation solvers don't always play well when the symbols we want to solve for inside a summation. We can work around this by doing a manual rewrite here.

```
FOC_mu = -N*m/s + Sum(y[i], (i,1,N))/s
mu_hat = solve(FOC_mu, m)
mu_hat
```

$$\left[\frac{\sum_{i=1}^{N}y_i}{N}\right]$$

There are also various simplification commands that could help us here. Warning: sometimes they help, sometimes they don't, sometimes you have to trial and error your way into something that looks better. Some commands you may want to know here:

- `simplify` like it says, it will do its best to make it simple.
- `expand` will expand the expression as much as it can
- `cancel` will try to cancel any common expressions in fractions
- `doit` will evaluate the expression as best it can
- `collect` will try to factor out the terms you list

There are others, but that's a good start.

```
FOC_mu = L.diff(m)
solve(FOC_mu, m)
simplify(FOC_mu)
simplify(FOC_mu).doit()
simplify(FOC_mu).expand()
simplify(FOC_mu).expand().doit()
solve(simplify(FOC_mu).expand().doit(),m)
```

$$[]$$

$$\frac{\sum_{i=1}^{N}(-\mu + y_i)}{\sigma^2}$$

$$\frac{\sum_{i=1}^{N}(-\mu + y_i)}{\sigma^2}$$

$$\frac{\sum_{i=1}^{N} -\mu}{\sigma^2} + \frac{\sum_{i=1}^{N} y_i}{\sigma^2}$$

$$-\frac{N\mu}{\sigma^2} + \frac{\sum_{i=1}^{N} y_i}{\sigma^2}$$

$$\left[\frac{\sum_{i=1}^{N} y_i}{N}\right]$$

```
s2_hat = solve(FOC_s, s)
```

$$\left[\frac{\sum_{i=1}^{N}(\mu^2 - 2\mu y_i + y_i^2)}{N}\right]$$

Ok, so where are we here

$$\hat{\mu}_{MLE} = \frac{1}{N}\sum_{i=1}^{N} y_i$$

$$\hat{\sigma}^2_{\text{MLE}} = \frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{\mu}_{MLE})^2.$$

Final question, how do we know it's a maximum? We would have to check the second derivatives, otherwise known as the Hessian

$$H(y|\theta) = \sum_{i=1}^{N} D_{\theta\theta'} \log f(y_i|\theta).$$

What do we want to find here? Negative (scalar) or negative definite (matrix). In this case, we'll look at the two scalar versions

```
D2_mu = FOC_mu.diff(m)
D2_mu
```

$$-\frac{\sum_{i=1}^{N} 2}{2\sigma^2}$$

Good!

Turning to the variance estimator, let's make our notation a little easier by defining a vector $e = (y_i - \hat{\mu}_{MLE})_{i=1}^N$, then $\hat{\sigma}^2_{MLE} = e'e/N$. From here we get

```
D2_s = FOC_s.diff(s)
e = symbols("e", real=True)
D2_s = D2_s.subs([(s,e*e/N),(m,mu_hat[0])])
D2_s = D2_s.subs([(Sum((y[i]-mu_hat[0])**2, (i,1,N)), e*e)])
D2_s
```

$$-\frac{N^3}{2e^4}$$

Also good!

Presumably you recognize the estimator for μ as the typical sample mean and for σ^2 we get *a* common, but not *the* common sample variance estimator. As a reminder, the typical sample variance estimator is

$$s^2 = \frac{1}{N-1}\sum_{i=1}^{N}(y_i - \hat{\mu})^2,$$

so what's the difference? Well for one thing the MLE is a biased estimator of σ^2. This gives us our first result of the day

Property A1 *The method of maximum likelihood is not unbiased.*

This seems like a bit of a downer, but no worries. We will see that some ML estimators are unbiased, while others are not. We'll see other results later on to make this approach attractive.

For now, however, we can try another example. Suppose that we have $y_1, \ldots, y_N$ forming a random sample from a Bernoulli distribution with parameter $\theta \in [0, 1]$.

1. What values can y_i take on? 0 or 1
2. What is the likelihood function for this problem?

$$\mathcal{L}(\theta|y) = \prod_{i=1}^{N} \theta_i^y (1-\theta)^{1-y_i}$$

3. What is the log-likelihood?

$$L(\theta|y) = \sum_{i=1}^{N} \log(\theta) y_i + \log(1-\theta)(1-y_i)$$

4. What is the MLE of θ? Start with the first order condition:

```
## We first need to declare what our variables are
t = symbols("theta", real=True, positive=True)
N = symbols("N", positive=True, integer=True, real=True)
i = symbols('i', cls=Idx)

## This will be used to sum over
y = IndexedBase('y')

## Now we can define the likelihood
L = log(t)*Sum(y[i], (i,1,N)) +  log(1-t)*Sum(1-y[i], (i,1,N))

FOC = L.diff(t)
##look to make sure we're clear to solve
FOC
```

$$-\frac{\sum_{i=1}^{N}(1-y_i)}{1-\theta}+\frac{\sum_{i=1}^{N}y_i}{\theta}$$

```
t_hat = solve(FOC, t)
t_hat
```

$$\left[\frac{\sum_{i=1}^{N}y_i}{N}\right]$$

The sample mean again!

Let's try one more. Suppose we have some data $y_1, \ldots, y_N$ that we believe to have come from a uniform distribution. Further suppose that we know that the lower bound of this distribution is 0, but we don't know the upper limit so our model is that $y_i \sim U[0, \theta]$. Find the MLE of θ.

1. The PDF here is

$$f(y \mid \theta) = \begin{cases} \frac{1}{\theta} & y \in [0, \theta] \\ 0 & \text{otherwise.} \end{cases}$$

2. The log-likelihood becomes

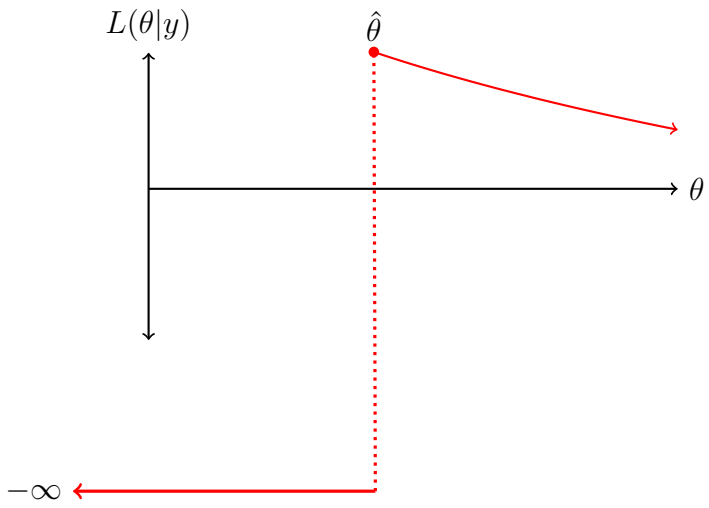
$$L(\theta \mid y) = \sum_{i=1}^{N} \begin{cases} -\log(\theta) & y_i \in [0, \theta] \\ -\infty & \text{otherwise} \end{cases}$$
$$= \begin{cases} -N \log(\theta) & y_i \in [0, \theta], \forall i \\ -\infty & \text{otherwise} \end{cases}$$

3. The first order condition is

$$D_\theta L(\theta|y) = \begin{cases} -N/\theta & y_i \in [0, \theta], \forall i \\ 0 & \text{otherwise} \end{cases}$$

Note this doesn't do us a lot of good. We know that the FOC will be 0 if we pick a θ that makes the log-likelihood $-\infty$, but we also know that that will not be a maximum. For the rest of this function we get something that is strictly decreasing in θ (negative first derivative). So what value maximizes a strictly decreasing function? (The smallest value we can put into it).

Figure 1.1: Log-likelihood for $U[0, \theta]$




```
set.seed(1)
y <- runif(500, max=3)
theta <- seq(1, 5, length=10000)
L <- function(theta,y){
  return(sum(ifelse(y <= theta, -log(theta), -Inf)))
```

```
}
LL <- sapply(theta, L, y=y)
theta[which.max(LL)]
```

```
## [1] 2.988599
```

```
max(y)
```

```
## [1] 2.988232
```

So we want the smallest value of θ, but only to what point? We don't θ to be smaller than any of sample values!

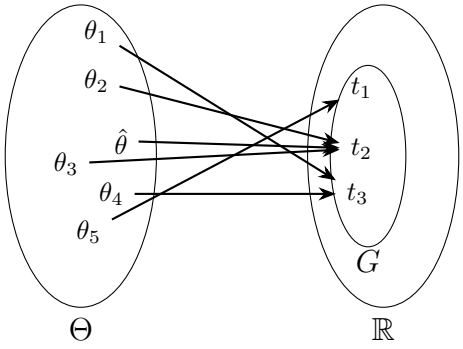
If we pick $\theta < \max(y)$ then $L(\theta \mid y) = -\infty$ (this is called a **zero likelihood problem**, basically it means that we have guessed a parameter value that is inconsistent with the data and model). So smallest value of θ that avoids this problem is $\max(y)$. This is the maximum likelihood estimator. So the MLE of θ is $\max(y)$. Is this biased or unbiased? What do you think happens to this bias as the sample size increases?

1.4 Properties of MLE

Now that we have some experience finding MLEs, let's talk a little bit about why we like them. We'll start with a very nice property called **invariance.**

Property A2 *If $\hat{\theta}$ is an MLE of θ, then $g(\hat{\theta})$ is an MLE of $g(\theta)$, where $g : \Theta \to \mathbb{R}$ is an arbitrary function of the parameters.*

Proof. Let G be the image of Θ under g (i.e., G is the set of values that $g(\theta)$ takes on).



For each element $t \in G$, define the restricted parameter space $G_t = \{\theta : g(\theta) = t\}$. These are

the values of θ that could produce this specific t. From the picture example we get

$$\begin{aligned} G_{t1} &= \{\theta_5\} \\ G_{t2} &= \{\theta_2, \theta_3, \hat{\theta}\} \\ G_{t3} &= \{\theta_4, \theta_1\} \end{aligned}$$

Now we can write the restricted maximization problem in terms of t

$$\begin{aligned} L_R(t) = &\max L(\theta|y) \\ &s.t.\ g(\theta) = t. \end{aligned}$$

Here $L_R(t)$ is a function that takes in t and returns the log-likelihood value that solves this constrained optimization problem. Let $\hat{t}$ be an MLE of $g(\theta)$, this means that

$$L_R(\hat{t}) = \max_{t \in G} L_R(t).$$

We now need to show that $\hat{t} = g(\hat{\theta})$ satisfies this condition. Two facts that are important for this:

1. $L_R(t)$ is the maximum of $L(\theta|y)$ over a subset of Θ
2. $\hat{\theta}$ maximizes $L(\theta|y)$ over all of Θ

Together, these facts mean that $L_R(t) \leq L(\hat{\theta}|y)$ for all $t \in G$. In other words, if you give me a t, I can give you $L_R(t)$, and we know that this value is at most $L(\hat{\theta}|y)$.

Consider $\hat{t} = g(\hat{\theta})$ (t_2 in the picture). This places $\hat{\theta} \in G_{\hat{t}}$. Since $\hat{\theta}$ maximizes L over all of Θ, it must also be the maximum over this subset, and so $L_R(\hat{t}) = L(\hat{\theta}|y)$, which as we previously showed is the maximum value L_R can take on. □

In practice this means that once we estimate the parameters with maximum likelihood we can use those estimates to form other estimates, which will also be maximum likelihood estimates. Some common examples,

1. The MLE of the standard deviation is the square root of the MLE of the variance
2. If we need to estimate a quantity that we know to be strictly positive (constrained optimization), we can estimate log of it (unconstrained optimization) and then un-log it to get the MLE.
3. When we get to fitting nonlinear regression models with MLE, literally every marginal effect or predicted value

Let's return to the Bernoulli example above where we have $y_1, \ldots, y_N$ forming a random sample from a Bernoulli distribution with parameter $\theta \in [0, 1]$. We said that the MLE of θ is the sample mean. What is the MLE of the variance of this distribution?

- Recall that for a Bernoulli, the variance is $\theta(1-\theta)$, and so the MLE of the variance is $\hat{\theta}(1-\hat{\theta})$.

The second property of MLE that we want to show is consistency. We already know that MLEs are not guaranteed to be unbiased (some are, some aren't), but maybe we want to know that they're asymptotically unbiased at least.

As a reminder, we say that an estimator $\hat{\theta}$ is consistent for parameter θ if $\hat{\theta} \xrightarrow{p} \theta^*$. This notation is read as saying that $\hat{\theta}$ **converges in probability** to θ^*, which means that for some $\varepsilon > 0$ and some $\delta > 0$ there is a sample size N such that for all $N' > N$ $\Pr(|\hat{\theta} - \theta^*| > \varepsilon) < \delta$. In words, this says that as N increases, the difference between $\hat{\theta}$ and θ^* is vanishingly small.

We'll need a few additional assumptions here:

Assumption A2 *The parameter space Θ is compact (closed and bounded).*

This is often ignored in practice, but it makes our proofs a little easier. To help you sleep at night, just imagine we set some imaginary large and silly bounds on the parameter space ahead of time that are far away from any reasonable guess at the truth.

To remind ourselves:

Bounded Every point in the set is within a certain distance of all the others. It has upper and lower bounds in each dimension

Closed A closed set is one contains its boundary points, and every sequence/limit of points within the set that converges does so to a point in the set (no holes).

Some examples:

1. The interval $[0, 1]$ is closed and bounded
2. The interval $[0, \infty)$ is closed but not bounded
3. The interval $(0, \infty)$ is not bounded and not closed
4. The interval $(0, 1)$ is bounded, but not closed

Assumption A3 *The density $f(y; \theta)$ is continuously differentiable in θ for all $y \in supp(y)$ and the support of y does not change with θ. The expected value $\mathrm{E}[\log f(y|\theta)]$ exists.*

Property A3 *Under Assumptions A1-A3, an MLE $\hat{\theta}$ exists.*

Assumption A4 (Identification) *If* $\theta_1 \neq \theta_2$*, then* $f(y;\theta_1) \neq f(y;\theta_2)$

Basically, this means that for any two different parameter values, the distributions will be different. Think about the normal distribution, any different value of μ produces a different normal distribution. Basically, this says for any two guesses of θ we can say what makes them different. This assumption can fail in cases where changing θ_1 can be offset by changing θ_2. For example, in ideal point estimation, a common problem is multiplying everyone by -1 will produce identical results, violating this assumption.

Assumption A5 *Additional technical conditions to avoid zero likelihood problems and apply the uniform law of large numbers.*

These are sometimes referred to as regularity conditions for maximum likelihood estimation. Nearly everything we do going forward will satisfy these. However, just because a model doesn't satisfy these conditions, doesn't mean that it's not consistent or anything else, just that we can't use these upcoming results out of the box.

Theorem 1 (Uniform LLN for MLE) *Under Assumptions A1-A5,*

$$\sup_{\theta \in \Theta} \left| \frac{1}{N} \sum_{i=1}^{N} \log f(y_i|\theta) - \mathrm{E}[\log f(y|\theta)] \right| \xrightarrow{p} 0.$$

In case you forgot, the supremum is the smallest possible upper bound of a set. If the supremum is in the set, it is also the maximum. The ULLN is important for us, because it allows us to talk about convergence of a random function rather than just a sequence of random variables.

Property A4 *Under Assumptions A1-A5, maximum likelihood estimates are consistent.*

Before proving this we will remind ourselves of two useful facts. The first is the law of the unconscious statiscian.

Theorem 2 (*The Law of the Unconscious Statistician)* *Let* X *be a random variable with pmf/pdf* f *and* $\mathrm{E}[X]$ *is finite. For any real valued function* g

$$\mathrm{E}[g(X)] = \sum_x g(x) f(x)$$

for discrete X *and*

$$\mathrm{E}[g(X)] = \int_x g(x) f(x) dx$$

for continuous X.

The second is that for positive x, $log(x) \leq x - 1$ and this inequality is strict everywhere except $x = 1$.

Proof. To prove Property A4, we will proceed in three steps. First, we will show that θ^* is a maximum of the population log-likelihood. Second, we will show that θ^* is unique in maximizing the population log-likelihood. Finally, we will show that the sample log-likelihood converges to the population log-likelihood and so the MLE converges to

Step 1 We start by defining the sample and population likelihoods:

$$\begin{aligned} L_N(\theta; y_i) &= \frac{1}{N}\sum_{i=1}^{N} \log f(y_i|\theta) \\ L^*(\theta) &= \mathrm{E}\left[\log f(y|\theta)\right] \\ &= \int_{-\infty}^{\infty} f(y|\theta^*) \log f(y|\theta) dy \quad \text{Law of the unconscious statistican.} \end{aligned}$$

The first thing we want to show is that θ^* is a maximizer of L^*. The FOC here is

$$\begin{aligned} D_\theta L^*(\theta) &= D_\theta \int_{-\infty}^{\infty} f(y_i|\theta^*) \log f(y_i|\theta) dy \\ &= \int_{-\infty}^{\infty} D_\theta f(y_i|\theta^*) \log f(y|\theta) dy \quad \text{Leibniz rule} \\ &= \int_{-\infty}^{\infty} f(y|\theta^*) \frac{D_\theta f(y|\theta)}{f(y|\theta)} dy. \end{aligned}$$

We would like this to be 0 at $\theta = \theta^*$,

$$\int_{-\infty}^{\infty} f(y|\theta^*) \frac{D_\theta f(y|\theta^*)}{f(y|\theta^*)} dy = D_\theta \int_{-\infty}^{\infty} f(y|\theta^*) dy = D_\theta 1 = 0$$

So θ^* is indeed an optimum. We'll skip, but assert, the second order condition for now (maybe it'll be in a problem set or exam).

Step 2 We now want to show that for $\theta \neq \theta^*$,

$$L^*(\theta) < L^*(\theta^*).$$

$$
\begin{aligned}
L^*(\theta) - L^*(\theta^*) &= \mathrm{E}\left[\log f(y|\theta) - \log f(y|\theta^*)\right] \\
&= \mathrm{E}\left[\log \frac{f(y|\theta)}{f(y|\theta^*)}\right] \\
&\leq \mathrm{E}\left[\frac{f(y|\theta)}{f(y|\theta^*)} - 1\right] \\
\mathrm{E}\left[\frac{f(y|\theta)}{f(y|\theta^*)} - 1\right] &= \int \left(\frac{f(y|\theta)}{f(y|\theta^*)} - 1\right) f(y|\theta^*)dy \\
&= \int f(y|\theta) - f(y|\theta^*)dy \\
&= \int f(y|\theta)dy - \int f(y|\theta^*)dy \\
&= 1 - 1 = 0.
\end{aligned}
$$

This establishes that $L^*(\theta) - L^*(\theta^*) \leq 0$, to make that strict, note that by Assumption A1 (identification), $f(\theta) \neq f(\theta^*)$ unless $\theta = \theta^*$

Step 3 We are now ready to show the last part which is that the sample likelihood function converges to the population likelihood. The ULLN does all the work for us here under our assumptions,

$$\sup_{\theta \in \Theta} |L_N(\theta|y) - L^*(\theta)| \xrightarrow{p} 0.$$

This means that as $N \to \infty$, the sample likelihood gets closer to the population likelihood and so the MLE obtained by maximizing it $\hat{\theta}_N$ will closer to the MLE obtained by maximizing the population function, which as we showed about is θ^*.

Another way to see that is:

$$
\begin{aligned}
L^*(\theta^*) - L_N(\hat{\theta}_N) &= \mathrm{E}\left[\log f(y|\theta^*)\right] - \frac{1}{N}\sum_{i=1}^{N} \log f(y_i|\hat{\theta}_N) & \\
\frac{1}{N}\sum_{i=1}^{N} \log f(y_i|\hat{\theta}_N) &\xrightarrow{p} \mathrm{E}\left[\log f(y|\hat{\theta})\right] & \text{ULLN} \\
\mathrm{E}\left[\log f(y|\hat{\theta})\right] &= \mathrm{E}\left[\log f(y|\theta^*)\right] & \text{Previous steps} \\
L^*(\theta^*) - L_N(\hat{\theta}_N) &\xrightarrow{p} \mathrm{E}\left[\log f(y|\theta^*)\right] - \mathrm{E}\left[\log f(y|\theta^*)\right] & \text{Slutsky} \\
&= 0. &
\end{aligned}
$$

□

This is good news. So far we've seen that the finite sample properties are questionable (maybe it's biased, maybe it's not), but we now at least know that this bias goes to 0 as N increases.

Assumption A6 *The true value of θ is interior to Θ.*

Assumption A7 *The density $f(y;\theta)$ is thrice differentiable with respect to θ.*

Assumption A8 *The Fisher information matrix*

$$I(\theta) = \mathrm{E}\left[(D_\theta \log f(y|\theta))(D_\theta \log f(y|\theta))'\right],$$

exists and is finite for all y such that $f(y|\theta) > 0$

These additional assumptions will allow us to characterize the large sample variance and distribution of MLEs. As before, nearly every example you come across will satisfy these conditions, but there is at least one example we looked at before that didn't. What was it? Estimating θ in the $U[0,\theta]$ case. Here, θ was not an interior solution, it was the edge of Θ, in this case we would need to derive the variance and distribution from the estimator itself rather than these results.

We have introduced this new matrix $I(\theta)$, which we're calling the Fisher information; what is it? In short, $I(\theta)$ tells us, on average, how informative each piece of data is at estimating θ. The more informative a sample is, the smaller the variance should be. That is, as $I(\theta)$ increases, the variance of the estimator will decrease. Notice that the definition is just the variance of the first order conditions, so the more variance we have in the first derivatives, the less certain we are that we're at the "true" maximum, the less informative the data are. Additionally, note that in an iid sample of size N, the total information we have is thus $NI(\theta)$. We can rewrite the information as a function of second derivatives (Hessian), to make that more clear.

Result 1 (Information matrix equality) *Under assumptions A3-A8, the Fisher information $I(\theta)$ can also be written as*

$$I(\theta) = -\mathrm{E}[D_{\theta\theta'} \log f(y|\theta)].$$

Intuitively, this means that the Fisher information is reflected in the curvature of the log-likelihood. The flatter it is, the less information we have. The more pointy it is the more certain we are that we got a good answer.

We can be a little more precise about this relationship between variance and information. Consider a random sample $y_1, \ldots, y_N$ from a distribution $f(y|\theta)$ and an arbitrary estimator $\hat{\theta}_A$ with $\mathrm{Var}(\hat{\theta}_A) < \infty$.

To be clear, we're looking to say something about $\text{Var}(\hat{\theta}_A)$, and we'll rely on the **Cauchy-Schwarz inequality** to say it. The inequality is

$$\text{Var}(\hat{\theta}_A) \geq \text{Cov}(s(\theta|y), \hat{\theta}_A) \, \text{Var}(s(\theta|y))^{-1} \, \text{Cov}(\hat{\theta}_A, s(\theta|y)).$$

So these we need to describe the two right-hand terms. The variance is the easier one. As we saw above, it's the variance of the gradients which is just the total Fisher information in the sample:

$$\text{Var}(s(\theta|y)) = NI(\theta).$$

The other term

$$\begin{aligned}
\text{Cov}(s(\theta|y), \hat{\theta}_A) &= \text{E}[s(\theta|y)\hat{\theta}'_A] - \text{E}[s(\theta|y)] \, \text{E}[\hat{\theta}_A]' \\
&= \text{E}[s(\theta|y)\hat{\theta}'_A] \\
&= \int_y \dots \int_y f(y|\theta) s(\theta|y) \hat{\theta}'_A dy_1 \dots dy_N \quad \text{Law of US} \\
&= \int_y \dots \int_y D_\theta f(y|\theta) \hat{\theta}'_A dy_1 \dots dy_N.
\end{aligned}$$

Note that also by Law of the Unconscious Statistician we get

$$\text{E}[\hat{\theta}_A] = \int_y \dots \int_y f(y|\theta) \hat{\theta}_A dy_1 \dots dy_N.$$

Taking the derivative of this with respect to θ we get

$$D_\theta \, \text{E}[\hat{\theta}_A] = \int_y \dots \int_y D_\theta f(y|\theta) \hat{\theta}_A dy_1 \dots dy_N = \text{Cov}(s(\theta|y), \hat{\theta}_A).$$

So now we can go back to the inequality to get

Result 2 (Cramér-Rao lower bound or the information inequality) *Under assumptions, A6-A8, the lower bound on the variance of an estimator is given as:*

$$\text{Var}(\hat{\theta}_A) \geq \left(D_\theta \, \text{E}[\hat{\theta}_A] \right) [NI(\theta)]^{-1} \left(D_\theta \, \text{E}[\hat{\theta}'_A] \right).$$

This result tells us that the lowest variance we can get for any estimator of θ, under out regularity conditions, is a function of the estimator's expected value, the sample size, and the Fisher information. Now suppose that the estimator is unbiased, then the above simplifies, with

$$D_\theta \, \text{E}[\hat{\theta}_A] = D_\theta \theta = I,$$

and then

$$\text{Var}(\hat{\theta}_A) \geq [NI(\theta)]^{-1}.$$

So under our regularity conditions, the smallest variance we can get for an unbiased estimator is the inverse of the Fisher information for the whole sample. Estimators that obtain the CR lower bound given a fixed expected value and sample size are the most efficient estimators (i.e., they use all the available Fisher information).

Before we move into the next result, we need to remind ourselves about Taylor series expansions and the mean value theorem. For ease, I'll present them in scalar notation, but they extend to the matrix cases we'll use.

Theorem 3 (Mean value theorem) *Let $f : [x_1, x_2] \to \mathbb{R}$ be a continuous, differentiable function, then there exists a point $\bar{x} \in (x_1, x_2)$ such that*

$$D_x f(\bar{x}) = \frac{f(x_2) - f(x_1)}{x_2 - x_1}$$

and a version of Taylor's theorem

Theorem 4 (Taylor's theorem) *Let $k \in \mathbb{Z}$ and let $f : \mathbb{R} \to \mathbb{R}$ be $k+1$-differentiable at some point $a \in \mathbb{R}$. Then*

$$f(x) = \sum_{j=0}^{k} \frac{D_x^j f(a)}{j!}(x-a)^j + R_{k+1}(x).$$

We group these together because they are intimately related. To see this, let $x_1 = a$ and $x_2 = x$ and rewrite the mean value theorem to be

$$\begin{aligned} D_x f(\bar{x}) &= \frac{f(x) - f(a)}{x - a} \\ f(x) &= f(a) + D_x f(\bar{x})(x-a) \end{aligned}$$

which is a Taylor expansion with $k = 0$ and $R_0(x) = Df(\bar{x})(x-a)$. It turns out we can generalized this combo by letting

$$R_{k+1}(x; \bar{x}) = \frac{D^{k+1} f(\bar{x})}{(k+1)!}(x-a)^{k+1}.$$

The main thing to note here is that we don't know, and often don't need to know what $\bar{x}$ is. Just that is exists, which is to say there will be such a value between x and a.

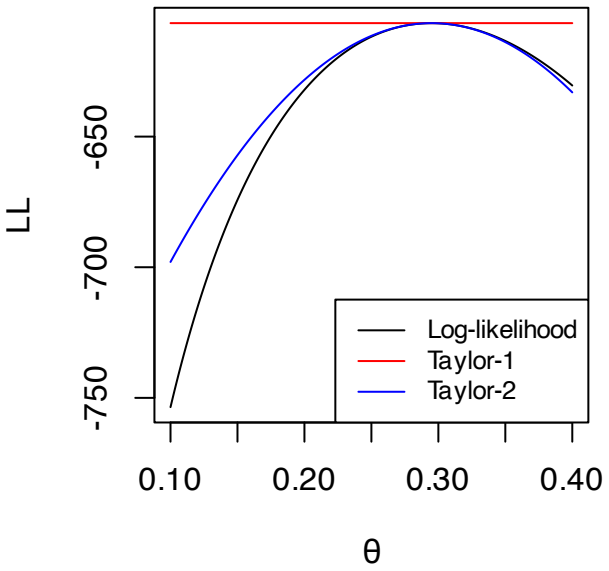
For our purposes, we will often be working with the log-likelihood function and setting x to be true but unknown value, a to be the MLE, and $\bar{x}$ to just be something between these two. In this case, as a converges to x by the consistency of MLE, $\bar{x}$ also converges to x faster, making the remainder convergence to 0. Let's illustrate these results with the binomial likelihood:

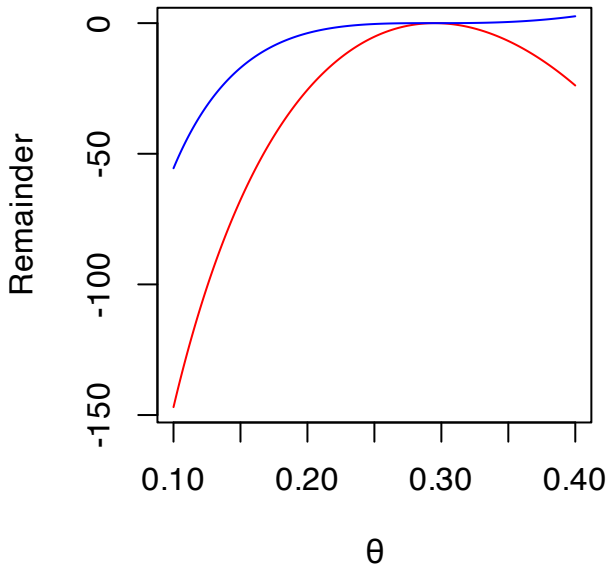
```
set.seed(123)
theta <- .3 # Truth
N <- 1000 # Sample size

## Data
y <- rbinom(N, size = 1, prob=theta)

## functions
L <- function(theta, y){ #log likelihood
  return(sum(dbinom(y, size=1,prob=theta, log=TRUE)))
}
s <- function(theta, y){ #score
  return(sum((y-theta)/(theta*(1-theta))))
}
H <- function(theta, y){ #hessian
  top <- -theta^2+2*theta*y-y
  bottom <- theta^2*(theta^2-2*theta+1)
  return(sum(top/bottom))
}
DH <- function(theta, y){ #third deriv
  top <- 2*(theta^3 - 3*theta^2*y + 3*theta*y - y)
  bottom <- theta^3*(theta^3 - 3*theta^2 + 3*theta - 1)
  return(sum(top/bottom))
}
T1 <- function(theta, y,a){ #first order Taylor
  return(L(a, y)+ (theta-a)*s(a,y))
}
T2 <- function(theta, y,a){ #second order Taylor
  return(T1(theta, y,a) + ((theta-a)^2 * H(a,y))/2)
}

theta.hat <- mean(y) #MLE of theta
```

```
theta.hat
```

```
## [1] 0.295
```

```
Theta <- seq(0.1,.4,length=5000) #Range of theta's
Lout <- sapply(Theta, L, y=y)
T1.out <- sapply(Theta, T1, y=y, a=theta.hat)
T2.out <- sapply(Theta, T2, y=y, a=theta.hat)

par(mfrow=c(1,2))
plot(Lout~Theta, type='l',
     xlab=expression(theta),
     ylab="LL")
lines(T1.out~Theta, col='red')
lines(T2.out~Theta, col='blue')
legend("bottomright",
       lty=1,
       c("Log-likelihood",
         "Taylor-1",
         "Taylor-2"),
       cex=.8,
       col=c("black","red", "blue"))
R1 <- Lout -T1.out
R2 <- Lout -T2.out
plot(R1~Theta, type='l',
     ylab="Remainder",
     col='red', xlab=expression(theta))
lines(R2~Theta, col='blue')
```






```
## Why is the first order approx flat? Because s(theta; y)=0


## Strong improvement from first order to second order

## Let's look at true theta

## polynomial approx
c(L(theta,y),
  T1(theta, y, theta.hat),
  T2(theta, y, theta.hat))
```

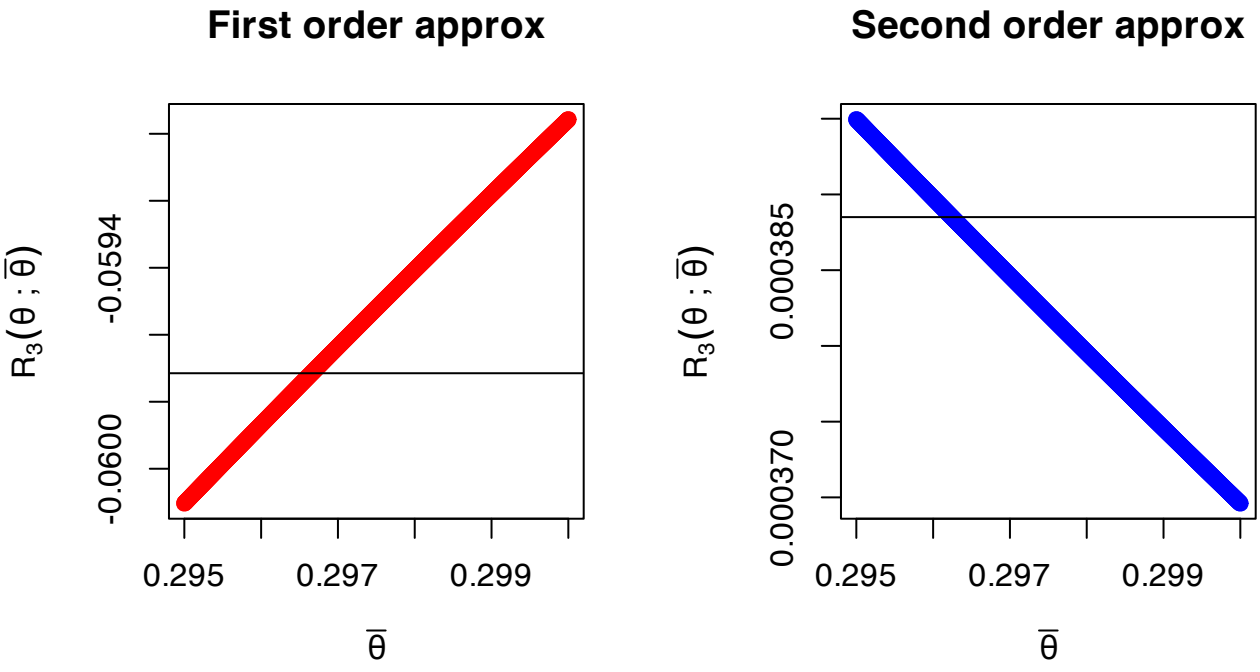
```
## [1] -606.6278 -606.5681 -606.6282
```

```
## remainder at this point
c(L(theta,y)- T1(theta, y, theta.hat),
  L(theta,y)-T2(theta, y, theta.hat))
```

```
## [1] -0.0597148737  0.0003885041
```

```
## Illustrating the mean value form of the remainder

## Set the range of bar{theta}: [theta,hat(theta)]
Theta.bar <- seq(theta, theta.hat, length=1000)
par(mfrow=c(1,2),mar=c(5, 4.5, 4, 2) + 0.1)
plot(sapply(Theta.bar,
```

```
            \(x){((theta-theta.hat)^2 * H(x,y))/2})~Theta.bar,
     ylab=expression(R[3](theta~";"~bar(theta))),
     xlab=expression(bar(theta)),
     main="First order approx",
     col="red")
abline(h=L(theta,y)- T1(theta, y, theta.hat))


plot(sapply(Theta.bar,
            \(x){((theta-theta.hat)^3 * DH(x, y=y))/6})~Theta.bar,
     ylab=expression(R[3](theta~";"~bar(theta))),
     xlab=expression(bar(theta)),
     main="Second order approx",
     col="blue")
abline(h=L(theta,y)- T2(theta, y, theta.hat))
```



We are now ready to present the next property of MLEs,

Property A5 *Under Assumptions A1-A8, the MLE* $\hat{\theta_N}$ *has an asymptotic distribution*

$$\sqrt{N}(\hat{\theta}_N - \theta^*) \xrightarrow{d} N(0, I(\theta^*)^{-1})$$

Proof. We begin with a mean-value characterization of $s(\hat{\theta}_N|y)$ around the true value θ^*, with $\bar{\theta}$ between $\hat{\theta}_N$ and θ^*. Note that because $\bar{\theta}$ is between these two terms, it will converge to θ^* as N increases

$$
\begin{aligned}
s(\hat{\theta}_N|y) &= s(\theta^*|y) + H(\bar{\theta}|y)(\hat{\theta}_N - \theta^*) && \\
0 &= s(\theta^*|y) + H(\bar{\theta}|y)(\hat{\theta}_N - \theta^*) && \text{FOC} \\
(\hat{\theta}_N - \theta^*) &= \left[-H(\bar{\theta}|y)\right]^{-1} s(\theta^*|y) && \text{Rearrange} \\
\sqrt{N}(\hat{\theta}_N - \theta^*) &= \left[-\frac{1}{N}H(\bar{\theta}|y)\right]^{-1} \sqrt{N}\frac{1}{N}s(\theta^*|y) && \\
\frac{1}{N}H(\bar{\theta}|y) &\xrightarrow{p} I(\theta^*) && \text{LLN} \\
\frac{1}{N}s(\theta^*|y) &\xrightarrow{p} 0 && \text{LLN} \\
\sqrt{N}\left(\frac{1}{N}s(\theta^*|y)\right) &\xrightarrow{d} N\left(0, I(\theta^*)\right) && \text{CLT} \\
\sqrt{N}(\hat{\theta}_N - \theta^*) &\xrightarrow{d} N\left(0, I(\theta^*)^{-1}I(\theta^*)I(\theta^*)^{-1}\right) && \text{Slutsky/CMT} \\
\sqrt{N}(\hat{\theta}_N - \theta^*) &\xrightarrow{d} N\left(0, I(\theta^*)^{-1}\right). &&
\end{aligned}
$$

Which is what we wanted to show. □

Immediately from this we get our next property:

Property A6 *Under Assumptions A1-A8, the MLE $\hat{\theta}$ is asymptotically efficient compared to any consistent estimator of θ.*

This follows from the fact that the MLE

1. Is consistent (asymptotically, unbiased)
2. Has an asymptotic variance $\frac{1}{N}I(\theta)^{-1}$, which is the CR lower-bound for an unbiased estimator.

So now we have that among all consistent estimators, none can be more efficient than the MLE. This is one of the main justifications for using MLE.

To estimate the variance under these assumptions, we'll typically just evaluate the information

matrix at the MLE in one of three ways

$$\begin{aligned}
\widehat{\text{avar}}(\hat{\theta}) &= \frac{1}{N} I(\hat{\theta})^{-1} \\
\widehat{\text{avar}}(\hat{\theta})_E &= -\frac{1}{N} \text{E}[D_{\theta\theta'} \log f(y|\hat{\theta})]^{-1} && \text{Exp of hessian at MLE} \\
\widehat{\text{avar}}(\hat{\theta})_H &= -H(\hat{\theta}|y)^{-1} = -\left[\sum_{i=1}^{N} D_{\theta\theta'} \log f(y_i|\hat{\theta})\right]^{-1} && \text{Inv. of minus Hessian} \\
\widehat{\text{avar}}(\hat{\theta})_{OPG} &= \left[\sum_{i=1}^{N} \left[D_\theta \log f(y_i|\hat{\theta})\right] \left[D_\theta \log f(y_i|\hat{\theta})\right]'\right]^{-1} && \text{Outer product of gradients}
\end{aligned}$$

The first is the one you'll probably use the least, it involves figuring out an analytic expression of the expected value of second derivative and then plugging in the MLE. The second is a direct substitute for the first: just used the actual second derivatives (not their expectation) at the MLE. The third is sometimes the easiest as you only need the first derivaties. Under our assumptions, these three different estimators are identical asymptotically. In-sample they may be different, but should be very similar. It shouldn't matter which one you pick.

One computational trick for the OPG estimator is to first define the vector-value log-likelihood

$$L_v(\theta|y) = (\log f(y_i|\theta))_{i=1}^N$$

this takes in the length-k vector θ and returns the $N \times 1$ vector of log-likelihood of each individual observation. The derivative of a vector-valued function is called its Jacobian, which is an $N \times k$ matrix

$$J(\theta|y) := D_\theta L_v(\theta|y) = \begin{bmatrix} D_{\theta_1} \log f(y_1|\theta) & D_{\theta_2} \log f(y_1|\theta) & \dots D_{\theta_k} \log f(y_1|\theta) \\ D_{\theta_1} \log f(y_2|\theta) & D_{\theta_2} \log f(y_2|\theta) & \dots D_{\theta_k} \log f(y_2|\theta) \\ \vdots & \vdots & vdots \\ D_{\theta_1} \log f(y_N|\theta) & D_{\theta_2} \log f(y_N|\theta) & \dots D_{\theta_k} \log f(y_N|\theta) \end{bmatrix},$$

then the OPG estimator becomes

$$\widehat{\text{avar}}(\hat{\theta})_{OPG} = J(\theta|y)' J(\theta|y).$$

1.4.1 Hypothesis testing

We can now start thinking about hypothesis testing. We'll start with the idea of comparing two nested models $f(y|\theta)$ and $f(y|\theta')$ using the data y. By nested we mean that we can write θ' as some function of the full parameter vector θ, e.g., $h(\theta) = \theta'$. Our intuition here will be

to consider the unrestricted log-likelihood that gives us the MLE $\hat{\theta}$ and ask ourselves: Is the maximum log-likelihood that we get if the null hypothesis is true different enough from the full/unrestricted model that we prefer the full model?

Let $c(\theta) := h(\theta) - \theta$ then the hypotheses are

$$H_0 : c(\theta) = 0$$
$$H_A : c(\theta) \neq 0.$$

For illustration suppose the log-likelihood function looks like Figure 1.2. Here the red line denotes the null hypothesis restriction. So if the null is true, we only look at the case where it crosses zero.

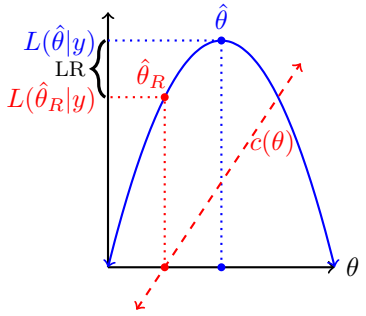


Figure 1.2: Visualizing the likelihood ratio (LR) test

For illustration suppose we have a normal model where we estimate the mean and variance. Then we have the following:

$$\begin{aligned}
H_0 :&\mu = 0.5 \\
H_A :&\mu \neq 0.5 \\
c(\theta) &= \mu - 0.5 = 0 \\
\theta &= (\mu, \sigma^2) \\
L(\theta|y) &= \sum_{i=1}^{N} \log(\phi(y_i|\theta)) = -\frac{N\log(\sigma^2)}{2} - \frac{N\log(2\pi)}{2} - \frac{\sum_{i=1}^{N}(y_i - \mu)^2}{2\sigma^2} \\
\hat{\mu} &= \bar{y} \\
\widehat{\sigma^2} &= \frac{1}{N}(y_i - \hat{\mu})^2 \\
\widehat{\sigma^2}_R &= \frac{1}{N}(y_i - 0.5)^2 \\
\hat{\theta} &= (\hat{\mu}, \widehat{\sigma^2}) \\
\hat{\theta}_R &= (0.5, \widehat{\sigma^2}_R)
\end{aligned}$$

We know that if $\hat{\theta}_R \neq \hat{\theta}$, then $L(\hat{\theta}_R|y) < L(\hat{\theta}|y)$ by our identification assumption and the definition of the MLE. The question we're asking of the data is: Is $L(\hat{\theta}_R|y)$ notably worse than $L(\hat{\theta}|y)$? To put this another way, what can we say about the gap between the two? We know that $L(\hat{\theta}|y)$ is always larger so the gap is weakly positive. What else can we discover about it?

Let's start by considering a Taylor series expansion of the likelihood at the true value and assuming that the null hypothesis is correct (i.e., the restrictions imposed by θ_R are correct) and around the unrestricted estimates $\hat{\theta}$. We assumed that L is three times differentiable (Assumption A7) so we can get a second order expansion:

$$L(\theta_R|y) = L(\hat{\theta}|y) + s(\hat{\theta}|y)'(\theta_R - \hat{\theta}) + \frac{1}{2}(\theta_R - \hat{\theta})'[H(\hat{\theta}|y)](\theta_R - \hat{\theta}) + R(\theta_R; \bar{\theta}).$$

Here $R(\theta_R; \bar{\theta})$ is the remainder term with $\bar{\theta}$ in some value between θ_R and $\hat{\theta}$.

We can rearrange terms

$$\begin{aligned} 2(L(\hat{\theta}|y) - L(\theta_R|y)) &= (\theta_R - \hat{\theta})' \left[-H(\hat{\theta}|y)\right] (\theta_R - \hat{\theta}) - 2R(\theta_R; \bar{\theta}), \\ &= \sqrt{N}(\theta_R - \hat{\theta})' \left[-\frac{1}{N}H(\hat{\theta}|y)\right] \sqrt{N}(\theta_R - \hat{\theta}) - 2R(\theta_R; \bar{\theta}), \end{aligned}$$

and consider some asymptotics

$$\begin{aligned} R(\theta_R; \bar{\theta}) &\xrightarrow{p} 0 \\ -\frac{1}{N}H(\hat{\theta}|y) &\xrightarrow{p} I(\theta_R) \\ \sqrt{N}(\theta_R - \hat{\theta}) &\xrightarrow{d} N\left(0, I(\theta_R)^{-1}\right) \\ (\sqrt{N}(\theta_R - \hat{\theta}))' \left[-\frac{1}{N}H(\hat{\theta}|y)\right]^{1/2} &\xrightarrow{d} N(0,1) \text{ Matrix version of a } z \text{ transformation} \\ (\sqrt{N}(\theta_R - \hat{\theta}))' \left[-\frac{1}{N}H(\hat{\theta}|y)\right] (\sqrt{N}(\theta_R - \hat{\theta})) &\xrightarrow{d} \chi^2_J \\ 2(L(\hat{\theta}|y) - L(\theta_R|y)) &\xrightarrow{d} \chi^2_J, \end{aligned}$$

where J is the number of elements in θ_R that are not equal to $\hat{\theta}$. In other words, J is the number of restrictions implied by the hypothesis. In most cases we can count up the number of equal signs in the null hypothesis and that will be J.

Property A7 *Under Assumptions A1-A8, the null hypothesis $H_0 : c(\theta) = 0$ implies an LR statistic*

$$2(L(\hat{\theta}|y) - L(\hat{\theta}_R|y)) \xrightarrow{d} \chi^2_J.$$

where $\hat{\theta}_R$ is found by maximizing the restricted likelihood (if necessary)

$$\hat{\theta}_R = \underset{\theta}{\operatorname{argmax}}\, L(\theta|y)$$
$$s.t.\ c(\theta) = 0.$$

Note that we only need to refit the model if the hypothesis involves fixing some parameters but not others. In most regression contexts this does involve fitting a second model. For example, suppose we have

$$\mathrm{E}[y_i|X] = \beta_0 + x_{i1}\beta_1 + x_{i2}\beta_2.$$

An omnibus test would be test the null that $\beta_1 = \beta_2 = 0$. In this case, the LR test would compare fitting the full model against one where

$$\mathrm{E}[y_i|X] = \beta_0,$$

which we would need to fit to find β_0.

The LR test has a nice intuition, it can handle any linear or non-linear functions of the parameters, and it doesn't even require us to estimate the variance/covariance of the parameters we're considering. The downside is that it can require fitting two different models. Fitting a second model may be trivial or it could be costly. So what can we do with just a single model? Well we know that MLEs are asymptotically normal, which means that functions of MLEs are also asymptotically normal (because they are themselves MLEs). As a result, all the familiar asymptotically justified hypothesis tests you used in 602 are on the table.

Let's start with the more common linear hypotheses. A linear hypothesis is one that can be written as one or more linaer equations. These include basic single parameter hypotheses like $\beta_k = b$, but also fancier ones like $\beta_k + \beta_\ell = b$, $\beta_k = \beta_\ell$, or
$\beta_k = 0$ & $\beta_\ell = 1$. Hypotheses like these are implied by many common research questions (e.g., are two or more effects identical? Do some effects cancel out? Are a set of race dummies jointly significant?) and we'll encounter some of these in practice going forward.

What's notable about hypotheses tests like these as that they can all be written as a set of linear equations

$$A\beta = b.$$

As an example, suppose we still have

$$\mathrm{E}[y_i|X] = \beta_0 + x_{i1}\beta_1 + x_{i2}\beta_2.$$

and consider the hypothesis that $\beta_1 = \beta_2$. Here $A = [0, 1, -1]$ and $b = 0$, to see this, just muliply it out

$$\begin{aligned} A\beta &= b \\ 0 \times \beta_0 + 1 \times \beta_1 - 1 \times \beta_2 &= 0 \\ \beta_1 - \beta_2 &= 0. \end{aligned}$$

Under a different null where $\beta_0 = 0$ and $\beta_1 = 1$, we have $A = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \end{bmatrix}$ and $b = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$. Let J be the number of rows in A and b, this number matches the number of restrictions (equal signs) in the null hypothesis.

Then because we know that MLEs are asymptotically normal, we can test linear hypotheses based on a Wald test

Property A8 *Under Assumptions A1-A8, the null hypothesis $H_0 : A\theta = b$ implies a Wald test*

$$(A\hat{\theta} - b)' \left[A\widehat{\text{avar}}(\hat{\theta})A'\right]^{-1} (A\hat{\theta} - b) \xrightarrow{d} \chi^2_J.$$

Note that if $J = 1$, we have the simpler z test where

$$\frac{\hat{\theta}_j - b}{\sqrt{\widehat{\text{avar}}(\hat{\theta}_j)}} \xrightarrow{d} N(0, 1).$$

Not all interesting tests are linear however. Sometimes we may have a length J nonlinear hypothesis or transformation in the form of a continuously differentiable function $c(\theta) = 0$. Returning to the regression example this could be something like $-\beta_1/(2\beta_2) = b$ or $\exp(\theta) = 1$.

In order to make progress here, we will need to know the distribution of $c(\hat{\theta})$. This is a question about a distribution of a nonlinear function of a normally distributed random variable.

Due to invariance we know that $c(\hat{\theta})$ is itself an MLE and as such is asymptotically normal distributed due to the **delta method**, which we can state

Theorem 5 (The Delta Method) *Let $\hat{\theta}$ be a random variable where*

$$\hat{\theta}_N \xrightarrow{p} \theta \sqrt{N}(\hat{\theta} - \theta) \quad \xrightarrow{d} N(0, \Sigma)$$

, and let c be a continuous and differentiable function. Then

$$\sqrt{N}(c(\hat{\theta}) - c(\theta)) \xrightarrow{d} N\left(0, [D_\theta c(\theta)] I(\theta)^{-1} [D_\theta c(\theta)]'\right).$$

The proof follows from another combination of Taylor and the mean-value theorem.

Notably, because MLEs satisfy the conditions of the delta method, then we can use this expression to find the variance of transformed parameters. Note that if $c(\hat{\theta})$ is a singleton then we can construct a z test based on just this expression. So if the θ is the only parameter and we want to know if $\exp(\theta) = 1$ then we could have a z test with

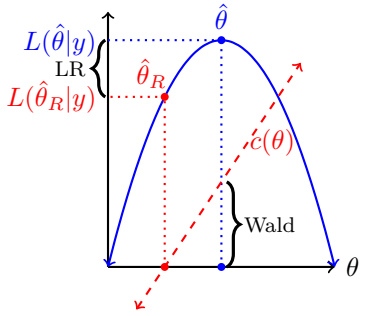
$$z = \frac{\exp(\hat{\theta}) - 1}{\sqrt{(D_{\hat{\theta}} \exp(\hat{\theta}))^2 \operatorname{Var}(\hat{\theta})}} = \frac{\exp(\hat{\theta}) - 1}{\sqrt{\exp(\hat{\theta})^2 \operatorname{Var}(\hat{\theta})}} = \frac{\exp(\hat{\theta}) - 1}{\exp(\hat{\theta}) \operatorname{se}(\hat{\theta})} \overset{a}{\sim} N(0, 1)$$

When $c(\hat{\theta})$ is vector-valued, we can apply the logic from the linear hypothesis test we can get to a usable test statistic for the hypothesis that $c(\theta) = 0$ and

Property A9 *Under Assumptions A1-A8, the null hypothesis $H_0 : c(\theta) = 0$ implies a Wald statistic*

$$c(\hat{\theta})' \left[D_\theta c(\hat{\theta}) \widehat{\operatorname{avar}}(\hat{\theta}) D_\theta c(\hat{\theta})' \right]^{-1} c(\hat{\theta}) \xrightarrow{d} \chi^2_J.$$

Going back to our picture, we can see how Wald tests differs from the LR test.



Let's consider an example with 10 iid observations from a $N(1, 1)$. In this case, suppose we want to test the hypothesis that $\mu = 0.5$. We can do this with either the LR or Wald test. Recall that the MLE for μ is the sample mean and the MLE for σ^2 is $N^{-1} \sum (y_i - \hat{\mu})^2$. For the LR test we will compare the unrestricted likelihood evaluated at the MLEs

$$L(\theta|y) = \sum_{i=1}^{N} \log f(y_i|\theta),$$

to the likelihood evaluated at the θ_R. Here θ_R is?

$$\hat{\mu}_R = 0.5$$
$$\hat{\sigma}^2_R = \frac{1}{N}\sum_{i=1}^{N}(y_i - 0.5)^2.$$

```
set.seed(1)
y <- rnorm(10, 1,1)
mu.mle <- mean(y)
s2.mle <- mean((y-mu.mle)^2)
s2.mle.R <- mean((y-0.5)^2)

## LR test
L.unrestricted <- sum(dnorm(y, mu.mle, sqrt(s2.mle), log=TRUE)) #at hat(theta)
L.unrestricted
```

```
## [1] -11.18548
```

```
L.restricted <- sum(dnorm(y, 0.5, sqrt(s2.mle.R), log=TRUE)) #at hat(theta)_R
L.restricted
```

```
## [1] -13.92272
```

```
LR.test <- 2*(L.unrestricted-L.restricted)
LR.test
```

```
## [1] 5.474474
```

```
pchisq(LR.test, lower=FALSE, df=1)
```

```
## [1] 0.01929617
```

```
## Wald test: need a covariance matrix

## vcov 1: (-E(H(\theta|y)))^{-1}
E.hessian <- matrix(c(-10/s2.mle, 0,0,-10/(2*s2.mle^2)), 2,2)
V1 <- solve(-E.hessian)

## vcov 2: (-(H(\hat\theta|y)))^{-1}

Obs.hessian <- matrix(c(-10/s2.mle, (10*mu.mle-sum(y))/(s2.mle^2),
```

```
                        (10*mu.mle-sum(y))/(s2.mle^2), -10/(2*s2.mle^2)), 2,2)
V2 <- solve(-Obs.hessian)

## Vcov 3: OGP
Jac <- cbind((y-mu.mle)/s2.mle,
             -1/(2*s2.mle)+(y-mu.mle)^2 / (2*s2.mle^2))
## In Jac, the rows are score at each observation.
OPG <- t(Jac) %*% Jac
V3 <- solve(OPG)



list(V1, V2, V3)
```

```
## [[1]]
##           [,1]       [,2]
## [1,] 0.0548383 0.00000000
## [2,] 0.0000000 0.06014478
##
## [[2]]
##           [,1]       [,2]
## [1,] 0.0548383 0.00000000
## [2,] 0.0000000 0.06014478
##
## [[3]]
##              [,1]        [,2]
## [1,]  0.05888948 -0.02025712
## [2,] -0.02025712  0.10129166
```

```
wald.stat <- (mu.mle -0.5)/sqrt(V1[1,1])
wald.stat
```

```
## [1] 2.699693
```

```
pnorm(abs(wald.stat), lower=FALSE)*2
```

```
## [1] 0.006940344
```

What if the hypothesis was $H_0 : \mu = 0$ & $\sigma^2 = 1$?

```r
## LR test
L.unrestricted <- sum(dnorm(y, mu.mle, sqrt(s2.mle), log=TRUE)) #at hat(theta)
L.unrestricted
```

```
## [1] -11.18548
```

```r
L.restricted <- sum(dnorm(y, 0, 1, log=TRUE)) #at hat(theta)_R; no refit required
L.restricted
```

```
## [1] -18.34072
```

```r
LR.test <- 2*(L.unrestricted-L.restricted)
LR.test
```

```
## [1] 14.31047
```

```r
pchisq(LR.test, lower=FALSE, df=2)
```

```
## [1] 0.0007807642
```

```r
## Wald setup
theta.hat <- c(mu.mle, s2.mle)
A <- diag(2)
b <- c(0,1)

## with the Observed Hessian
wald <- t(A%*% theta.hat -b) %*% solve(A %*% V2 %*% A) %*% (A%*% theta.hat -b)
wald
```

```
##          [,1]
## [1,] 26.76681
```

```r
pchisq(wald, lower=FALSE, df=2)
```

```
##             [,1]
## [1,] 1.5405e-06
```

```r
## With the OPG
wald <- t(A%*% theta.hat -b) %*% solve(A %*% V3 %*% A) %*% (A%*% theta.hat -b)
wald
```

```
##          [,1]
## [1,] 21.80857
```

```r
pchisq(wald, lower=FALSE, df=2)
```

```
##              [,1]
## [1,] 1.837934e-05
```

Finally for giggles let's consider the following non-linear hypothesis

$$H_0 : \exp(\mu + \sigma^2/2) = 1$$

against the alternative that it doesn't. In this case our c function takes two inputs and returns a single output

$$c(\theta) = \exp(\mu + \sigma^2/2) - 1.$$

For an LR test, we would could either fit the model with the constraint or solve for one parameter and go from there. In this case, we can easily solve for $\sigma^2 = -2\mu$ and then we have

```r
from sympy import *
init_printing()

## We first need to declare what our variables are
m = symbols("mu", real=True)
s = symbols("sigma^2", positive=True)
i, N = symbols("i, N", positive=True, integer=True)


## This will be used to sum over
y = IndexedBase('y')

## Now we can define the likelihood
L = -N*log(2*pi)/2 -  N*log(s)/2 - Sum((y[i]-m)**2, (i,1,N))/ (2*s)


## Impose the null hypothesis
L = L.subs(s, -2*m)

solve(L.subs(s, -2*m).diff(m).expand().doit().simplify(),m)
```

There are two solutions here, in order to keep the variance positive, we need a negative mean under this null hypothesis.

```r
## restricted estimates (assume the null is true)
mu.R <- 1-sqrt(10 + sum(y^2))/sqrt(10)
s2.R <- -2*mu.R

L.unrestricted <- sum(dnorm(y, mu.mle, sqrt(s2.mle), log=TRUE)) #at hat(theta)
L.unrestricted
```

```
## [1] -11.18548
```

```r
L.restricted <- sum(dnorm(y, mu.R, s2.R, log=TRUE))
L.restricted
```

```
## [1] -22.61068
```

```r
LR.test <- 2*(L.unrestricted-L.restricted)
LR.test
```

```
## [1] 22.85041
```

```r
pchisq(LR.test, lower=FALSE, df=1)
```

```
## [1] 1.751124e-06
```

That's a bit of a hassle. The Wald version is much easier. We just need the derivative of c with respect to θ

$$D_\theta c(\theta) = (\exp(\mu + \sigma^2/2), \exp(\mu + \sigma^2/2)/2).$$

```r
### Much easier Wald test here
c.theta <- exp(mu.mle + s2.mle/2)-1
Dtheta <- c(exp(mu.mle + s2.mle/2),  exp(mu.mle + s2.mle/2)/2)
Wald.stat <- c.theta %*% solve(Dtheta %*% V2 %*% Dtheta) %*% c.theta
pchisq(Wald.stat, lower=FALSE, df=1)
```

```
##              [,1]
## [1,] 0.004288826
```

Note that based on our above analysis, that this should be equivalent to a linear hypothesis with

$$A = [2, 1], \quad b = 0,$$

you can verify that on your own.

1.5 Misspecification of the DGP

A quick time out may also be in order here. Notably, we have relied on the assumption that we know the density f that generates the data. This is, of course, absurd. We never really know the true DGP, but it puts us in a case where we return to the age-old truism that "all models are wrong, but some are useful." Given that we're admitting our ignorance regarding the model, we have two choices:

1. If all we're interest in is average marginal effects and all the quantities of interest are observable, then OLS may be more attractive, given it's some what weaker assumptions. However, the kinds of questions we can consider may be severely limited if we only allow ourselves to search for one-off, black-box average treatment effects in reduced form.
2. We accept that more often than not we're actually choosing a model that may be more or less **close** to the true DGP. Let's see where we can go with this.

Ok, let's suppose that the true model is $g(y|\theta)$. What happens if we fit the data using a model of $f(y|\beta)$ instead? To make progress here, we'll need to consider a measure of the "gap" between models. We don't really know how β and θ relate to each other, so instead of looking for the distance between them, we'll need to look at the divergence between f and g,

$$KL(g; f) = \int g(y|\theta) \log \frac{g(y|\theta)}{f(y|\beta)} dy.$$

This is known as the **Kullback-Liebler (KL) divergence** from f to g. If g is the true model then we get

$$KL(g; f) = \mathrm{E}\left[\log \frac{g(y|\theta)}{f(y|\beta)}\right].$$

A few things to point out

1. $KL(g; f) \geq 0$. To see this note that

$$KL(g; f) = \mathrm{E}\left[-\log \frac{f(y|\beta)}{g(y|\theta)}\right] \geq \mathrm{E}\left[1 - \frac{f(y|\beta)}{g(y|\theta)}\right] = \int g(y|\theta)dy - \int f(y|\beta)dy = 1 - 1 = 0.$$

2. If $f = g$ then $KL(g; f) = 0$, which is to say there is no divergence. This is the only case where the divergence is 0.

Before going on, let's consider two examples. First suppose that f and g are both normal

with means μ_f and μ_g, but the same variance. Then we have

$$\begin{aligned}
\log \frac{g(\theta)}{f(\beta)} &= \log \frac{\exp\left(-\frac{(y-\mu_g)^2}{2\sigma^2}\right)}{\exp\left(-\frac{(y-\mu_f)^2}{2\sigma^2}\right)} \\
&= \frac{\mu_f^2 - \mu_g^2 - 2y\left(\mu_f - \mu_g\right)}{2\sigma^2} \\
KL(g;f) &= \mathrm{E}\left[\log \frac{g(\theta)}{f(\beta)}\right] \\
&= \frac{\mu_f^2 - \mu_g^2 - 2\,\mathrm{E}[y]\left(\mu_f - \mu_g\right)}{2\sigma^2} \\
&= \frac{(\mu_f - \mu_g)^2}{2\sigma^2}.
\end{aligned}$$

In this case the divergence from f to g is proportional to the squared difference in means. Also note that in this case $KL(g;f) = KL(f;g)$, but that will rarely be the case.

Second, suppose that g is log-normal, but everything else is the same, then

$$KL(g;f) = \frac{\mu_f^2 - 2\mu_f \exp(\mu_g + \sigma^2) - \sigma^2 + \exp(2\mu_g + 2\sigma^2)}{2\sigma^2}$$

which will generally not be the same as $KL(f;g)$

OK, so let's consider then what happens when we fit the wrong model. First of all, we end up with our estimate is

$$\hat{\beta} = \underset{\beta}{\operatorname{argmax}} \frac{1}{N}\sum_{i=1}^{N} \log f(y_i|\beta).$$

From our discussion of consistency we know that this likelihood function converges to

$$\frac{1}{N}\sum_{i=1}^{N} \log f(y_i|\beta) \xrightarrow{p} \mathrm{E}_g[\log f(y|\beta)].$$

Note that g though, because this expectation is conditional on the true DGP g. What can we say about this quantity?

$$\begin{aligned}
\mathrm{E}_g[\log f(y|\beta)] &= \mathrm{E}_g[\log f(y|\beta)] - \mathrm{E}_g[\log g(y|\theta)] + \mathrm{E}_g[\log g(y|\theta)] \\
&= \mathrm{E}_g[\log g(y|\theta)] - \mathrm{E}_g\left[\log \frac{g(y|\theta)}{f(y|\beta)}\right] \\
&= \mathrm{E}_g[\log g(y|\theta)] - KL(g;f).
\end{aligned}$$

The first term doesn't depend on β, so the MLE $\hat{\beta}$ associated with maximizing $\mathrm{E}_g[\log f(y|\beta)]$

is the one that minimizes $KL(g; f)$. We'll take this as our next property

Property A10 *Let $y_1, \ldots, y_N$ be an iid sample from distribution $g(y|\theta)$ and let $KL(g(y|\theta); f(y|\beta))$ to have unique minimum $\beta = \beta^*$, then with regularity conditions on g and f, the MLE $\hat{\beta} \xrightarrow{p} \beta^*$*

In words, this means that the MLE obtained by fitting the data to the incorrect model f, will converge to the parameter values that make the incorrect model *as close as possible* to the unknown correct model. In this sense, we can rest somewhat easy as long as we think the model is close-enough or a reasonable approximation to the real model.

The next question becomes, however, what happens to the variance of the MLE in these cases? We derived the previous variance expression using the assumption of a correct model. So what changes now that we're even more uncertain?

The main thing, we need to think about here is what happens to the expected value when we go to apply the LLN.

$$\begin{aligned}
\sqrt{N}(\hat{\beta}_N - \beta^*) &= \left[-\frac{1}{N}H(\bar{\beta}|y)\right]^{-1}\sqrt{N}\left(\frac{1}{N}s(\beta^*|y)\right) \\
\frac{1}{N}H(\bar{\beta}|y) &\xrightarrow{p} \mathrm{E}_g\left[D_{\beta\beta'}\log f(\beta^*|y)\right] \\
\frac{1}{N}s(\beta^*|y) &\xrightarrow{p} 0 \\
\sqrt{N}\left(\frac{1}{N}s(\beta^*|y)\right) &\xrightarrow{d} N\left(0, \mathrm{Var}_g(D_\beta \log f(\beta^*|y))\right) \\
\sqrt{N}(\hat{\beta}_N - \beta^*) &\xrightarrow{d} N\left(0, \mathrm{E}_g\left[D_{\beta\beta'}\log f(\beta^*|y)\right]^{-1}\mathrm{Var}_g(D_\beta \log f(\beta^*|y))\,\mathrm{E}_g\left[D_{\beta\beta'}\log f(\beta^*|y)\right]^{-1}\right)
\end{aligned}$$

When we were confident in our model choice, we could employ the information equality, which told us that

$$\mathrm{Var}_g(D_\beta \log f(\beta^*|y)) = \mathrm{E}_g\left[D_{\beta\beta'}\log f(\beta^*|y)\right], \quad \text{if } f = g.$$

We don't have that reassurance anymore so we can't claim this equality. Instead, we will use the sample counterparts of these matrices to construct a sandwich estimator.

$$\begin{aligned}
\hat{\mathrm{E}}_g \left[D_{\beta\beta'} \log f(\beta^*|y) \right]^{-1} &= \left[\frac{1}{N} H(\hat{\beta}|y) \right]^{-1} \\
\widehat{\mathrm{Var}}_g (D_\beta \log f(\beta^*|y)) &= \frac{1}{N} \left[\sum_{i=1}^{N} \left[D_\beta \log f(y_i|\hat{\beta}) \right] \left[D_\beta \log f(y_i|\hat{\beta}) \right]' \right] \\
\widehat{\mathrm{avar}}(\hat{\beta})_{\text{robust}} &= H(\hat{\beta}|y)^{-1} \left[\sum_{i=1}^{N} \left[D_\beta \log f(y_i|\hat{\beta}) \right] \left[D_\beta \log f(y_i|\hat{\beta}) \right]' \right] H(\hat{\beta}|y)^{-1} \\
&= \widehat{\mathrm{avar}}(\hat{\beta})_H \widehat{\mathrm{avar}}(\hat{\beta})_{OPG}^{-1} \widehat{\mathrm{avar}}(\hat{\beta})_H .
\end{aligned}$$

In the MLE world, this is what people mean when they say they're using robust standard errors. Unlike in the linear model, it's not always clear what to say that they're robust to. Also unlike the linear case, we are not guaranteed to have unbiased or consistent estimates in this case due to the misspecification. Instead we are forming good standard errors for an estimate of β that gets our incorrect model as close as possible to the truth. But we may not know much about how β relates to θ other than this abstract understanding.

2 GLMs with continuous(ish) data

We're now ready to move towards applied regression work. Our main technology is going to be moving to what is known as the generalized linear model (GLM). A GLM consists of 2 parts:

1. A conditional distribution model for the outcome variable y given some covariates X

$$y_i \mid X \overset{iid}{\sim} f(y|\theta, X).$$

2. A **link function** g that links the conditional expectation function (CEF) of y given X using a linear-in-parameters specification $X\beta$.

$$\mathrm{E}[y \mid X] = g^{-1}(X\beta)$$

Starting with step 1, there are often many possible distributions we could impose. Really any distribution that avoids a zero-likelihood problem is fair game, but often we'll restrict ourselves to a core set of commonly used distributions.

2.1 Normal GLM

Let's start with some thing that should be familiar, the normal model. When considering data y that are more-or-less continuous we might choose a distribution that

1. Has support on the entire real line (i.e., avoids 0 likelihood problems)
2. Is easy to work with
3. Appears in often enough in the real world that it seems like a good default starting point.

What's a normal distribution that satisfies these conditions? The normal distribution! So let's write down our model and link function to be:

$$y_i|X \overset{iid}{\sim} N(x_i'\beta, \sigma^2).$$

Here we are adjusting the above normal model by replacing the unconditional mean μ with a conditional expectation function (CEF) $\mathrm{E}[y_i|X] = x_i'\beta$.

Note that we are sliding in all of the "classical" linear model assumptions in here. Specifically, this is identical to writing out the standard linear model with normal errors and the following assumptions:

1. Linear-in-the-parameters functional form

$$y_i = x_i'\beta + \varepsilon_i, \quad \varepsilon_i|X \overset{iid}{\sim} N(0, \sigma^2),$$

2. Strict exogeneity $\mathrm{E}[\varepsilon_i|X] = 0$ (i.e., no omitted variables and no lagged values of y in a time series setting)
3. Independence (each observation is an independence draw from a distribution with...)
4. Homoskedasticity (... constant variance conditional on the observables)
5. The conditional distribution of ε (normal)

You may recall that only the first 2 of these are necessary for OLS to provide unbiased and consistent estimates of β. It is biased but consistent under a weaker version of 2. The first 4 guarantee that OLS is BLUE. To remind ourselves, under these assumptions the OLS estimator has the following properties

$$\hat{\beta}_{\text{OLS}} = (X'X)^{-1}(X'y) = \left(\sum_{i=1}^{N} x_i x_i'\right)^{-1} \left(\sum_{i=1}^{N} x_i y_i\right)$$

$$\hat{\sigma}^2_{\text{OLS}} = \frac{1}{N-k}\sum_{i=1}^{N}(y_i - x_i'\hat{\beta}_{\text{OLS}})^2$$

$$\hat{\beta} \overset{asy}{\sim} N(\beta, \text{avar}(\hat{\beta})) \qquad \text{Assumptions 1-2}$$

$$\widehat{\text{avar}}(\hat{\beta}) = (X'X)^{-1}\left(\sum_{i=1}^{N} x_i x_i' \hat{\varepsilon}_i^2\right)(X'X)^{-1} \qquad \text{Assumptions 1-3}$$

$$\begin{aligned}\widehat{\text{Var}}(\hat{\beta}|X) &= \hat{\sigma}^2_{\text{OLS}}(X'X)^{-1} \\ &= \hat{\sigma}^2_{\text{OLS}}\left(\sum_{i=1}^{N} x_i x_i'\right)^{-1} \qquad \text{Assumptions 1-4}\end{aligned}$$

$$\frac{\hat{\beta}_j - \beta_j}{s.e.(\beta_j)} \sim t_{N-k} \qquad \text{Assumptions 1-5}$$

With assumptions 1-5, we can fit the model with MLE. The likelihood function is the same normal likelihood from before and our regularlity conditions from the last section are satisfied. The only difference is replace the unconditional expected value μ with the CEF $x_i'\beta$, giving us

$$\begin{aligned}L(\beta, \sigma^2|y) &= -\frac{N}{2}\log(2\pi) - \frac{N}{2}\log(\sigma^2) - \frac{1}{2\sigma^2}\sum_{i=1}^{N}(y_i - x_i'\beta)^2 \\ &= -\frac{N}{2}\log(2\pi) - \frac{N}{2}\log(\sigma^2) - \frac{1}{2\sigma^2}(y - X\beta)'(y - X\beta)\end{aligned}$$

With first order conditions

$$\begin{aligned}D_\beta L(\beta, \sigma_\varepsilon^2 \mid y) &= \frac{1}{\sigma_\varepsilon^2}\sum_{i=1}^{N} x_i(y_i - x_i'\beta) \\ 0 &= \sum_{i=1}^{N}(x_i y_i - x_i x_i'\beta) \\ 0 &= \sum_{i=1}^{N} x_i y_i - \sum_{i=1}^{N} x_i x_i'\beta \\ \hat{\beta}_{MLE} &= \left(\sum_{i=1}^{N} x_i x_i'\right)^{-1}\left(\sum_{i=1}^{N} x_i y_i\right) = (X'X)^{-1}X'y.\end{aligned}$$

This is, of course, the OLS estimator. This gives us something to remark on: the GLM of normally distributed y is a linear model and the MLE of β in this model is equivalent to the OLS estimator under the 5 assumptions above.

However, the estimator of σ^2, like with the sample variance, is not the unbiased estimator

you're used to seeing here.

$$\begin{aligned} D_{\sigma^2} L(\beta, \sigma^2 \mid y) &= -\frac{N}{2\sigma^2} + \sum_{i=1}^{N} \frac{1}{2\sigma^4}(y_i - \beta' x_i)^2 \\ 0 &= -N + \frac{1}{\sigma^2}\sum_{i=1}^{N}(y_i - \beta' x_i)^2 \\ \hat{\sigma}^2_{MLE} &= \frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{\beta}' x_i)^2 = \frac{\hat{\varepsilon}'\hat{\varepsilon}}{N}, \end{aligned}$$

To find the variance of these estimates, we'll use the Hessian to find the Fisher Information:

$$H(\theta \mid y) = \begin{bmatrix} -\frac{1}{\sigma^2}\sum_{i=1}^{N} x_i x_i' & -\frac{1}{\sigma^4}\sum_{i=1}^{N}(y_i - \beta' x_i)x_i \\ -\frac{1}{\sigma^4}\sum_{i=1}^{N}(y_i - \beta' x_i)x_i & \sum_{i=1}^{N}\frac{1}{2\sigma^4} - \frac{1}{\sigma^6}(y_i - \beta' x_i)^2 \end{bmatrix},$$

At a set of estimates $(\hat{\beta}, \hat{\sigma}^2)$, we could plug these in and get the observed Hessian estimator of the variance, but this is actually an easy case for calculating the expected value of the Hessian.

Specifically, the Fisher Information for this iid sample is

$$\begin{aligned} NI(\theta) &= -\operatorname{E}\left[H(\theta \mid y)\right] \\ &= N \begin{bmatrix} \frac{1}{\sigma^2}\operatorname{E}[x_i x_i'] & \frac{1}{\sigma^4}\operatorname{E}[x_i(y_i - \beta' x_i)] \\ \frac{1}{\sigma^4}\operatorname{E}[x_i'(y_i - \beta' x_i)] & -\frac{1}{2\sigma^4} + \frac{1}{\sigma_\varepsilon^6}\operatorname{E}[(y_i - \beta' x_i)^2] \end{bmatrix} \\ &= N \begin{bmatrix} \sigma^2 \operatorname{E}[x_i x_i'] & 0 \\ 0 & \frac{1}{2\sigma^4} \end{bmatrix} \\ \operatorname{avar}(\theta) &= \frac{1}{N} I(\theta)^{-1} = \frac{1}{N}\begin{bmatrix} \sigma^2 \operatorname{E}[x_i x_i']^{-1} & 0 \\ 0 & 2\sigma^4 \end{bmatrix} \\ \widehat{\operatorname{avar}}(\theta) &= \begin{bmatrix} \hat{\sigma}^2\left[\sum_{i=1}^{N} x_i x_i'\right]^{-1} & 0 \\ 0 & 2\hat{\sigma}^4/N \end{bmatrix}. \end{aligned}$$

Note that the upper is asymptotic OLS covariance matrix with homoskedastic errors. This makes sense given our setup here.

We can consider interpreting the model, note that the link function *is* the linear specification of the CEF. This give us the very easy interpretation that each β represents the average marginal effect of that specific X on y, holding the others fixed, i.e.,

$$AME(x_j) = D_{X_j} \operatorname{E}[y_i | X] = \beta_j.$$

We can also form predictions of the expected outcome given some input.

$$\mathrm{E}[y^*|X = x^*] = x^{*\prime}\beta.$$

In both these cases, standard errors follow from ordinary variance results. If we're in a case where we **really** believe the exogeneity assumptions, then and only then, can we interpret β_j casually.

You spent all of last semester working with linear models, so we're not going to spend a lot of time here except to point out that there are at three obvious cases where this will be the "wrong" model.

1. Non-spherical errors due to heteroskedasticity, non-independence, or both.
2. Omitted variables
3. The "true" model is actually another distribution, maybe log-normal or binomial.

Regarding the first point, you should remember the following two facts from last semester:

1. The OLS estimator is unbiased, consistent, but inefficient with non-spherical errors so long as strict exogeneity ($\mathrm{E}[\varepsilon_i \mid X] = 0$) holds.
2. OLS is biased, inefficient, but still consistent with non-spherical errors so long as weak exogeneity ($\mathrm{E}[\varepsilon_i \mid x_i] = 0$) holds.

Because the MLE of β is equivalent to the OLS estimate, the MLE inherits these properties. As such, we know that the MLEs are robust to some incorrect model choices. Using the homoskedastic normal model despite knowing it's likely a simplification of the true DGP makes it a **pseudo-likelihood** (PL) estimator. This means that we optimized a likelihood function we knew to be wrong, but we think it is "close enough" to the model we have in mind to still be interested in the estimates.
In these cases, our MLE robust standard errors should be used and will be interesting. Additionally, you can show that these robust standard errors are identical to what you called robust standard errors last semester, so we know that they are robust to heteroskedastity.

An alternative approach is to consider expanding our model to reflect whatever variance assumptions we want to make. In this case, we might rewrite the model to be

$$y|X \sim N(X\beta, \Omega).$$

In this case we're saying that our entire observed data is a single draw from an N-dimensional multivariate normal.

This creates what is confusingly called the **general linear model** (not generalized, not GLM), with log-likelihood

$$L_{GL}(\theta \mid y) = -\frac{1}{2}\log(\det(\Omega)) - \frac{N\log(2\pi)}{2} - \frac{1}{2}(y - X\beta)'\Omega^{-1}(y - X\beta).$$

Exactly what Ω looks like depends on what you're modeling. In an ideal world, we'd want to know everything in Ω, but that is $(N^2 - N)/2 + N$ different parameters plus the k βs , and we only have N observations. We cannot estimate more parameters than we have data! Simplifications are in order.

If we're worried about heteroskedasticity, then Ω is an $N \times N$ diagonal matrix with elements σ_i^2, can we do anything with that? Not yet, we're down to $N + k$ parameters, but that's still too many. This means we have to impose a simplification like

$$\sigma_i^2(z_i; \gamma) = \exp(z_i'\gamma).$$

This makes the log-likelihood:

$$L_{GL}(\theta \mid y) = \frac{-N\log(2\pi)}{2} - \frac{1}{2}\sum_{i=1}^{N}\left(z_i'\gamma + \frac{(y_i - x_i'\beta)^2}{\exp(z_i'\gamma)}\right).$$

Whether it is worth imposing this additional functional form and fitting the complicated models just to claw back efficiency is a value judgement, but my personal take here is that it is rarely worth that effort.

Regarding point 2, this is always the problem with observational social science. You should have spend some time last year talking about omitted variable bias, how to choose which covariates to include in a model, and solutions to endogeneity problems, including instrumental variables. Some quick recap for this linear model:

Suppose that the true model is

$$y = X_1\beta_1 + X_2\beta_2 + \varepsilon,$$

but we only fit the restricted model that includes X_1. The OLS estimates of the restricted model are

$$\hat{\beta}_R = (X_1'X_1)^{-1}X_1'y.$$

We can rewrite this

$$\begin{aligned}\hat{\beta}_R &= (X_1'X_1)^{-1}X_1'(X_1\beta_1 + X_2\beta_2 + \varepsilon) \\ &= (X_1'X_1)^{-1}X_1'X_1\beta_1 + (X_1'X_1)^{-1}X_1'X_2\beta_2 + (X_1'X_1)^{-1}X_1'\varepsilon \\ &= \beta_1 + (X_1'X_1)^{-1}X_1'X_2\beta_2 + (X_1'X_1)^{-1}X_1'\varepsilon,\end{aligned}$$

which has an expected value of

$$\mathrm{E}[\hat{\beta}_R \mid X] = \beta_1 + (X_1'X_1)^{-1}X_1'X_2\beta_2.$$

As you can see this is **not** always equal to β_1. In this case the OLS estimator is unbiased only if $(X_1'X_1)^{-1}X_1'X_2\beta_2 = 0$. A few ways that we get unbiasedness in this case are

1. $\beta_2 = 0$, which is to say that the variables in X_2 are irrelevant
2. $X_1'X_2 = 0$, which is related to the correlation of the variables in X_1 and X_2.

This tells us that in the linear model (normal GLM), we only need to worry about omitted variables that are correlated with both the included variable of interest and the outcome. We often refine this further by talking about the "backdoor criteria." Under this criteria, a candidate control variable should have a theoretical (pre-treatment) relationship to **both** X and y, but should not itself be a result of changing X (not post-treatment). The reason we want to avoid post-treatment variables, is that, substantively, we want to know the full effect of X on y. This effect of interest includes the part that effects y through its effect on post-treatment variables. So controlling for these post-treatment factors results in removing an interesting part of the true effect.

Finally, with respect to point 3, we know that we have different. We already said, that with this specific distribution, the MLEs of β are unbiased and consistent under strict exogeneity even when if the DGP is non-normal. So long as the CEF is linear and we believe strict exogeneity then normal MLE with robust standard errors is a good estimator. However, if we think the CEF is likely to be non-linear, then we may want to consider alternative models.

2.1.1 The log-normal

In many cases, we find ourselves considering data are "sort of" continuous. Specifically, it may be data are weakly/strictly positive (e.g., foreign aid). In these cases, we have some choices as to what's the best way to consider these data.

1. Do we leave the data in "raw" levels and fit a linear model (level changes in x and y)?
2. Do we take the log of y and fit a log-linear model (level changes in x and percentage

changes y)?

The former is a regular linear model, or the normal GLM. The latter is a log-normal model

$$\begin{aligned} y_i|X &\overset{iid}{\sim} \log N(x_i'\beta, \sigma^2), \quad y_i > 0 \\ \mathrm{E}[y_i|X] &= \exp(x_i'\beta + \sigma^2/2) \\ \log(y_i)|X &\overset{iid}{\sim} N(x_i'\beta, \sigma^2) \\ f(y_i|\theta, X) &= \frac{1}{y_i\sqrt{2\sigma^2}} \exp\left(\frac{-1}{2\sigma^2}(\log(y_i) - \mu)^2\right) \end{aligned}$$

Because $\log(y)|X$ is normally distributed so we end up back with the same linear model where the MLE of β is

$$\hat{\beta} = (X'X)^{-1}X'\log(y).$$

Of course, this changes how we interpret values of β and produce expected values of y given a profile X.

Normal model In the normal model we have our standard interpretation. "On average, a one [unit of X_1] increase in X_1 is associated with a [$\hat{\beta}_1$] [unit of y] change in y, holding the other regressors constant "

Log-normal model In the log-normal model, we start with the conditional expectation and take the derivative with respect to X_1

$$\begin{aligned} D_{X_1}\mathrm{E}[y_i|X] &= D_{X_1}\exp(x_i'\beta + \sigma^2/2) \\ &= \exp(x_i'\beta + \sigma^2/2)\beta_1 \\ &= \mathrm{E}[y_i|X]\beta_1 \\ \frac{D_{X_1}\mathrm{E}[y_i|X]}{\mathrm{E}[y_i|X]} &= \beta \\ 100 \times \frac{D_{X_1}\mathrm{E}[y_i|X]}{\mathrm{E}[y_i|X]} &= 100 \times \beta \end{aligned}$$

This top line is the instantaneous change in $\mathrm{E}[y_i|X]$ as X_1 increases, while the bottom is the observed expected value. This gives us something that looks like percentage change. "On average, a one [unit of X_1] increase in X_1 is associated with a [$\hat{\beta}_1 \times 100$] percent change in [y], holding the other regressors constant." This is an (often good) approximation, however, because the instantaneous change is not the same as a true change of +1 since this is a non-linear function.

Another approach is to just add 1 to X_1 and see what happens

$$\begin{aligned}
\mathrm{E}[y_i|X+1] - \mathrm{E}[y_i|X] &= \exp(x_i'\beta + \beta_1 + \sigma^2/2) - \exp(x_i'\beta + \sigma^2/2) \\
&= \exp(x_i'\beta + \sigma^2/2)\exp(\beta_1) - \exp(x_i'\beta + \sigma^2/2) \\
&= \mathrm{E}[y_i|X](\exp(\beta_1) - 1) \\
\frac{\mathrm{E}[y_i|X+1] - \mathrm{E}[y_i|X]}{\mathrm{E}[y_i|X]} &= \exp(\beta) - 1 \\
100 \times \frac{\mathrm{E}[y_i|X+1] - \mathrm{E}[y_i|X]}{\mathrm{E}[y_i|X]} &= 100 \times (\exp(\beta) - 1)
\end{aligned}$$

This gives us something that looks like percentage change. "On average, a one [unit of X_1] increase in X_1 is associated with a [$100 \times (\exp(\beta) - 1)$] percent change in [$y$], holding the other regressors constant." This second interpretation is correct, but the earlier one is often a decent approximation.

Another way to approach interpretation is to focus on the average marginal effect (AME).

$$\begin{aligned}
AME(X_1) &= \mathrm{E}[D_{X_1}\,\mathrm{E}[y_i|X]] \\
&= \mathrm{E}[\beta_1 \exp(x_i'\beta + \sigma^2/2)] \\
&= \beta_1 \exp(\sigma^2/2)\,\mathrm{E}[\exp(x_i'\beta)] \\
\widehat{AME(X_1)} &= \hat{\beta}_1 \exp(\hat{\sigma}^2/2)\frac{1}{N}\sum_{i=1}^{N}\exp(x_i'\hat{\beta})
\end{aligned}$$

Of course, the log-normal model can only rationalize weakly positive variables. That hasn't stopped people from applying it to cases where some of the observed values of y_i are 0. So how do we avoid a zero-likelihood problem? The two main ways are to either

1. Use a different distribution (we'll see that Poisson is a good alternative here).
2. Add some positive constant c to every value of y_i, with $c = 1$ being the most common choice. Although there is litle justification for this over smaller values. In this case, the model actually becomes

$$\begin{aligned}
y_i + c|X &\overset{iid}{\sim} \log N(x_i'\beta, \sigma^2), \quad y_i + c > 0 \\
\mathrm{E}[y_i|X] &= \exp(x_i'\beta + \sigma^2/2) - c \\
\log(y_i + c)|X &\overset{iid}{\sim} N(x_i'\beta, \sigma^2),
\end{aligned}$$

This doesn't affect any of the above formulas or interpretations except we're now implicitly dicussing changes in $y + c$, in some cases this is minor and is likely safely

ignored (e.g., what difference does 1 dollar make to something like GDP? it's a rounding error) in other cases it seems weirder.

2.2 Comparing non-nested models

Comparative model testing da great tool for choosing among competing specifications. We've done some of these before in the context of nested models.

In those cases we compared a restricted model to an unrestricted model using a Wald test. With non-nested model testing we want to kick that up a notch. Consider two GLMs

$$y_i|X \overset{iid}{\sim} f(y|\theta_1)$$
$$y_i|Z \overset{iid}{\sim} g(y|\theta_2)$$

We want to know if f or g is a better model of y.

Technical take: The first model is **nested** within the second model if for all possible values of θ_1 we can find a θ_2 such that $f(y_i \mid \theta_1) = g(y_i \mid \theta_2)$ for all (x, z, y). When $f = g$, model are nested if $X \subseteq Z$ or vice-versa.

They are **non-nested and overlapping** if there are some for some (x, z, y) for which we can find θ_1 and θ_2 that make $f(y_i \mid \theta_1) = g(y_i \mid \theta_2)$. This is typically when $f = g$, neither $X \subseteq Z$ nor $X \subseteq Z$, but $Z \cap X \neq \emptyset$.

Finally, models are **strictly non-nested** if there are no θ_1 and θ_2 such that $f(y_i \mid \theta_1) = g(y_i \mid \theta_2)$ for any (x, z, y). This is the case when f and g are different or if $f = g$ and $Z \cap X = \emptyset$.

Note that when we think about the normal and log-normal we have

$$\begin{aligned} \mathrm{E}[y_i|X] &= \beta' x_i & \text{Model I: } y_i \mid x_i &\sim N(\beta' x_i, \sigma_f^2) \\ \mathrm{E}[y_i|Z] &= \exp(\gamma' z_i + \sigma_g^2/2) & \text{Model II: } y_i \mid x_i &\sim \text{Log-Normal}(\gamma' z_i, \sigma_g^2) \end{aligned}$$

These two models are strictly non-nested even if they contain the same covariates because the non-linear nature of the log-normal makes it impossible to set the parameters in such a way to get to the normal distribution. The question we face, is how to determine which of these is the better model?

Vuong (1989) proposes a two-part test. First, he proposes a test for whether the models are distinguishable. Based on the results of that test, there are different tests for model comparison. The first test of whether the models overlapping or not is a bit cumbersome; we'll skip the derivation.

However, if we reject the null of indistinguishable models we can build Vuong's non-nested test based the idea of KL divergence. Specifically suppose there is a true model h then we can think about the divergence between both proposed models and the truth

$$KL(h; f) = \mathrm{E}\left[\log \frac{h(y|\theta)}{f(y|\theta_1)}\right]$$
$$KL(h; g) = \mathrm{E}\left[\log \frac{h(y|\theta)}{g(y|\theta_2)}\right]$$

We want to know which one is closer to the truth, so we can look at the difference of these values

$$\begin{aligned} KL(h; g) - KL(h; f) &= \mathrm{E}\left[\log \frac{h(y|\theta)}{g(y|\theta_1)} - \log \frac{h(y|\theta)}{f(y|\theta_1)}\right] \\ &\quad \mathrm{E}\left[\log h(y|\theta) - \log g(y|\theta_1) - \log h(y|\theta) + \log f(y|\theta_1)\right] \\ &\quad = \mathrm{E}\left[\log f(y|\theta_1) - \log g(y|\theta_1)\right] \end{aligned}$$

In this case, if g is better then $KL(h; g) < KL(h; f)$ and this whole term is negative. Alternatively if f is better than $KL(h; g) > KL(h; f)$ and this whole term is positive. Note that the last line is the expected value of likelihood ratio for these two models, which we can estimate with

$$LR_N(\hat{\theta}_1, \hat{\theta}_2) = \frac{1}{N} \sum_{i=1}^{N} \log \frac{f(y_i \mid \theta_1)}{g(y_i \mid \theta_2)}.$$

With this in hand, we can write the null hypothesis as

$$\begin{aligned} H_0 &: LR(\theta_1, \theta_2) = 0 \\ H_f &: LR(\theta_1, \theta_2) > 0 \quad \text{Model I is better} \\ H_g &: LR(\theta_1, \theta_2) < 0 \quad \text{Model II is better} \end{aligned}$$

Note that this means we will be conducting an unusual two-sided hypothesis test to see which model 'wins.' Further note that both models can be terrible, but the Vuong test helps us see which one is "closer" to the truth. To form the test statistic we get a familiar-ish setup

$$\begin{aligned} V &= \frac{LR_N(\hat{\theta}_1, \hat{\theta}_2)}{\hat{\omega}/\sqrt{N}} \\ &= \frac{\sum_{i=1}^{N} \left[\log f(y_i \mid \theta_1) - \log g(y_i \mid \theta_2)\right]}{\sqrt{N}\hat{\omega}} \end{aligned}$$

where $\hat{\omega}^2$ is the estimated variance of the LR such that

$$\hat{\omega}^2 = \frac{1}{N}\sum_{i=1}^{N}\left(\log\left(\frac{f(y_i \mid x_i, \hat{\theta}_1)}{g(y_i \mid x_i, \hat{\theta}_2)}\right)\right)^2 - \left(\frac{1}{N}\sum_{i=1}^{N}\log\left(\frac{f(y_i \mid x_i, \hat{\theta}_1)}{g(y_i \mid x_i, \hat{\theta}_2)}\right)\right)^2.$$

The main result from Vuong (1989) is that for if $\omega > 0$ then $V \xrightarrow{d} N(0,1)$ under the null. The first test is whether $\omega = 0$, if we fail that test, then there is not enough variance in the LR to move onto the second test. If H_f is true than $V \xrightarrow{p} \infty$, while under H_g ,$V \xrightarrow{p} -\infty$. If V is greater than a critical z^* we conclude that Model 1 is better, and if V is less than $-z^*$ we conclude Model II is better. So if we're testing at the 5% level, we set $z^* = 1.96$ and compare. Note that if V is between these cutpoints, we can't conclude that either is better.

Note that as in all model comparisons we need the outcome variable y to be the same in each model and for both models to be fit using the same sample. In this case, we can compare the normal model to the Poisson.

Alternative methods of model comparison also exist. These include the Akaike information criteria (AIC) and the Bayesian (or Schwarz) information criteria (BIC). These can be used for either nested or non-nested models. Both take the form:

$$IC(\theta) = -2L(\hat{\theta}|y) + kp.$$

In this case k is the number of parameters in the model, while p is a penalty term. For the AIC $p = 2$ and for the BIC $p = \log(N)$. Note that once we have more than 7 observations, the BIC more heavily penalizes adding additional parameters. Some versions of Vuong's test include a similar penalty:

$$V_p = \frac{LR_N(\hat{\theta}_1, \hat{\theta}_2) - \log(N)(k_1 - k_2)/2}{\hat{\omega}/\sqrt{N}}.$$

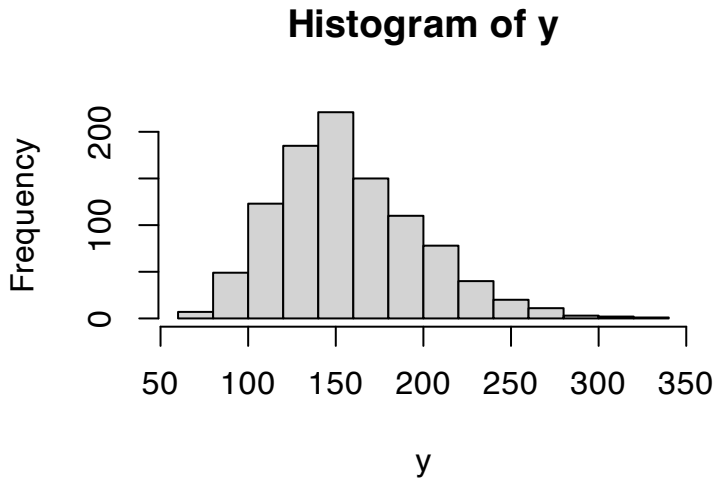
Let's look at a quick example with some simulated data where I know the true model, but you don't

```
summary(dat)
```

```
##        y                X1               X2               X3
##  Min.   : 64.79   Min.   : 9.441   Min.   :0.7114   Min.   : 61.26
##  1st Qu.:129.72   1st Qu.:18.678   1st Qu.:1.6517   1st Qu.:125.56
##  Median :151.66   Median :21.361   Median :1.9970   Median :147.37
```

```
##  Mean   :157.36   Mean   :21.472   Mean   :2.0085   Mean   :153.23
##  3rd Qu.:182.33   3rd Qu.:24.363   3rd Qu.:2.3321   3rd Qu.:178.73
##  Max.   :326.01   Max.   :37.341   Max.   :3.7693   Max.   :326.52
##        X4
##  Min.   : 3.00
##  1st Qu.:13.00
##  Median :15.00
##  Mean   :15.19
##  3rd Qu.:18.00
##  Max.   :30.00
```

```
hist(y)
```



We will illustrate two ways to use Vuong's test. The first is comparative theory test and the second is comparative model selection. These are deeply connected. Let $x_i = (1, x_{i1}, x_{i2}, x_{i3})'$ and $z_i = (1, x_{i3}, x_{i4})'$ We will have 4 models of interest

$$y_i \mid X \overset{iid}{\sim} N(x_i'\beta, \sigma_1^2)$$
$$y_i \mid Z \overset{iid}{\sim} N(z_i'\gamma, \sigma_2^2)$$
$$y_i \mid X \overset{iid}{\sim} \log N(x_i'\beta, \sigma_3^2)$$
$$y_i \mid Z \overset{iid}{\sim} \log N(z_i'\gamma, \sigma_4^2)$$

```
m1 <- lm(y~X1+X2+X3, data=dat, x=TRUE, y=TRUE)
m2 <- lm(y~X3+X4, data=dat, x=TRUE, y=TRUE)
m3 <- lm(log(y)~X1+X2+X3, data=dat, x=TRUE, y=TRUE)
m4 <- lm(log(y)~X3+X4, data=dat, x=TRUE, y=TRUE)
```

```r
summary(m1)
```

```
##
## Call:
## lm(formula = y ~ X1 + X2 + X3, data = dat, x = TRUE, y = TRUE)
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -22.235  -2.688  -0.146   2.733  40.661
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)  2.224822   1.264549   1.759   0.0788 .
## X1          -0.043403   0.041450  -1.047   0.2953
## X2           1.880957   0.352345   5.338 1.16e-07 ***
## X3           0.993885   0.004223 235.363  < 2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 5.422 on 996 degrees of freedom
## Multiple R-squared:  0.9823, Adjusted R-squared:  0.9823
## F-statistic: 1.847e+04 on 3 and 996 DF,  p-value: < 2.2e-16
```

```r
summary(m2)
```

```
##
## Call:
## lm(formula = y ~ X3 + X4, data = dat, x = TRUE, y = TRUE)
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -21.072  -2.780  -0.051   2.680  40.283
##
## Coefficients:
##             Estimate Std. Error t value Pr(>|t|)
## (Intercept) 0.454588   0.986509   0.461    0.645
```

```
## X3          0.994895   0.004199 236.927  < 2e-16 ***
## X4          0.293748   0.045024   6.524 1.09e-10 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 5.383 on 997 degrees of freedom
## Multiple R-squared:  0.9826, Adjusted R-squared:  0.9825
## F-statistic: 2.812e+04 on 2 and 997 DF,  p-value: < 2.2e-16
```

```
summary(m3)
```

```
##
## Call:
## lm(formula = log(y) ~ X1 + X2 + X3, data = dat, x = TRUE, y = TRUE)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -0.30562 -0.02429  0.01075  0.03532  0.19690
##
## Coefficients:
##               Estimate Std. Error t value Pr(>|t|)
## (Intercept)  4.059e+00  1.337e-02 303.608  < 2e-16 ***
## X1          -3.289e-04  4.382e-04  -0.751  0.45306
## X2           1.276e-02  3.725e-03   3.425  0.00064 ***
## X3           6.187e-03  4.464e-05 138.575  < 2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.05733 on 996 degrees of freedom
## Multiple R-squared:  0.9507, Adjusted R-squared:  0.9506
## F-statistic:  6403 on 3 and 996 DF,  p-value: < 2.2e-16
```

```
summary(m4)
```

```
##
## Call:
## lm(formula = log(y) ~ X3 + X4, data = dat, x = TRUE, y = TRUE)
##
```

```
## Residuals:
##      Min       1Q   Median       3Q      Max
## -0.30499 -0.02446  0.01190  0.03565  0.19257
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept) 4.042e+00  1.044e-02 387.000  < 2e-16 ***
## X3          6.195e-03  4.445e-05 139.364  < 2e-16 ***
## X4          2.276e-03  4.767e-04   4.774 2.08e-06 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.05699 on 997 degrees of freedom
## Multiple R-squared:  0.9512, Adjusted R-squared:  0.9511
## F-statistic:  9724 on 2 and 997 DF,  p-value: < 2.2e-16
```

```
k1 <- length(m1$coef)+1
k2 <- length(m2$coef)+1

AIC1 <- -2*logLik(m1) + 2*k1
AIC2 <- -2*logLik(m2) + 2*k2
rbind(c(AIC(m1), AIC(m2)),
      c(AIC1, AIC2))
```

```
##          [,1]     [,2]
## [1,] 6224.938 6209.482
## [2,] 6224.938 6209.482
```

```
AIC3 <- -2*logLik(m3) + 2*k1
AIC4 <- -2*logLik(m4) + 2*k2
rbind(c(AIC(m3), AIC(m4)),
      c(AIC3, AIC4))
```

```
##           [,1]      [,2]
## [1,] -2874.075 -2886.866
## [2,] -2874.075 -2886.866
```

```
N <- nrow(dat)
BIC1 <- -2*logLik(m1) + log(N)*k1
BIC2 <- -2*logLik(m2) + log(N)*k2
rbind(c(BIC(m1), BIC(m2)),
      c(BIC1, BIC2))
```

```
##          [,1]     [,2]
## [1,] 6249.477 6229.113
## [2,] 6249.477 6229.113
```

```
BIC3 <- -2*logLik(m3) + log(N)*k1
BIC4 <- -2*logLik(m4) + log(N)*k2
rbind(c(BIC(m3), BIC(m4)),
      c(BIC3, BIC4))
```

```
##           [,1]      [,2]
## [1,] -2849.536 -2867.235
## [2,] -2849.536 -2867.235
```

```
### We can't currently compare log to non-log unless we
### use the log-normal likelihood for the log-normal models.
### As in

ln.aic3 <- -2*sum(dlnorm(dat$y,
                         meanlog = m3$fitted.values,
                         sdlog = sqrt(mean(m3$residuals^2)),
                         log=TRUE))+2*k1
ln.aic4 <- -2*sum(dlnorm(dat$y,
                         meanlog = m4$fitted.values,
                         sdlog = sqrt(mean(m4$residuals^2)),
                         log=TRUE))+2*k1
## These are comparable
c(AIC(m1), AIC(m2),
  ln.aic3, ln.aic4)
```

```
## [1] 6224.938 6209.482 7177.037 7166.246
```

```
###### Vuong test #####
## we need the likelihood ratio at each point
normal.ll <- function(theta, y,X){
  s2 <- theta[length(theta)]
  beta <- theta[-length(theta)]
  mu <- X %*%  beta
  return(-log(2*pi*s2)/2-(y-mu)^2 / (2*s2))
}

log.normal.ll <- function(theta, ln.y,X){
  s2 <- theta[length(theta)]
  beta <- theta[-length(theta)]
  mu <- X %*%  beta
  return(-log(2*pi*s2)/2-ln.y-(ln.y-mu)^2 / (2*s2))
}

s1.hat <- mean(m1$residuals^2)
s2.hat <- mean(m2$residuals^2)
s3.hat <- mean(m3$residuals^2)
s4.hat <- mean(m4$residuals^2)



LR1 <- normal.ll(c(m1$coef,s1.hat), y=m1$y, X=m1$x)
LR2 <- normal.ll(c(m2$coef,s2.hat), y=m2$y, X=m2$x)

LR3 <- log.normal.ll(c(m3$coef,s3.hat), ln.y=m3$y, X=m3$x)
LR4<- log.normal.ll(c(m4$coef,s4.hat), ln.y=m4$y, X=m4$x)

## 1 versus 2
LR <- LR1-LR2
omega2 <- mean(LR^2) - mean(LR)^2
LR <- sum(LR)
V <- LR/(sqrt(N*omega2))
V
```

```
## [1] -1.681634
```

```
pnorm(V) ## reject the null in favor of alt. that 2 is better
```

```
## [1] 0.04631988
```

```
## 3 versus 4
LR <- LR3 - LR4
omega2 <- mean(LR^2) - mean(LR)^2
LR <- sum(LR)
V <- LR/(sqrt(N*omega2))
pnorm(V) ## reject the null in favor of alt. that 2 is better
```

```
## [1] 0.03019618
```

```
## So far we have evidence that theory 2 is better than theory 1

## 1 versus 3
LR <- LR1 - LR3
omega2 <- mean(LR^2) - mean(LR)^2
LR <- sum(LR)
V <- LR/(sqrt(N*omega2))
pnorm(V) ## fail to reject the null of equal fit when the alt. is that 2 is better
```

```
## [1] 1
```

```
pnorm(V, lower=F) ## reject the null in favor of alt. that 1 is better
```

```
## [1] 1.223237e-29
```

2.3 Normal and log-normal panel models

Let's briefly, consider (review?) normal/log-normal panel model. We replace the classic linear model assumptions with the following.

1. The data generating process is linear-in-the-parameters, such that

$$y_{it} = \alpha + \beta' x_{it} + \gamma' z_i + \varepsilon_{it}.$$

   With panel data we will use $i = 1, \ldots, N$ to denote "units" and $t = 1, \ldots, T$ to denote within-unit observations. This could be individuals t within neighborhood i or the

periods of time t where we observe state or country i. Note that the covariates x vary within units, while the z variables reflect unit-level traits (i.e., constant within each unit).

2. Each unit $(x_i, z_i, \varepsilon_i)$ is drawn iid. Instead of iid observations, we have iid units with some likely correlation within the units.

3. There is strict exogeneity within units, $\mathrm{E}[\varepsilon_{it}|x_i, z_i] = 0$.

4. Additional technical assumptions that allow us to apply a law of large numbers and central limit theorem (CLT).

5. Homoskedasticty and normality as needed.

Under 1-4, the OLS estimator is unbiased and consistent for α, β and γ. As such the MLEs of the normal-GLM model will be unbiased and consistent.
However, the standard errors from the normal-GLM will be wrong, why? They're still based on iid, normal and spherical errors. In this case, however, we haven't said anything about the distribution of the errors or the dependencies within units. In fact, we strongly suspect that within units there is likely some correlation. So we will want to adjust our standard errors to correct this.

We could try robust standard errors. These do reflect various types of model misspecification, however, we also know that they do not include independence violations because they are still derived with independence in mind. So what can we do?

We can tweak the robust approach a bit by deriving it under these panel assumptions of unknown within-unit correlation but cross-sectional independence. This will give us what are commonly referred to as **clustered standard errors** or **cluster robust standard errors**. Basically, what we want to do is treat each unit as an independent draw from a size T multivariate distribution. Then we reapply the analysis we previously did with misspecification to get

$$\widehat{\mathrm{avar}}(\hat{\theta})_C = \widehat{\mathrm{avar}}(\hat{\theta})_H \left[\sum_{i=1}^{N} \left(s_i(\hat{\theta} \mid y) s_i(\hat{\theta} \mid y)' \right) \right] \widehat{\mathrm{avar}}(\hat{\theta})_H,$$

where

$$s_i(\hat{\theta} \mid y) = \sum_{t=1}^{T} D_\theta \log f(y_{it}|\hat{\theta}).$$

Note that this takes the typical kind of "sandwich" form that we're used to seeing for robust standard errors, but here the filling is the inverse OPG estimator **within each unit**. These

are then summed over all the units. As with the clustered variance matrix you should have seen last semester with the linear model, the intuition here is that each of these inverse OPG estimators provides an estimate of the variance of the score function within that unit. We then average these variances to get a good estimate of the within-unit variance-covariance in the score function, which tells us how uncertain we are about the parameters based on the shape of the pseudo likelihood function.

As with robust standard errors, we should make a few notes here. First, we're averaging over the number of units rather than the number of total observations. We need N to be large, while T can be any size.

Suppose that some of the unit-level variables are fully unobserved to us.

$$\begin{aligned} y_{it} &= \alpha_i + \beta' x_{it} + \varepsilon_{it} \\ \alpha_i &= \alpha + \gamma_1' z_i + u_i, \end{aligned}$$

with u_i unobserved. Looking at the second line, we can think of this as a situation where each unit has a unit-specific constant α_i that is composed of unit-specific traits plus unobserved features.

When dealing with panel data like this, there are two primary models of interest. The first is called the **random effects** model. The details are here, but we'll skip this in class, because like modeling the heteroskedasticity, there aren't a lot of times where this is obviously the best thing to do. In the RE world, we assume that the information in the unobserved u_i are just random deviations from some overall constant term once we condition on the observables , i.e., we rewrite the above to be

$$\begin{aligned} y_{it} &= \alpha + \beta' x_{it} + \gamma_1' z_i + u_i + \varepsilon_{it} \\ u_i &:= \gamma_2' \tilde{z}_i. \end{aligned}$$

With

$$\begin{aligned}
\mathrm{E}[\varepsilon_{it}|x_i, z_i] &= 0 \\
\mathrm{E}[u_i|x_i, z_i] &= 0 \\
\mathrm{E}[u_i\varepsilon_i|x_i, z_i] &= 0 \\
\mathrm{E}[\varepsilon_{it}\varepsilon_{jt'}|x_i, z_i] &= 0 \\
\mathrm{E}[u_i^2|x_i, z_i] &= \sigma_u^2 \\
\mathrm{E}[\varepsilon_{it}^2|x_i, z_i] &= \sigma_\varepsilon^2 \\
u_i &\sim N(0, \sigma_u^2) \\
\varepsilon_{it} &\sim N(0, \sigma_\varepsilon^2),
\end{aligned}$$

for all $i \neq j$ or $t \neq t'$.

Under these assumptions, we can create the likelihood function as we have a new distributional assumption

$$y_i|x_i, z_i \sim N\left(\left[\alpha + \beta' x_{it} + \gamma' z_i\right]_{t=1}^{T_i}, \Sigma\right),$$

where $\Sigma = I\sigma_\varepsilon^2 + \sigma_u^2$. This is a multivariate normal distribution and each unit i is realization from this distribution. Let $\theta = (\alpha, \beta, \gamma)$ and $\mathbf{x}_i = (1, x_i, z_i)$, then we have

$$L(\theta, \sigma_u^2, \sigma_\varepsilon^2|y) = \sum_{i=1}^{N} -\frac{1}{2}\log(\det(\Sigma_i)) - \frac{1}{2}(y_i - \theta'\mathbf{x}_i)'\Sigma_i^{-1}(y_i - \theta'\mathbf{x}_i),$$

which we can simplify a bit further using some tedious math

$$\begin{aligned}
\det(\Sigma_i) &= (T\sigma_\alpha^2 + \sigma_\varepsilon^2)(\sigma_\varepsilon^2)^{T-1} \\
\frac{1}{2}\log(\det(\Sigma_i)) &= \frac{1}{2}\log(T\sigma_\alpha^2 + \sigma_\varepsilon^2) + \frac{T-1}{2}\log(\sigma_\varepsilon^2) \\
\frac{1}{2}(y_i - \theta'\mathbf{x}_i)'\Sigma_i^{-1}(y_i - \theta'\mathbf{x}_i) &= \frac{1}{2}[(\varepsilon_i - \omega\bar{\varepsilon}_i)/\sigma_\varepsilon]'[(\varepsilon_i - \omega\bar{\varepsilon}_i)/\sigma_\varepsilon] \\
\omega &= 1 - \frac{\sigma_\varepsilon}{\sqrt{T\sigma_\alpha^2 + \sigma_\varepsilon^2}} \\
\bar{\varepsilon}_i &= \frac{1}{T}\sum_{t=1}^{T} y_{it} - \theta'\mathbf{x}_{it}.
\end{aligned}$$

Under the RE assumptions, this is the correct likelihood function and will be consistent and asymptotically efficient. However, to the extent that you think the RE assumptions are not true, you can (should?) use the clustered covariance matrix on top of this.

(Pick up here) The second approach is to non-parametrically estimate all the unit-specific

(invariant) heterogeneity by fitting a model of the form

$$y_{it} = \delta_i + \beta' x_{it} + \varepsilon_{it}.$$

Here $\delta_i = \alpha_i + \gamma' z_i + u_i$ includes all of the invariant features of unit i and does not impose any assumptions on u_i. This approach, called the fixed-effects model, avoids the RE's extra assumptions on what the within-unit correlation looks like. The cost here is that we have added N new parameters to estimate, which can get iffy as N increases. We can fit this model using either OLS or the normal pseudo-likelihood estimator (they're still the same) with clustered standard errors.

3 Poisson GLMs, related models, and numeric optimization

Count data are similar to continuous data. These are weakly positive values that are unbounded above $\{0, 1, 2, \ldots\}$. The most common way to consider count data is do use the Poisson distribution. The Poisson has a single parameter λ which reflects both expected number of events within an observation period and the variance of this number. Having mean equal the variance is an odd quirk that we'll discuss more later. These could be the number of terrorist attacks each year, the number of wrongful convictions in a given year, or the number of Prussian soldiers killed by accidental horse kicks.

This basics are

$$\begin{aligned} f(y|\lambda) &= \frac{\lambda^y \exp(-\lambda)}{y!}, \quad y = 0, 1, 2, \ldots \\ &= \frac{\lambda^y \exp(-\lambda)}{\Gamma(y_i + 1)}, \quad y = 0, 1, 2, \ldots \\ \mathrm{E}[y] &= \mathrm{Var}(y) = \lambda. \end{aligned}$$

For reasons that will become clear later, we will replace the factorial with the gamma function, Γ. The gamma function generalizes factorials to non-integers (and other things), but for positive integers $\Gamma(y) = (y-1)!$.

To make the GLM, we impose a link function on the CEF

$$\begin{aligned} \mathrm{E}[y_i|X] &= \lambda_i = \exp(x_i'\beta). \\ y_i|X &\overset{iid}{\sim} \text{Poisson}(\exp(x_i'\beta)) \end{aligned}$$

How do we interpret β here? It's actually the same as in the log-normal! So no major points

of difference here.

Let $\odot$ represent element-by-element multiplication, then we can write the log-likelihood

$$\begin{aligned} L(\beta|y) &= \sum_{i=1}^{N} y_i(x_i'\beta) - \exp(x_i'\beta) - \log\Gamma(y_i+1) \\ &\propto \sum_{i=1}^{N} y_i(x_i'\beta) - \exp(x_i'\beta) \\ &\propto \mathbf{1}'(y \odot X\beta - \exp(X\beta)), \end{aligned}$$

where $\mathbf{1}$ is an $N \times 1$ vector of all ones, and thus serves as a summation operator, The score

$$\begin{aligned} s(\beta|y) &= \sum_{i=1}^{N} x_i'y_i - x_i'\exp(x_i'\beta) \\ &= \sum_{i=1}^{N} x_i'(y_i - \exp(x_i'\beta)) \\ &= \mathbf{1}'(X \odot (y - \exp(X\beta))), \end{aligned}$$

and the Hessian

$$\begin{aligned} H(\beta|y) &= \sum_{i=1}^{N} -x_i x_i' \exp(x_i'\beta) = -X'\Omega X \\ \Omega &= I\exp(X\beta) \\ \mathrm{Var}(\hat{\beta}) &= (X'\hat{\Omega}X)^{-1} = (\tilde{X}'\tilde{X})^{-1} \\ \tilde{X} &= \left[1\sqrt{\exp(X'\beta)} \quad X_1 \odot \sqrt{\exp(X'\beta)} \quad \ldots \quad X_K \odot \sqrt{\exp(X'\beta)}\right], \end{aligned}$$

where I is an $N \times N$ identity matrix.

Note that these are remarkably similar to the normal model. The score takes the form of $\sum x_i'(y_i - \mathrm{E}[y_i|x_i])$. The difference here is just in what form the expected value takes. Likewise the Hessian is very similar in that we get $\mathrm{Var}(y_i|X)X'X$. In the normal case $\mathrm{Var}(y_i|X) = \sigma^2$, in the Poisson case $\mathrm{Var}(y_i|X) = \lambda = \exp(x_i'\beta)$. Indeed we get something back that looks like a GLS covariance matrix. However, we have a problem here, we no longer have a closed form solution to the estimation problem. This means, that we need to learn a little bit about numerical optimization before we can make progress.

3.1 Numerical methods and computational tools

At this point we'd really love to work with some real data. However, there is still one fly in the ointment: we don't have solutions for some of these problems. We know how to find the MLE for the normal and log-normal GLMs because closed form solutions exist. However, we do not have solutions for the Poisson or negative binomial. This means we'll have to use numerical/computation methods to solve the optimization problem.

There are more optimization algorithms than I can shake a stick at and we could spend several semesters on just optimization. However, that's not why we're here so we're going to focus, at least for now, on the approaches that will do you the most good/are most commonly used.

In each case that we're going to consider, we'll focus on how to solve the FOC. This will generally be a good approach if you have a well-behaved likelihood that satisfies the regularity conditions we've covered. If you find yourself in a different situation (e.g., you have a discontinuous likelihood or score function), then you may need to explore other methods. But such things exist, so don't lose heart.

3.1.1 Bisection (skip for time)

We'll begin with one of the most basic, but most powerful methods of finding the zero point of an equation: **bisection** This is an incredible fast and efficient algorithm. Unfortunately it only really works efficiently for single parameter problems. So we're going to focus on $s(\theta|y) = 0$ for a singleton θ. We'll also rely on the compactness of Θ here by placing bounds on the problem such that we believe $\theta \in [a, b]$.

Under our regularity conditions, s will be a continuous on a compact set. As such, the intermediate mapping theorem tells us solving this equation means finding values $a < b$ such that $\text{sign}(s(a)) \neq \text{sign}(s(b))$, why?

1. Continuity guarantees that if the sign changes from a to b then s has crossed 0.
2. For any a and b that we find, we can shrink our searching interval (a, b) and repeat until we get arbitrarily close to the zero point.

Algorithm 1 lists the procedure more formally. We start with an interval $[a, b]$ and set our current guess of $\hat{\theta}$ to the mean of this interval $\hat{\theta}_k = (a + b)/2$. We then shrink our interval to be either $[a, \hat{\theta}_k]$ or $[\hat{\theta}_k, b]$ depending on whether $\text{sign}(s(\hat{\theta}_k)) \neq \text{sign}(s(a))$ or not.

The good news is that bisection is completely fool-proof and can be very quick. However, you do have to know what initial interval make sense for your problem which may be more

Algorithm 1: Bisection algorithm

Input: A continuous, one-dimensional function `f`; interval points $a < b$ where $\texttt{sign}(\texttt{f}(a)) \neq \texttt{sign}(\texttt{f}(b))$; a convergence tolerance $\varepsilon > 0$.
Output: A root x^* such that $|\texttt{f}(x^*)| < \varepsilon$

```
while b − a > ε do
    x ← (a + b)/2
    if sign(f(x)) == sign(a) then
        a ← x
    else
        b ← x
return x
```

or less reasonable. The R function `optimize` implements a slightly fancier version of this.

How quick is it? Let's consider that with each step we cut the interval under consideration in half, such that after k iterations the remaining interval size is

$$\frac{b_0 - a_0}{2^k},$$

where the 0 subscript denotes the original search interval. Given that x^* could be at either end of the interval, this also provides an upper bound on the error:

$$|\hat{\theta}_k - \hat{\theta}| = \frac{b_0 - a_0}{2^k} \leq \epsilon,$$

which we want to be less than our tolerance ϵ. We can rearrange terms such that for a fixed ϵ

$$\begin{aligned} \log(b_0 - a_0) - N\log(2) &\leq \log(\epsilon) \\ k &\geq \frac{\log(b_0 - a_0) - \log(\epsilon)}{log(2)} \\ k &\geq 1.44\left[\log(b_0 - a_0) - \log(\epsilon)\right]. \end{aligned}$$

If we set $\epsilon = 1e - 5$ we get

$$k \geq 1.44\log(b_0 - a_0) + 16.6,$$

so if the original interval is length 1, we need *at least* 17 iterations to get to convergence, but it could take more. But, because it's increasing in logged distance between a_0 and b_0, it doesn't get too bad as that initial interval increases. If it's 1,000,000, we're looking at at least 37 iterations.

One question that comes up is what happens if you don't have a good idea about what the boundry points should be? We can answer that with the following bracketing algorithm:

Algorithm 2: BRACKETING

Input: A smooth, differentiable function `f`; starting value for the low end of the bracket a_0; an initial length for the interval $\epsilon > 0$; a step size to increase the interval by k.
Output: An interval (a, c) that contains a local minimum of f

$a \leftarrow x_0$
$b \leftarrow a_0 + \varepsilon$
$f_a \leftarrow \texttt{f}(a)$
$f_b \leftarrow \texttt{f}(b)$
if $fb > fa$ **then**
    $b \leftarrow x_0$
    $a \leftarrow a_0 + \varepsilon$
    $f_a \leftarrow \texttt{f}(a)$
    $f_b \leftarrow \texttt{f}(b)$
    $\varepsilon \leftarrow -\varepsilon$
while *true* **do**
    $c \leftarrow b + \varepsilon$
    $f_c \leftarrow \texttt{f}(c)$
    if $f_c > f_b$ **then**
        return $\texttt{sort}(a, c)$
    else
        $a \leftarrow b$
        $b \leftarrow c$
        $f_b \leftarrow f_c$
        $s \leftarrow sk$

The intuition here is that once we find three points $a < b < c$ where $f(b) < f(a)$ and $f(b) < f(c)$, then a local minima must be between b and c.

Let's an example. Suppose we have $y_i, \ldots, y_N$ are iid from a Poisson(λ) and we're too lazy to recall that the MLE for λ is the sample mean. The point here is to illustrate how close we can get to the true solution of $\hat{\lambda} = \bar{y}$. The likelihood function is

$$L(\lambda|y) = \sum_{i=1}^{N} y_i \log(\lambda) - \lambda - log(y!),$$

with score

$$s(\lambda|y) = -N + \frac{1}{\lambda} \sum_{i=1}^{N} y_i.$$

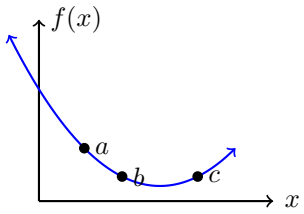


Figure 3.1: Illustrating the bracketing setup

We can use the bracketing step to find an initial search range and then go from there.

```
## Generate data
set.seed(1)
y <- rpois(1000, lambda = 5)
l.mle <- mean(y) #true MLE
print(l.mle)
```

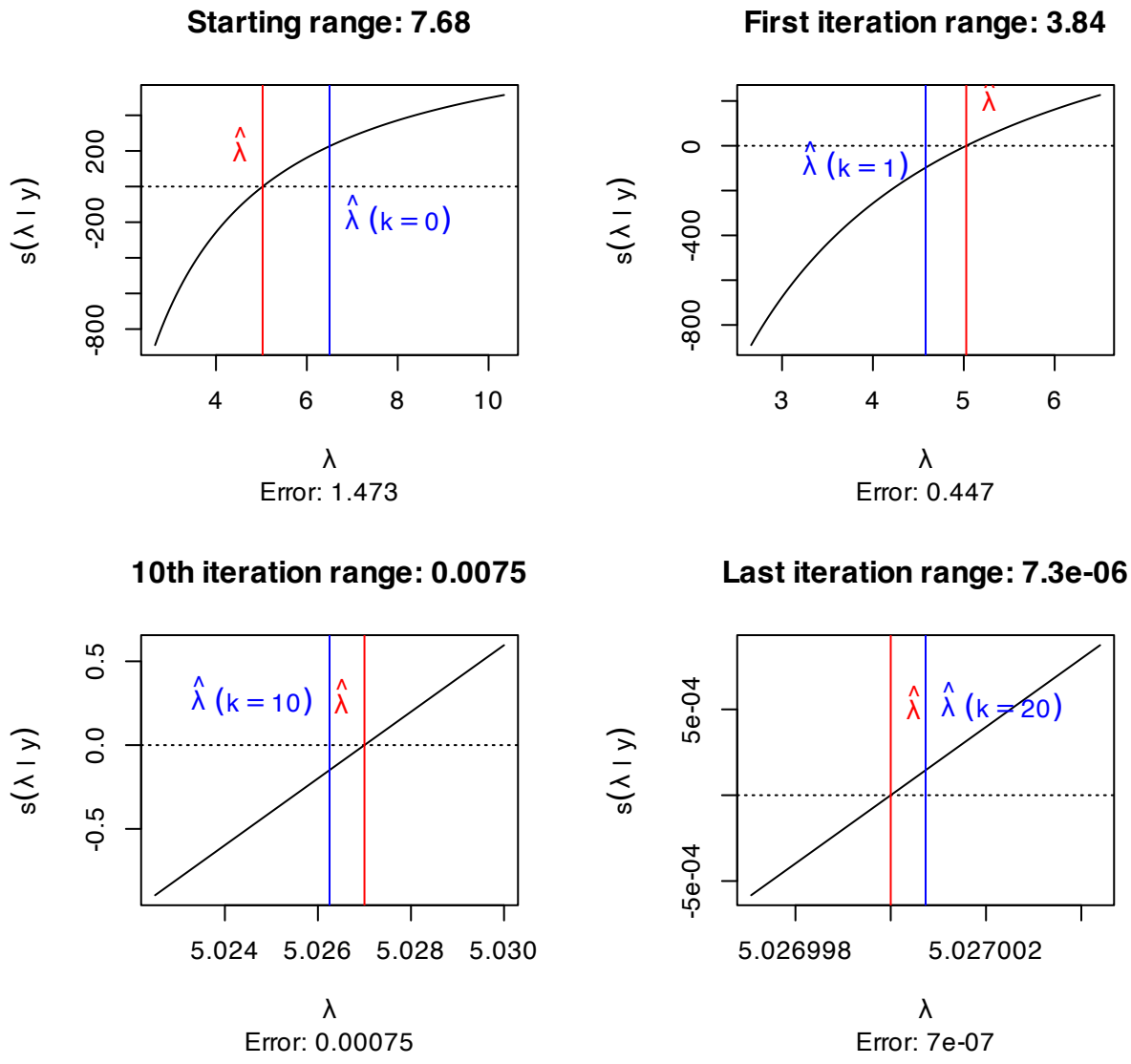
```
## [1] 5.027
```

```
l <- function(lam,y){
  N <- length(y) # length 1
  rump <- sum(-lfactorial(y))# length 1
  sumy <- sum(y)# length 1

  ## because we've condensed everything
  ## related to the data a length 1 vector
  ## we can pass a vector of s2 guesses
  ## of any length and get the score for
  ## each one of them all at once
  LL <- -N*lam + log(lam)*sumy + rump
  return(-LL)
}

s <- function(lam,y){
  N <- length(y) # length 1
  sumy <- sum(y)# length 1

  ## because we've condensed everything
  ## related to the data a length 1 vector
```

```
  ## we can pass a vector of s2 guesses
  ## of any length and get the score for
  ## each one of them all at once
  score <- -N +sumy/lam
  return(-score)
}



bracket <- function(f, a=0, h=1e-2){
  ya <- f(a)
  b <-   a + h
  yb <-  f(b)
  if (yb > ya){
    b <- a
    a <- a + h
    yb <- ya
    ya <- f(a)
    h = -h
  }

  yc <- -Inf
  while( TRUE){
    c <- b + h
    yc <- f(c)
    if(yc > yb){
      break
    }
    a <- b
    b <- c
    yb <- yc
    h <- h*2
  }
  return(sort(c(a, c)))
}
```

```
ab <- bracket(\(x){l(x, y=y)},a=.1)
print(ab)
```

```
## Set up the search
tol <- diff(ab)# initial gap for tolerance check condition
k <- 0 # iteration counter
eps <- 1e-5 # acceptable gap

while(tol> eps){ #bisection
  x <- mean(ab)
  check <- diff(sign(s(c(ab[1],x),y)))
  if(check==0){
    ab[1] <- x
  }else{
    ab[2] <-x
  }
  tol <- diff(ab)
  k <- k+1
}
cat(c("True MLE: ", round(l.mle, 4),
      " Numeric MLE: ", round(x, 4), "\n"))
cat("Final tolerance: ", round(tol, 6), "\n")
cat("Number of iterations: ", k, "\n")

optimize(f = l,
         interval = c(2,11),
         y=y)
```



3.1.2 Newton's method

Bisection is really good when you can actually use it, but it's not that often that you have only a single parameter to worry about. For most of the cases we'll consider this semester we'll be using method based on work by Newton (yes that Newton).

Following past conventions, everything in this section is presented in terms of minimizing a function. This reflects nearly all optimization textbooks and most software implementations. As such, our problem is now

$$\hat{\theta} = \underset{\theta}{\operatorname{argmin}}(-L(\theta|y)).$$

We are now looking to minimize -1 times the log-likelihood as this is the same as maximizing the log-likelihood.

For ease let's rewrite our functions in negative form

$$\begin{aligned}\ell(\theta|y) &:= -L(\theta|y)\\ r(\theta|y) &:= -s(\theta|y)\\ h(\theta|y) &:= -H(\theta|y)\end{aligned}$$

In dealing with multi-dimension optimization, many methods rely on a framework that looks something like:

1. Create a local approximation that is either linear or quadratic (e.g., a first or second order Taylor approximation) at the current guess θ_k
2. Compute a **descent direction** d_k using 1st and/or 2nd derivative information. This will be a vector of length θ. Each element in d_k reflects whether we want to increase or decrease the corresponding element in θ
3. Select a step size α_k. This is a scalar that reflects how far we want to move the elements of θ in direction d_k.
4. Compute the next guess $\theta_{k+1} = \theta_k + \alpha_k d_k$.
5. Repeat until convergence

So now we just need to find methods for finding d_k and α_k that we like.
For ease, we can start with $\alpha_k = 1$. This means we only need to choose d_k. Here we'll take a step back and look at step 1 of the above list. Let's write out the second-order Taylor approximation around θ_k

$$\ell(\theta|y) \approx \ell(\theta_k|y) + r(\theta_k|y)'(\theta - \theta_k) + \frac{1}{2}(\theta - \theta_k)'h(\theta_k|y)(\theta - \theta_k).$$

Solving the FOC for θ we get

$$\theta_{k+1} = \theta_k \underbrace{-h(\theta_k|y)^{-1}r(\theta_k|y)}_{d_k}.$$

Iterating this updating step until it reachs a pre-specified tolerance gives is **Newton's method,** or the **Newton-Raphson** algorithm, with a more formal version presented in Algorithm 3.

The intuition is that we find the linear tangent (slope) of r and follow that until you get to the zero point, make this your next guess and repeat. In this sense, the method is a type of "predictor-corrector" method. Consider finding the root of $f(x) = x^2$ starting at $x_0 = 1$. The first step is shown in Figure 3.2

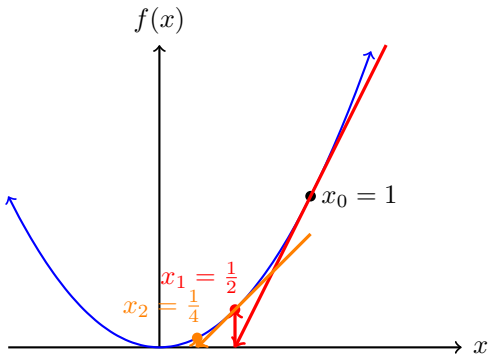
Algorithm 3: NEWTON-RAPHSON ALGORITHM

Input: A smooth, differentiable, and possible vector-valued function `f`; a Jacobian function `J`; a starting value x_0, a convergence tolerance $\varepsilon > 0$; and the maximum number of iterations to try K.

Output: A root x^* such that $\max |\texttt{f}(x^*)| < \varepsilon$

```
k ← 0
x ← x_0
while max |f(x)| > ε do
    x ← x − J(x)^{-1} f(x)
    k ← k + 1
    if k ≥ K then
        break
return x
```

Figure 3.2: First two Newton step when $f(x) = x^2$ and $x_0 = 1$



Note that in this case, like with bisection, we are actually just looking for roots of the first order conditions. So to translate Algorithm 3, the function we're solving the roots for f is the negative score function r and it's Jacobian is the negative Hessian h. Note that we set a maximum number of iterations, this is because Newton's method can be sensitive to starting values. What this means is that while Newton's method can work well in a lot of cases, it can sometimes jump the rails.

We're going to continue with the normal data, but now we are going to estimate both the mean and variance numerically. But one new wrinkle emerges. In the previous example we set bounds on the parameters values ahead of time. Newton methods, however, are designed to be unconstrained.

What will happen if it tries to consider $\sigma^2 \leq 0$ in a normal GLM or $r < 0$ in the negative binomial? Without changing anything, we can see that this could produce `NA` values in the

likelihood while taking the score function into territory we don't want to be in.

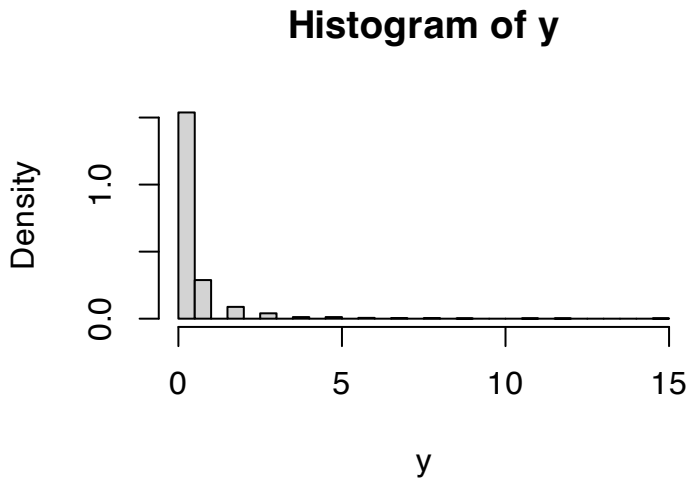
To solve this problem, we typically rewrite our likelihood equation to estimate $\eta = \log(\sigma^2)$ instead of σ^2 and $\alpha = \log(r)$ instead of r. The logged value can be either positive or negative so we don't have to rely on the algorithm just never going to the impossible area. And by the invariance property, A2, we know that the MLE of σ^2 will be $\exp(\widehat{\log(\sigma^2)})$. This requires rewriting all our functions however, thankfully the chain rule and sympy make this very easy.

Let's consider the case of Poisson regression with some fake data to make sure it works how we want it to.

```
set.seed(1)
N <- 1000
X1 <- rnorm(N)
X2 <- runif(N, -2,2)
beta <- c(-2, 1, -1)
X <- cbind(1, X1, X2)
y <- rpois(N, lambda = exp(X%*%beta))
summary(y)
```

```
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##   0.000   0.000   0.000   0.441   0.000  15.000
```

```
hist(y,freq = FALSE,breaks = "scott")
```



We can then code our functions:

```
l <- function(beta, y,X){
  XB <- X%*%beta
```

```
  lambda <- exp(XB)
  LL <- y*XB - lambda
  return(-sum(LL))
}

r <- function(beta, y,X){
  XB <- drop(X%*%beta)
  lambda <- exp(XB)
  score <- X*(y-lambda)
  return(-colSums(score))
}

h <- function(beta, y,X){
  XB <- drop(X%*%beta)
  lambda <- exp(XB)
  bread <- diag(lambda)
  H <- t(X) %*% bread %*% X
  return(H)
}
```

```
eps <- 1e-5
K <- 50
k <- 0
b <- rep(0, 3) ## inital guess
while(max(abs(r(b,y,X))) > eps){
  b <- b - solve(h(b,y,X)) %*% r(b,y,X)
  k <- k + 1
  b
  if(k>=K){
    break
  }
}
print(b)
```

```
##         [,1]
##    -1.979542
## X1  1.024866
```

```
## X2 -1.011962
```

```
print(r(b,y,X))
```

```
##                          X1           X2
## 2.923108e-06 2.570975e-06 1.152137e-06
```

What are the standard errors for these estimates? We could use either the Hessian or OPG estimates, or combine them for robust standard errors.

```
J <-function(beta, y,X){ #Jacobian, just the score, but no -colsums
  XB <- drop(X%*%beta)
  lambda <- exp(XB)
  score <- X*(y-lambda)
  return(score)
}

Vh <- solve(h(b, y=y,X=X))
Vopg <- solve(t(J(b, y,X)) %*% J(b, y,X))



Vrobust <- Vh %*% solve(Vopg) %*% Vh
SE.hess <- sqrt(diag(Vh))
SE.opg <- sqrt(diag(Vopg))
SE.robust <- sqrt(diag(Vrobust))
round(cbind(SE.hess, SE.opg, SE.robust),3)
```

```
##    SE.hess SE.opg SE.robust
##      0.094  0.090     0.099
## X1   0.046  0.049     0.044
## X2   0.058  0.052     0.064
```

Some small differences, but overall very similar as we might expect here. The main advantages of NR are that it's fairly easy and works well if you have the Hessian and reasonably starting values.

3.1.3 BFGS

The most common Quasi-Newton method is the BFGS algorithm. It is a true work horse of optimization in that it should nearly always be the first thing you try and it'll work very well

like 99% of the time.

Two things are different here vis-a-vis basic Newton. The first is that we're no longer fixing the step size α_k. Instead, we will allow it to change at each optimization step. Allowing for a dynamic step size means that will take larger steps that step moves us a lot closer to a minimum and we'll take smaller steps if we're not sure. For a given value of θ_k and d_k we can find the optimal step size by selecting the value of α_k that produces the highest log-likelihood value:

$$\alpha_k = \min_{\alpha} \ell(\theta_k + \alpha d_k).$$

Note that this is a single parameter optimization, which can often be found very quickly.

The main reason quasi-Newton methods exist is that it is often very hard to code, evaluate, or invert the Hessian. So the common thread among quasi-Newton approaches is to come up with different ways to approximate it. To do this we replace d_k with

$$d_k = -Q_k r(\theta_k|y),$$

. where Q_k approximates the inverse Hessian. One very easy approach is the use the OPG for Q, this has the advantage of always being positive definite, and under the information equality, should be nearly identical to the inverse Hessian at a solution. This approach is called the **BHHH** algorithm. It can work well, but its hard to know how good of an approximation this is at any guesses of θ we make along the way. So it could be quite bad sometimes and we wouldn't know.

The **BFGS** algorithm approximates the inverted Hessian in a more complicated way. We'll skip it in class, but it is here in the notes. The basic idea is that we update an approximate Hessian in a way that should get closer to the truth as we move along. For notational ease define

$$\gamma = r(\theta_{k+1}|y) - r(\theta_k|y)$$
$$\delta = \theta_{k+1} - \theta_k$$

and now set

$$Q^{k+1} = Q^k - \left(\frac{\delta\gamma' Q_k + Q^k\gamma\delta'}{\delta'\gamma}\right) + \left(1 + \frac{\gamma' Q_k \gamma}{\delta'\gamma}\right)\left(\frac{\delta\delta'}{\delta'\gamma}\right),$$

and the update is now

$$\theta_{k+1} = \theta_k - \alpha_k Q_k r(\theta_k|y).$$

The intuition here is to think about how we could update the Hessian by coming up with a way to move it slightly towards the next step. We then see how that update changes if we

pass it through the inverse. Don't worry about remember the nuts and bolts of this, just know it's there when you need it. If you're interested, the algorithm is

Algorithm 4: BFGS ALGORITHM

Input: A smooth, differentiable, function $\mathtt{f}$; a gradient function containing the first derivatives $\mathtt{g}$; a starting value x_0; an initial guess at the inverse Hessian Q_0; a convergence tolerance $\varepsilon > 0$; and the maximum number of iterations to try K.

Output: A root x^* such that $\max |\mathtt{g}(x^*)| < \varepsilon$

$k \leftarrow 0$
$x \leftarrow x_0$
$Q \leftarrow Q_0$
$g \leftarrow \mathtt{g}(x)$
while $\max |g| > \varepsilon$ **do**
  $d \leftarrow -Qg$
  $\alpha \leftarrow \mathtt{minimize}_\alpha(\mathtt{f}(x + \alpha d))$
  $\delta \leftarrow \alpha d$
  $x \leftarrow x + \delta$
  $\gamma \leftarrow \mathtt{g}(x) - g$
  $Q \leftarrow Q - \left(\frac{\delta\gamma' Q + Q\gamma\delta'}{\delta'\gamma}\right) + \left(1 + \frac{\gamma' Q \gamma}{\delta'\gamma}\right)\left(\frac{\delta\delta'}{\delta'\gamma}\right)$
  $g \leftarrow \mathtt{g}(x)$
  $k \leftarrow k + 1$
  if $k \geq K$ **then**
    break
return x

Three things to note here:

1. In this algorithm f is our negative log-likelihood ℓ and g is the negative score r. We do not use h.
2. Within each step we solve the additional 1D minimization problem associated with the step size α. This can be done with bisection using the bracketing algorithm at each step to get bounds.
3. At the end of this we get an estimate of the inverse negative hessian Q. This is an estimate of $\text{avar}(\hat{\beta})$

```
x0 <- rep(0,3) #starting values
names(x0) <- paste0("beta", 0:2)
fit <- optim(x0, #initial guess
             fn = l, #negative log-likelihood function
             gr=r, #gradient/score function
```

```
            y=y, #additional arguments for fn and gr
            X=X, #additional arguments for fn and gr
            method="BFGS", #optimization algorithm
            hessian=TRUE) #return the last guess at the hessian Q^{-1}
fit
```

```
## $par
##     beta0     beta1     beta2
## -1.979543  1.024866 -1.011962
##
## $value
## [1] 357.5014
##
## $counts
## function gradient
##       47       12
##
## $convergence
## [1] 0
##
## $message
## NULL
##
## $hessian
##           beta0     beta1     beta2
## beta0  441.0000  459.9922 -479.3131
## beta1  459.9922  947.3923 -477.4114
## beta2 -479.3131 -477.4114  823.1200
```

```
h(fit$par, y,X) ## pretty close match
```

```
##                     X1        X2
##     440.9999  459.9920 -479.3130
## X1  459.9920  947.3914 -477.4112
## X2 -479.3130 -477.4112  823.1197
```

```
## est, SEs, and common z tests
beta.hat <- fit$par
```

```
SE <- sqrt(diag(solve(fit$hessian)))
round(cbind(Est = beta.hat,
            SE = SE,
            z.stat= beta.hat/SE,
            p.val = 2*pnorm(abs( beta.hat/SE), lower=FALSE)),
      4)
```

```
##           Est     SE   z.stat p.val
## beta0 -1.9795 0.0943 -20.9956     0
## beta1  1.0249 0.0463  22.1216     0
## beta2 -1.0120 0.0576 -17.5592     0
```

We can compare this to the build in command for Poisson regression, `glm`.

```
df1 <- data.frame(y=y, X1=X1, X2=X2)
fit1 <- glm(y~X1+X2,
            data=df1,
            family=poisson)
summary(fit1)
```

```
##
## Call:
## glm(formula = y ~ X1 + X2, family = poisson, data = df1)
##
## Coefficients:
##             Estimate Std. Error z value Pr(>|z|)
## (Intercept) -1.97954    0.09428  -21.00   <2e-16 ***
## X1           1.02487    0.04633   22.12   <2e-16 ***
## X2          -1.01196    0.05763  -17.56   <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for poisson family taken to be 1)
##
##     Null deviance: 1557.75  on 999  degrees of freedom
## Residual deviance:  668.65  on 997  degrees of freedom
## AIC: 1221.6
##
```

```
## Number of Fisher Scoring iterations: 6
```

```
## with robust standard errors
library(sandwich)
library(lmtest)
```

```
## Loading required package: zoo
```

```
##
## Attaching package: 'zoo'
```

```
## The following objects are masked from 'package:base':
##
##     as.Date, as.Date.numeric
```

```
VR <- vcovHC(fit1, type="HC0")
coeftest(fit1,VR)
```

```
##
## z test of coefficients:
##
##              Estimate Std. Error z value  Pr(>|z|)
## (Intercept) -1.979542   0.099431 -19.909 < 2.2e-16 ***
## X1           1.024866   0.044350  23.108 < 2.2e-16 ***
## X2          -1.011962   0.064033 -15.804 < 2.2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

The final thing I want to stress is the use of analytic derivatives like we've been using over numerical approximations. When possible, we always provide a function for the first derivative. If we don't, R will approximate this derivative using a method called finite differences. This can be okay, but it can also result in slower code and less precise estimates.

Recall the definition of a derivative as

$$D_x f(x) = \lim_{h \to 0} \frac{f(x+h) - f(x)}{h}.$$

Finite difference (FD) approximations just choose a small value of h and plug it in,

```
library(numDeriv)
grad(l,x0, y=y,X=X) #finite differences
```

```
## [1]  559.0000 -471.6402  438.1017
```

```
r(x0,y,X)
```

```
##                   X1        X2
##  559.0000 -471.6402  438.1017
```

Small differences, but basically right. The problems though are that small differences can become larger differences and as the problem gets larger, FD can take more and more time.

```
system.time(replicate(1000,
                      grad(l,x0, y=y,X=X,
                           method="simple",
                           method.args=list(eps=1e-5))))
```

```
##    user  system elapsed
##   0.089   0.003   0.094
```

```
system.time(replicate(1000,r(x0,y,X)))
```

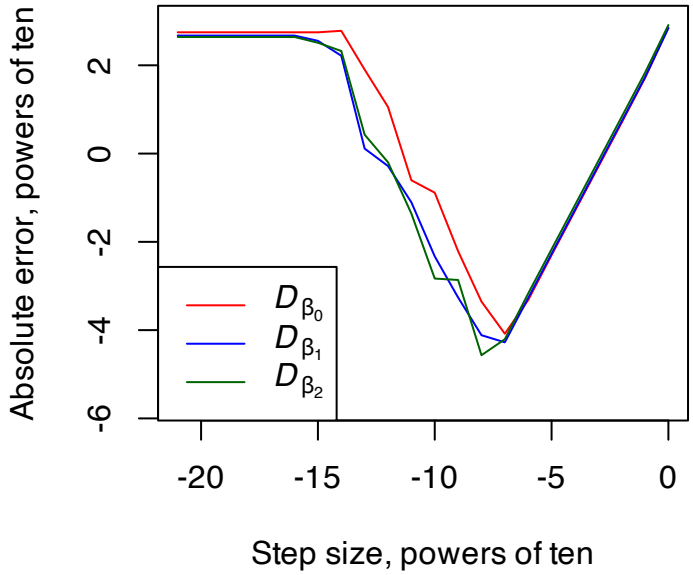
```
##    user  system elapsed
##   0.029   0.000   0.030
```

In this case, roughly a factor of 20.

The other thing is how to choose h? Theoretically, smaller is better, but computationally weird things can happen if you get closer and closer to dividing by zero. For example

```
powers <- seq(-21,0, by=1)
err <- matrix(0, 22, 3)
true <- r(c(0,0,0), y,X)
for(i in 1:22){
  h <- 1*10^(powers[i])
  FD <- grad(l,x0, y=y,X=X,
             method="simple",
             method.args=list(eps=h))
  err[i, ] <- c(abs(FD-true))
}
plot(log(abs(err[,1]),base=10)~powers,
     col="red", type="l",
     ylab="Absolute error, powers of ten",
```

```
     xlab="Step size, powers of ten",
     ylim=c(-5.7,3))
lines(log(abs(err[,2]),base=10)~powers, col="blue")
lines(log(abs(err[,3]),base=10)~powers, col="darkgreen")

legend("bottomleft",expression(italic(D[beta[0]]),
                               italic(D[beta[1]]),
                               italic(D[beta[2]])),
       col=c("red", "blue", "darkgreen"), lty=1)
```



In this case we get the lowest error around $h = 1 \times 10^{-7}$, but who knows if that will typically be right. Experience suggests that between 1×10^{-10} and 1×10^{-5} tends to be a good spot, but writing the function is better. Finite differences are very good for testing your coded derivatives though, because they should match closely as in.

Congratulations! you're now ready for the real world of regression modeling with MLE!

3.2 Back to the Poisson

Returning to the Poisson model recall the basics:

$$y_i|X \overset{iid}{\sim} \text{Poisson}(\exp(x_i'\beta))$$
$$\text{E}[y_i|X] = \lambda_i = \exp(x_i'\beta).$$

This makes the log-likelihood

$$\begin{aligned} L(\beta|y) &= \sum_{i=1}^{N} y_i(x_i'\beta) - \exp(x_i'\beta) - \log\Gamma(y_i+1) \\ &\propto \sum_{i=1}^{N} y_i(x_i'\beta) - \exp(x_i'\beta), \end{aligned}$$

with score

$$\begin{aligned} s(\beta|y) &= \sum_{i=1}^{N} x_i' y_i - x_i' \exp(x_i'\beta) \\ &= \sum_{i=1}^{N} x_i'(y_i - \exp(x_i'\beta)). \end{aligned}$$

We can now estimate β using BFGS.

Once we do that, note that even though $\hat{\beta}$ is interpreted the same as in the log-normal, the AME estimate is slightly different here because of that variance term is no longer in the expected value:

$$\begin{aligned} AME(X_1) &= D_{X_1}\,\mathrm{E}[y_i|X] \\ &= \beta_1 \exp(x_i'\beta) \\ \widehat{AME(X_1)} &= \hat{\beta}_1 \frac{1}{N} \sum_{i=1}^{N} \exp(x_i'\hat{\beta}) \end{aligned}$$

I said we'd come back to the Poisson quirk where the variance equals the mean. There are two things to mention here. First, there is some heteroskedasticity built into the Poisson, each observation has its own variance term. Second, its a very specific form of heteroskedasticity where, for each observation, the variance equals the conditional mean.
Is this likely? Hard to say, but consider a count of terrorist attacks within a year. If it is the case that groups deciding to commit an attack are more likely to commit a 2nd, which makes them more likely to do a 3rd, and so on, then this positive feedback means that the variance will exceed the mean. This is called **overdispersion**. What do we do then?

One option is to accept that this is a distributional violation and use robust standard errors. We've had mixed opinions about this option. We know it's pretty good in the normal and log-normal cases with non-spherical errors. But we've also said there are cases where we're getting correct standard errors for an estimate with unknown bias. So which case are we in here?

Let's find out. Suppose that $y|X$ has an arbitrary distribution such that $y_i \geq 0$ and $\mathrm{E}[y_i|X] = \exp(x_i'\beta_0)$. We want to know what happens if we estimate β using Poisson

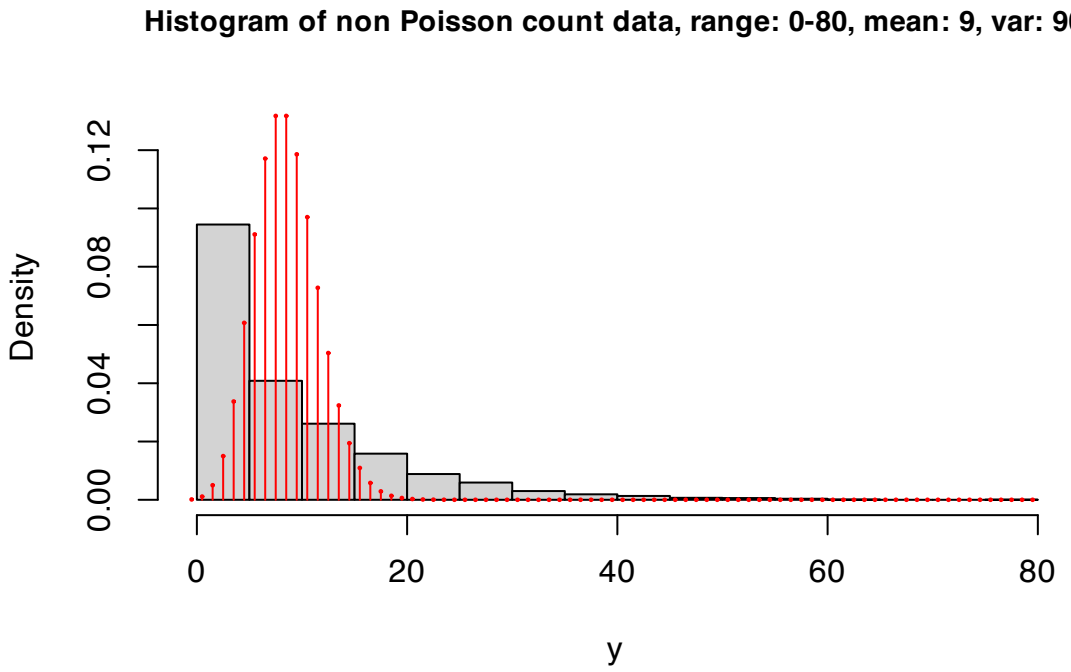


Figure 3.3: Overdispersed data

regression.

From our previous work, we know that $\hat{\beta}$ will converge to a value of β that maximizes the expected value of the incorrect likelihood over the true likelihood, $\mathrm{E}_g[\log f(y|\beta)]$, where g is the true likelihood. Let's (a) assume there is a unique such value, and (b) find out if that value is β_0. Starting with the sample likelihood

$$\begin{aligned}
L_N(\beta|y) &= \frac{1}{N}\sum_{i=1}^{N} y_i(x_i'\beta) - \exp(x_i'\beta) \\
\mathrm{E}[L_N(\beta|y)] &= \frac{1}{N}\sum_{i=1}^{N} \mathrm{E}_X[\mathrm{E}_g[y_i|X](x_i'\beta) - \exp(x_i'\beta)] \\
&= \frac{1}{N}\sum_{i=1}^{N} \mathrm{E}_X[\exp(x_i'\beta_0)(x_i'\beta) - \exp(x_i'\beta)] \\
\mathrm{E}[s_N(\beta|y)] &= \frac{1}{N}\sum_{i=1}^{N} \mathrm{E}[x_i'(\exp(x_i'\beta_0) - \exp(x_i'\beta))].
\end{aligned}$$

When will the score equal 0? If we set $\beta = \beta_0$. So β_0 is one solution here, and we assumed there was only one. If that assumption is correct then we've shown what we needed to show. So, for any weakly positive y with $\mathrm{E}[y_i|X] = \exp(x_i\beta)$, Poisson regression is consistent for β. This is true regardless of whether y is a count or not.

This is an example of a *quasi-likelihood* (QL) estimator. A quasi-likelihood is different from a

pseudo-likelihood in that with a QL, we know the distribution is actually wrong (not just a simplification), but we use it anyway because we have shown it has good properties. Note that we never mentioned the variance in the above derivation. So this tells us that even when the equal variance assumption is wrong, the estimates will be consistent. As a result, the quasi-Poisson estimator should always be paired with the robust variance matrix.

The `glm` function in R has a `quasipoisson` option for the family. It will return the same point estimates as the Poisson regression, but uses a special type of robust standard error that is slightly different than what we saw above.

$$\begin{aligned}
\widehat{\text{avar}}\hat{\beta}_{\text{QP}} &= \hat{\sigma}^2[-H(\hat{\beta}|y)]^{-1} \\
\hat{\sigma}^2 &= \frac{1}{N}\sum_{i=1}^{N}\hat{u}_i^2 \\
\hat{u}_i &= \frac{y_i - \exp(x_i'\beta)}{\sqrt{\exp(x_i'\beta)}} \qquad \text{Pearson/standarized residual}
\end{aligned}$$

This is something between the robust variance we're used to and variance we use when we assume the correct model. This versions assumes that

$$\text{Var}(y_i|X) = \sigma^2\lambda_i.$$

In other words, the conditional variance of y is not the same as it's conditional mean, but it is proportional. The fully robust variance matrix subsumes this and should probably be preferred.

3.3 Negative binomial GLM

Another option in the face of overdispersion is to choose a different distribution. Log-normal is an option, but then we have to do something about 0s and that's can be weird. The normal may be an option, but it might be nice to stick to percentage change given how we described the problem. The main alternative is what's known as the negative binomial.

Negative binomial regression came into popularity because it allows for overdispersion (i.e., it does not have the constant variance assumption). However, as we now know the constant variance assumption isn't that big a deal. Poisson regression with robust standard errors is robust to a range of misspecifications. The negative binomial GLM does not share this general robustness, it is only robust to this one misspecification in the Poisson. As such, it is falling out of fashion. We still cover it, because you may still come across it.

The negative binomial emerges in a few different ways. Classically, we can think of it as counting the number of iid Bernoulli trials needed before $r \in \mathbb{N}$ successes occur. Each trial succeeds with probability $p \in [0, 1]$. The PMF is

$$\begin{aligned} f(y|r,p) &= \binom{y+r-1}{y}(1-p)^y p^r, \quad y = 0, 1, 2, \ldots \\ &= \frac{\Gamma(y+r)}{\Gamma(r)\Gamma(y+1)}(1-p)^y p^r, \quad y = 0, 1, 2, \ldots \\ \mathrm{E}[y] &= \frac{r(1-p)}{p} \\ \mathrm{Var}(y) &= \frac{r(1-p)}{p^2}. \end{aligned}$$

However, given that we want to specify the link function for the conditional mean, we're going to rewrite this a little bit. Let's reparameterize the negative binomial in terms of its expected value $\lambda = \frac{r(1-p)}{p}$, this equivalent to setting $p = \frac{r}{r+\lambda}$. So let's define this new distribution

$$\mathrm{NegBin}_2(\lambda, r) := \mathrm{NegBin}\left(r, \frac{r}{r+\lambda}\right),$$

which we can write as

$$\begin{aligned} f(y|\theta) &= \frac{\Gamma(y+r)}{\Gamma(r)\Gamma(y+1)}\left(\frac{\lambda}{r+\lambda}\right)^y \left(\frac{r}{r+\lambda}\right)^r, \quad y = 0, 1, 2, \ldots \\ \mathrm{E}[y] &= \lambda \\ \mathrm{Var}(y) &= \lambda + \lambda^2/r. \end{aligned}$$

So now we have a model that allows for overdispersion through r. Because $r > 0$, the variance will always be greater than the mean. As $r \to 0$, the data are more overdispersed, while the variance gets closer to the mean as $r \to \infty$ (while allowing $p \to 1$ so that λ stays fixed). In fact, we have

$$\lim_{r\to\infty} \mathrm{NegBin}\left(r, \frac{r}{r+\lambda}\right) = \mathrm{Poisson}(\lambda).$$

The log-likelihood is then

$$\begin{aligned} L(\theta|y) = \sum_{i=1}^{N} & r\log(r) + \beta' x_i y_i - (y_i + r)\log\left(r + \exp(\beta' x_i)\right) \\ & - \log\left(\Gamma\left(y_i + 1\right)\right) + \log\left(\Gamma\left(y_i + r\right)\right) - \log\left(\Gamma\left(r\right)\right), \end{aligned}$$

In estimation, we will estimate $\alpha = \log(r)$ instead of r as we want to be able to search over

positive and negative numbers. The log-likelihood is now

$$L(\theta|y) = \sum_{i=1}^{N} \alpha \exp(\alpha) + \beta' x_i y_i - (y_i + \exp(\alpha)) \log\left(\exp(\alpha) + \exp(\beta' x_i)\right) \\ - \log\left(\Gamma\left(y_i + 1\right)\right) + \log\left(\Gamma\left(y_i + \exp(\alpha)\right)\right) - \log\left(\Gamma\left(\exp(\alpha)\right)\right).$$

For the score, we need to know something about the derivatives of the logged gamma function. Fortunately, these are defined for us the first derivative of the logged gamma function is called the digamma function often written as $\psi(y)$ With this we can get the score

$$s(\beta|y) = \sum_{i=1}^{N} x_i' \left[\frac{y_i \exp(\alpha) - \exp(x_i'\beta + \alpha)}{\exp(\alpha) + \exp(\beta' x_i)} \right]$$

$$s(\alpha|y) = \sum_{i=1}^{N} \exp(\alpha) \left[\alpha - \frac{y_i}{\exp(\alpha) + \exp(\beta' x_i)} - \log\left(\exp(\alpha) + \exp(\beta' x_i)\right) \right.$$
$$\left. + \psi(y_i + \exp(\alpha)) - \psi(\exp(\alpha)) + 1 - \frac{\exp(\alpha)}{\exp(\alpha) + \exp(\beta' x_i)} \right].$$

3.4 Application

Okay we're now ready to actually consider an application!

Here we'll consider a normal, log-normal, Poisson, and negative binomial to protest events in autocracies. This application is based on, but not an exact replication of Escribá-Folch, Meseguer, and Wright (2018; *AJPS*). In this paper, the authors consider the relationship between logged remittances (money sent by migrants aboard back to their home countries) and anti-government protests in autocracies. They create a new measure of propensity to protest using several different protest datasets. We will use just one of these underlying datasets to demonstrate the use of count data.

The outcome of interest, y_{it}, is a count of protests in country i during year t. Remittances, r_{it} are measured in logged US dollars, while autocracies, d_{it}, is a binary variable based on Geddes, et al (2014). Additional controls are denoted as x_{it}. Let

$$\mu_{it} = r_{it}\beta_1 + d_{it}\beta_2 + r_{it}d_{it}\beta_3 + x_{it}'\gamma + \alpha_i + \tau_t,$$

then are models of interest are

Normal:

$$y_{it} \mid r_{it}, x_{it} \overset{iid}{\sim} N(\mu_{it}, \sigma^2)$$

Log-normal (with $c \in \{0.01, 1\}$:

$$y_{it} + c \mid r_{it}, x_{it} \overset{iid}{\sim} \log N(\mu_{it}, \sigma^2)$$

Poisson:

$$y_{it} \mid r_{it}, x_{it} \overset{iid}{\sim} \text{Poisson}(\exp(\mu_{it}))$$

Negative binomial:

$$y_{it} \mid r_{it}, x_{it} \overset{iid}{\sim} \text{NB}(r, r/(r + \exp(\mu_{it})))$$

To recap, how does having remittances in logged for affect our interpretation of the point estimates?

Normal/linear:

$$\begin{aligned}
\mathrm{E}[y_{it}|X] &= r_{it}\beta_1 + \dots \\
\mathrm{E}[y_{it}|X] &= \log(s_{it})\beta_1 + \dots \quad s \text{ is unlogged remittances} \\
D_s \,\mathrm{E}[y_{it}|X] &= \beta_1/s_{it} \\
\beta_1 &= \frac{\partial \,\mathrm{E}[y_{it}|X]}{\partial s_{it}/s_{it}} \qquad \text{denominator is looks like \% change in } s \\
\beta_1/100 &= \frac{\partial \,\mathrm{E}[y_{it}|X]}{100\partial s_{it}/s_{it}} \qquad \text{denominator is looks like \% change in } s
\end{aligned}$$

For the normal model, we have that on average a 1% increase in remittances is associated with a $\hat{\beta}_1/100$ percentage change in protests in democracies, holding everything else fixed.

What about for the others with the exponential CEF?

$$\begin{aligned}
\mathrm{E}[y_{it}|X] &= \exp(\log(s_{it})\beta_1 + \dots) \\
D_s \,\mathrm{E}[y_{it}|X] &= \beta_1 \,\mathrm{E}[y_{it}|X]/s_{it} \\
\beta_1 &= \frac{\partial \,\mathrm{E}[y_{it}|X]/\,\mathrm{E}[y_{it}|X]}{\partial s_{it}/s_{it}} \qquad \text{both parts look like \% change} \\
\beta_1 &= \frac{\partial \,\mathrm{E}[y_{it}|X]/\,\mathrm{E}[y_{it}|X] \times 100}{\partial s_{it}/s_{it} \times 100}
\end{aligned}$$

For these model, we have that on average a 1% increase in remittances is associated with a $\hat{\beta}_1$ percentage change in protests in democracies, holding everything else fixed.

One thing we haven't discussed yet is the possible concern of omitted variables in the Poisson and negative binomial models. We know in the (log-)normal what the rules are: an omitted

variable does not lead to bias in estimating $\hat{\beta}_j$ if it is ether uncorrelated with the (logged) outcome or with X_j. Does this hold true in these other two models. The short answer is yes! These rules apply to any model where the inverse link function is either linear (normal) or exponential (e.g., log-normal, Poisson, negative binomial). So no new rules to learn here.

The main hypotheses of interest are: Do remittances increase protests within democracies? Within autocracies? and Is there are difference? How would we test these?

$$\begin{aligned} H_0(\text{dem}) &: \beta_1 = 0 \\ H_0(\text{auto}) &: \beta_1 + \beta_3 = 0 \\ H_0(\text{diff}) &: \beta_3 = 0 \end{aligned}$$

A few things I want you to keep in mind as we proceed here:

1. Note that there are plenty of 0s here. This means we need to tweak the outcome variable for the log-normal. This means that we cannot compare it directly to the others. Indeed, if we were naive enough to try, as we'll see, the comparison will be sensitive to what number we choose to add to the outcome variable (e.g., 0.01 versus 1 will produce different comparisons).
2. With the Poisson and negative binomial models with country-fixed effects we will see that there are some countries in the sample that never experience conflict. What is the MLE for their intercept?

To answer this second question, recall the score function for Poisson regression is

$$\begin{aligned} s(\theta; y_{it}) &= \sum_{i=1}^{N}\sum_{t=1}^{T} x'_{it}(y_{it} - \exp(x'_{it}\beta + \alpha_i)) \\ s(\alpha_i; y_i) &= \sum_{t=1}^{T}(-\exp(x'_{it}\beta + \alpha_i)) < 0 \end{aligned}$$

Since the score is strictly negative, we can never set it equal to 0, and it means that the log-likelihood is strictly increasing as we take $\alpha_i \to -\infty$. So the MLE of α_i is $-\infty$. Obviously, a computer will not give us that exactly so we'll get some very large negative number. The exact magnitude of this will depend on our tolerance choices in the BFGS algorithm. These problems do not arise in the normal and log-normal models because there are no "hard" limits on this distributions (i.e., the normal has support on the entire real line and the log-normal is weakly bounded by 0)

Does it matter that we have these arbitrary large estimates? It depends. There are two

additional questions we should ask here:

1. Does this affect the estimate of β?
2. Does this affect the AME?

To answer the former, we consider the Hessian. Specifically, what is the cross-partial derivative $D_{\beta\alpha_i}$.

$$D_{\beta\alpha_i} L(\theta \mid y) = \sum_{t=1}^{T} -x'_{it} \exp(x'_{it}\beta + \alpha_i)$$

$$\lim_{\alpha_i \to -\infty} D_{\beta\alpha_i} L(\theta \mid y) = 0.$$

So as α_i gets closer to its true ML estimate of $-\infty$, the all-zero unit provides increasingly less information on β. The answer to question 1 is thus no.

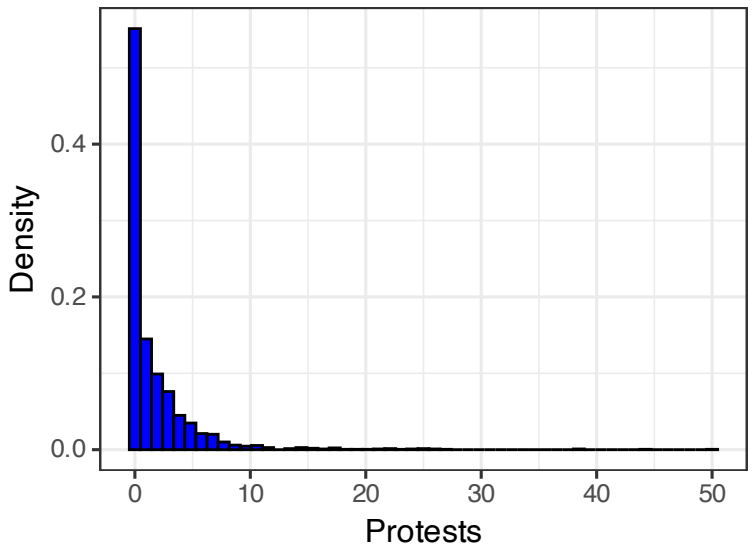
The answer to 2 should be obvious. The only difference between including or excluding these units in the AME is the addition of a bunch of units with infinite intercepts. The marginal effect of adding something to infinity? 0. So we are considering whether to include a bunch of 0s in our average marginal effect or not. The main consideration here may be to ask: Why are these units all 0? Is it because they cannot experience any y? In this case that would be the same as asking are there any countries that cannot experience protests? Probably not

Later on we'll consider other alternatives to the dummy variable approach that may also affect how we think about these marginal effects. For now, however, we press on.

In this application, I'll be using canned routines to show you how you would do this is practice. In your homeworks, I will let you know when you are or are not allowed to use the canned routines. Part of this course is to get you comfortable converting paper math to code, it is a useful skill to have when you come across methods that aren't canned or when you don't know what a canned routine does.

```
## basics
library(readstata13)
library(dplyr)
## econometric
library(lmtest)
library(sandwich)
library(MASS)
library(car)
library(nonnest2)
## tables and figures
```

```r
library(xtable)
library(ggplot2)

rm(list=ls())



protests <- read.dta13("Code/R/datasets/remittances_and_protests1.dta")
protests <- protests %>%
  filter(sample==1) %>%  ## authors' chosen sample
  mutate(sum.y = sum(banks_protest_count), .by=cowcode) %>% ## the infinite cases
  filter(sum.y >0)

## we'll keep everything comparable across models by limiting ourselves to the
## countries with protests




### Looking at our main variables####
ggplot(protests) +
  geom_histogram(aes(x=banks_protest_count, y=after_stat(density)),
                 fill="blue", color="black",
                 bins = nclass.scott(protests$banks_protest_count))+
  xlab("Protests")+
  ylab("Density")+
  theme_bw(12)
```




```
summary(protests$remit)
```

```
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##   4.278  12.865  14.433  14.367  16.230  20.026
```

```
dim(protests)
```

```
## [1] 2268   44
```

```
length(unique(protests$cowcode))
```

```
## [1] 88
```

```
summary(protests$year)
```

```
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##    1976    1987    1997    1995    2004    2010
```

```
#### Fit the models ####

## model formula we'll use today
f1 <- banks_protest_count ~ remit*dict +
  l1gdp + #GDP pc capita
  l1pop+ #population
  l1nbr5 + #neighboring protests
  l12gr+ #Economic growth
  l1migr+ #Net migration
  elec3+ #election
```

```
  factor(period)+
  factor(cowcode)-1

## Normal model
normal <- lm(f1, data=protests, x=TRUE, y=TRUE)
length(normal$resid) ## nothing lost to missingness
```

[1] 2268

```
## log normals
log.normal1 <- lm(update(f1, log(banks_protest_count+1)~.), data=protests,
                  y=TRUE, x=TRUE)
log.normal2 <- lm(update(f1, log(banks_protest_count+0.01)~.), data=protests,
                  y=TRUE, x=TRUE)

## Poisson and NB (glm and MASS::glm.nb)
poi <- glm(f1, data=protests, family=poisson, y=TRUE, x=TRUE)
nbreg <- glm.nb(f1, data=protests,y=TRUE, x=TRUE)

#### housekeeping before comparing estimates ####

##collect the models
mod.list <- list(normal,
                 log.normal1,
                 log.normal2,
                 poi,
                 nbreg)

## remove all the country and year coefficients to make this easier to look at
factors <- grep(names(normal$coefficients),pattern="factor")
Ests <- sapply(mod.list,\(x){x$coefficients[-factors]})
SE <- sapply(mod.list,\(x){sqrt(diag(vcov(x)))[-factors]})



## ordinary Hessian based standard errors
tab1 <- rbind(c(Ests),c(SE)) %>%
```

```
  matrix(., ncol=5) %>%
  round(., 3)
tab1[seq(2, 18,2),] <- paste0("(", tab1[seq(2, 18,2),], ")")

tab1 <- cbind(c(rbind(c("Remit",
                        "Autocracy",
                        "GDP pc",
                        "Pop",
                        "Neighboring protest",
                        "Growth",
                        "Net Migration",
                        "Election",
                        "Remit $\\times$ Autocracy"), "")),

              tab1)
colnames(tab1) <- c(" ",
                    "Normal",
                    "Log-normal (+1)",
                    "Log-normal (+0.01)",
                    "Poisson",
                    "Negative binomial"
)

## model information rows
## NOTE: R calls the r parameter from the negative binomial theta, we'll follow suit,
## but this is the r we discussed above

tab1 <- rbind(tab1,
              c("Observations", round(sapply(mod.list, \(x){length(x$fitted.values)}), 0
              c("Log-Likelihood", round(sapply(mod.list, logLik), 2)),
              c("$\\theta$", rep(" ", 4), round(nbreg$theta,2)))


print(xtable(tab1, align="llccccc"),
      include.rownames=FALSE,
      booktabs=TRUE,
```

```
      sanitize.text.function = \(x){x},
      caption.placement="top")
```

	Normal	Log-normal (+1)	Log-normal (+0.01)	Poisson	Negative binomial
Remit	-0.021	0.008	0.08	0.046	0.043
	(0.081)	(0.017)	(0.061)	(0.021)	(0.039)
Autocracy	-4.757	-1.072	-3.176	-2.139	-1.703
	(1.162)	(0.243)	(0.88)	(0.311)	(0.565)
GDP pc	1.283	0.285	0.861	0.693	0.694
	(0.387)	(0.081)	(0.294)	(0.104)	(0.197)
Pop	1.689	0.327	1.109	0.974	1.155
	(0.887)	(0.186)	(0.673)	(0.256)	(0.461)
Neighboring protest	-2.034	-0.063	0.443	-0.525	0.177
	(0.44)	(0.092)	(0.334)	(0.095)	(0.202)
Growth	-0.044	-0.013	-0.052	-0.025	-0.027
	(0.018)	(0.004)	(0.013)	(0.005)	(0.009)
Net Migration	0.67	0.061	0.073	0.126	0.069
	(0.181)	(0.038)	(0.137)	(0.036)	(0.08)
Election	0.346	0.062	0.232	0.136	0.22
	(0.156)	(0.033)	(0.118)	(0.039)	(0.077)
Remit × Autocracy	0.359	0.079	0.229	0.154	0.121
	(0.079)	(0.017)	(0.06)	(0.02)	(0.038)
Observations	2268	2268	2268	2268	2268
Log-Likelihood	-5680.8	-2135.69	-5052.05	-4265.03	-3431.1
θ					0.85

What can we take away from this? How do we interpret the coefficient on remittances? Using our basic approximations from above, we can say

Normal On average, 1 percent increase in remittances within democracies is associated with 0.0004 fewer protests events, holding everything else constant.

Log-normal On average, 1 percent increase in remittances within democracies is associated with a roughly 0.002 (or 0.05) percent more protests events, holding everything else constant.

Poisson On average, 1 percent increase in remittances within democracies is associated with a roughly 0.05 percent more protests events, holding everything else constant.

Negative binomial On average, 1 percent increase in remittances within democracies is associated with a roughly 0.04 percent more protests events, holding everything else constant.

Do we reject the null hypothesis that the effect of remittances within democracies is 0 in any of these cases? Just the Poisson

Repeating this with the clustered matrices (i.e., admitting to ourselves that these are pseudo likelihoods).

```
#### clustered variance matrices###
Vn <- vcovCL(normal, cluster=protests$cowcode)
Vln <- vcovCL(log.normal1, cluster=protests$cowcode)
Vln2 <- vcovCL(log.normal2, cluster=protests$cowcode)
Vp <- vcovCL(poi, cluster=protests$cowcode)
Vnb <- vcovCL(nbreg, cluster=protests$cowcode)
SE.list <- list(Vn, Vln,Vln2, Vp, Vnb)
SE <- sapply(SE.list,\(x){sqrt(diag(x))[-factors]})

## repeat the tabling process
tab1 <- rbind(c(Ests),c(SE)) %>%
  matrix(., ncol=5) %>%
  round(., 3)
tab1[seq(2, 18,2),] <- paste0("(", tab1[seq(2, 18,2),], ")")

tab1 <- cbind(c(rbind(c("Remit",
                        "Autocracy",
                        "GDP pc",
                        "Pop",
                        "Neighbor protest",
                        "Growth",
                        "Net Migration",
                        "Election",
                        "Remit $\\times$ Autocracy"), "")),

              tab1)
colnames(tab1) <- c(" ",
                    "Normal",
                    "Log-normal (+1)",
                    "Log-normal (+0.01)",
                    "Poisson",
```

```
                    "Negative binomial"
)

tab1 <- rbind(tab1,
              c("Observations", round(sapply(mod.list, \(x){length(x$fitted.values)}), (
              c("Log-Likelihood", round(sapply(mod.list, logLik), 2)),
              c("$\\theta$", rep(" ", 4), round(nbreg$theta,2)))

print(xtable(tab1, align="llccccc"),
      include.rownames=FALSE,
      booktabs=TRUE,
      sanitize.text.function = \(x){x},
      caption.placement="top")
```

	Normal	Log-normal (+1)	Log-normal (+0.01)	Poisson	Negative binomial
Remit	-0.021	0.008	0.08	0.046	0.043
	(0.089)	(0.026)	(0.095)	(0.048)	(0.052)
Autocracy	-4.757	-1.072	-3.176	-2.139	-1.703
	(1.547)	(0.416)	(1.436)	(0.79)	(0.848)
GDP pc	1.283	0.285	0.861	0.693	0.694
	(0.839)	(0.163)	(0.514)	(0.333)	(0.349)
Pop	1.689	0.327	1.109	0.974	1.155
	(1.357)	(0.284)	(1.061)	(0.774)	(0.714)
Neighbor protest	-2.034	-0.063	0.443	-0.525	0.177
	(1.396)	(0.208)	(0.634)	(0.371)	(0.38)
Growth	-0.044	-0.013	-0.052	-0.025	-0.027
	(0.019)	(0.005)	(0.018)	(0.01)	(0.012)
Net Migration	0.67	0.061	0.073	0.126	0.069
	(0.717)	(0.099)	(0.261)	(0.126)	(0.148)
Election	0.346	0.062	0.232	0.136	0.22
	(0.239)	(0.045)	(0.138)	(0.095)	(0.104)
Remit × Autocracy	0.359	0.079	0.229	0.154	0.121
	(0.115)	(0.03)	(0.1)	(0.052)	(0.058)
Observations	2268	2268	2268	2268	2268
Log-Likelihood	-5680.8	-2135.69	-5052.05	-4265.03	-3431.1
θ					0.85

What can we take away from this? The interpretations are unchanged, but the standard errors are much larger.

Moving to the original hypotheses of interest, we can test the 1st and 3rd by just looking at the table. We do see notable differences between democracies and autocracies here. For the remaining one? What do we want to know? We want to combine the coefficients $\beta_{\text{remit}} + \beta_{\text{remit} \times \text{auto}}$.

```
## deltaMethod from the car package is an easy way to
## combine covariates from a fitted model in either
## a linear or nonlinear way. We use the back ticks
## on remit:dict because it has that special symbol :
## the ` ` keeps it together
deltaMethod(normal, "remit+`remit:dict`", vcov=Vn)
```

```
##                       Estimate       SE    2.5 % 97.5 %
## remit + `remit:dict` 0.338208 0.097195 0.147710 0.5287
```

```
deltaMethod(log.normal1, "remit+`remit:dict`", vcov=Vln)
```

```
##                       Estimate       SE    2.5 % 97.5 %
## remit + `remit:dict` 0.087405 0.023474 0.041396 0.1334
```

```
deltaMethod(log.normal2, "remit+`remit:dict`", vcov=Vln2)
```

```
##                       Estimate       SE    2.5 % 97.5 %
## remit + `remit:dict` 0.309050 0.087446 0.137658 0.4804
```

```
deltaMethod(poi, "remit+`remit:dict`", vcov=Vp)
```

```
##                       Estimate       SE    2.5 % 97.5 %
## remit + `remit:dict` 0.200829 0.052075 0.098764 0.3029
```

```
deltaMethod(nbreg, "remit+`remit:dict`", vcov=Vnb)
```

```
##                       Estimate       SE    2.5 % 97.5 %
## remit + `remit:dict` 0.164336 0.056759 0.053092 0.2756
```

What might we conclude here?

What about AMEs?

```
## we can look at these in terms of the logged variable D_r E[y|X]
## or in terms of remittances D_s E[y|X]
## The former
normalAME <- c(normal$coefficients['remit'],
```

```
              normal$coefficients['remit'] + normal$coefficients["remit:dict"])

## matricies for counter factuals
X0 <- X1 <- poi$x
X1[,"dict"] <- 1
X1[,"remit:dict"] <- X1[,"remit"]
X0[,"dict"] <- 0
X0[,"remit:dict"] <- 0

log.normal1.s2.hat <- mean(log.normal1$residuals^2)
lnormalAME1 <- c(log.normal1$coefficients['remit']*
                   exp(log.normal1.s2.hat/2) *
                   mean(exp(X0 %*% log.normal1$coef)),
                 (log.normal1$coefficients['remit'] +
                    log.normal1$coefficients["remit:dict"])*
                   exp(log.normal1.s2.hat/2) *
                   mean(exp(X1 %*% log.normal1$coef)))




log.normal2.s2.hat <- mean(log.normal2$residuals^2)
lnormalAME2 <-  c(log.normal2$coefficients['remit']*
                    exp(log.normal2.s2.hat/2) *
                    mean(exp(X0 %*% log.normal2$coef)),
                  (log.normal2$coefficients['remit'] +
                     log.normal2$coefficients["remit:dict"])*
                    exp(log.normal2.s2.hat/2) *
                    mean(exp(X1 %*% log.normal2$coef)))

poissonAME <-  c(poi$coefficients['remit']* mean(exp(X0%*% poi$coefficients)),
                 (poi$coefficients['remit'] +
                    poi$coefficients["remit:dict"])*
                   mean(exp(X1 %*% poi$coefficients)))

nbAME <-  c(nbreg$coefficients['remit']* mean(exp(X0 %*% nbreg$coefficients)),
```

```
            (nbreg$coefficients['remit'] +
               nbreg$coefficients["remit:dict"])*
              mean(exp(X1 %*% nbreg$coefficients)))

ame.tab <- round(cbind(normalAME, lnormalAME1,lnormalAME2, poissonAME, nbAME), 3)
ame.tab <- cbind( c("Demo", "Auto"), ame.tab)
colnames(ame.tab) <- colnames(tab1)


print(xtable(ame.tab, align="llccccc", caption="AME of logged remittances"),
      include.rownames=FALSE,
      booktabs=TRUE,
      sanitize.text.function = \(x){x},
      caption.placement="top")
```

Table 3.1: AME of logged remittances

	Normal	Log-normal (+1)	Log-normal (+0.01)	Poisson	Negative binomial
Demo	-0.021	0.02	0.511	0.079	0.075
Auto	0.338	0.245	3	0.424	0.334

I'm going to leave the standard errors for these as a homework assignment, but for the non-normal models, this will involve the delta method.

```
## we can look at these in terms of the logged variable D_r E[y|X]
## or in terms of remittances D_s E[y|X]
## The latter
s <- mean(exp(protests$remit)/1000000) ## remittances in millions

normalAME <- c(normal$coefficients['remit'],
               normal$coefficients['remit'] + normal$coefficients["remit:dict"])/s
## matricies for counter factuals
X0 <- X1 <- poi$x
X1[,"dict"] <- 1
X1[,"remit:dict"] <- X1[,"remit"]
X0[,"dict"] <- 0
X0[,"remit:dict"] <- 0
```

```
log.normal1.s2.hat <- mean(log.normal1$residuals^2)
lnormalAME1 <- colMeans(cbind(log.normal1$coefficients['remit']*
                              exp(log.normal1.s2.hat/2) *
                              exp(X0 %*% log.normal1$coef),
                             (log.normal1$coefficients['remit'] +
                                log.normal1$coefficients["remit:dict"])*
                              exp(log.normal1.s2.hat/2) *
                              exp(X1 %*% log.normal1$coef))/s)



log.normal2.s2.hat <- mean(log.normal2$residuals^2)
lnormalAME2 <- colMeans(cbind(log.normal2$coefficients['remit']*
                              exp(log.normal2.s2.hat/2) *
                              exp(X0 %*% log.normal2$coef),
                             (log.normal2$coefficients['remit'] +
                                log.normal2$coefficients["remit:dict"])*
                              exp(log.normal2.s2.hat/2) *
                              exp(X1 %*% log.normal2$coef))/s)

poissonAME <-  colMeans(cbind(poi$coefficients['remit']* exp(X0%*% poi$coefficients),
                             (poi$coefficients['remit'] + poi$coefficients["remit:dict"
                              exp(X1 %*% poi$coefficients))/s)

nbAME <-  colMeans(cbind(nbreg$coefficients['remit']* exp(X0%*% nbreg$coefficients),
                        (nbreg$coefficients['remit'] + nbreg$coefficients["remit:dict"]
                          exp(X1 %*% nbreg$coefficients))/s)

ame.tab <- round(cbind(normalAME, lnormalAME1,lnormalAME2, poissonAME, nbAME), 3)
ame.tab <- cbind( c("Demo", "Auto"), ame.tab)
colnames(ame.tab) <- colnames(tab1)

print(xtable(ame.tab, align="llccccc", caption="AME of a 1 million USD increase in remit
      include.rownames=FALSE,
```

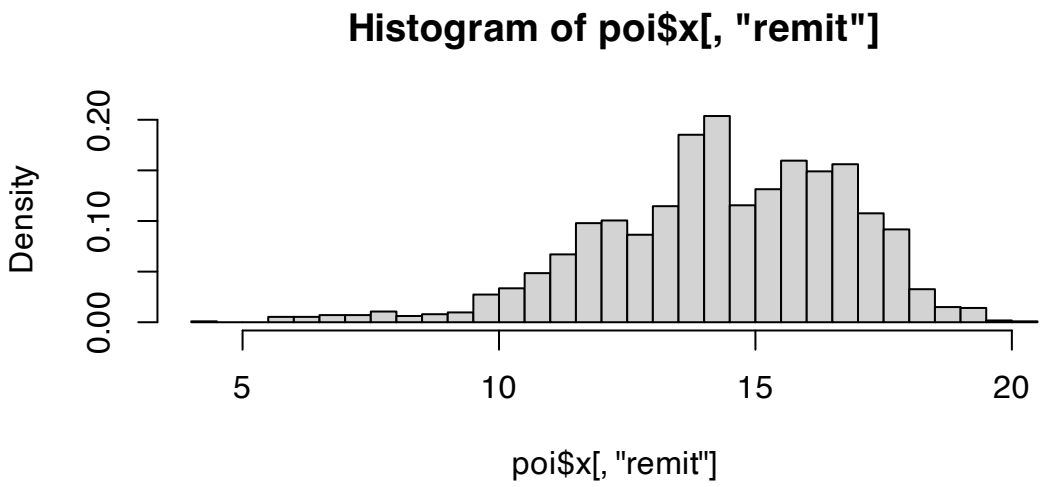
```
      booktabs=TRUE,
      sanitize.text.function = \(x){x},
      caption.placement="top")
```

```
## \begin{table}[ht]
## \centering
## \caption{AME of a 1 million USD increase in remittances}
## \begin{tabular}{lccccc}
##   \toprule
##   & Normal & Log-normal (+1) & Log-normal (+0.01) & Poisson & Negative binomial \\
##   \midrule
## Demo & -0.002 & 0.002 & 0.042 & 0.007 & 0.006 \\
##   Auto & 0.028 & 0.02 & 0.248 & 0.035 & 0.028 \\
##    \bottomrule
## \end{tabular}
## \end{table}
```

If we were interested in predictions, what would that look like? Suppose we want to show our readers how the expected number of protests change as remittances increase? We know how to find the expected number of protests, but we now need to fix other the values of the other covariates. Why? Because each prediction is based on the full profile of data. Some options here

1. One or more countries we're particularly interested in
2. An "average" profile. This typically means setting all over covariates to a mean or median value.

```
library(ggplot2)
## What to vary remittances over?
hist(poi$x[,"remit"],freq = FALSE,breaks = "scott")
```

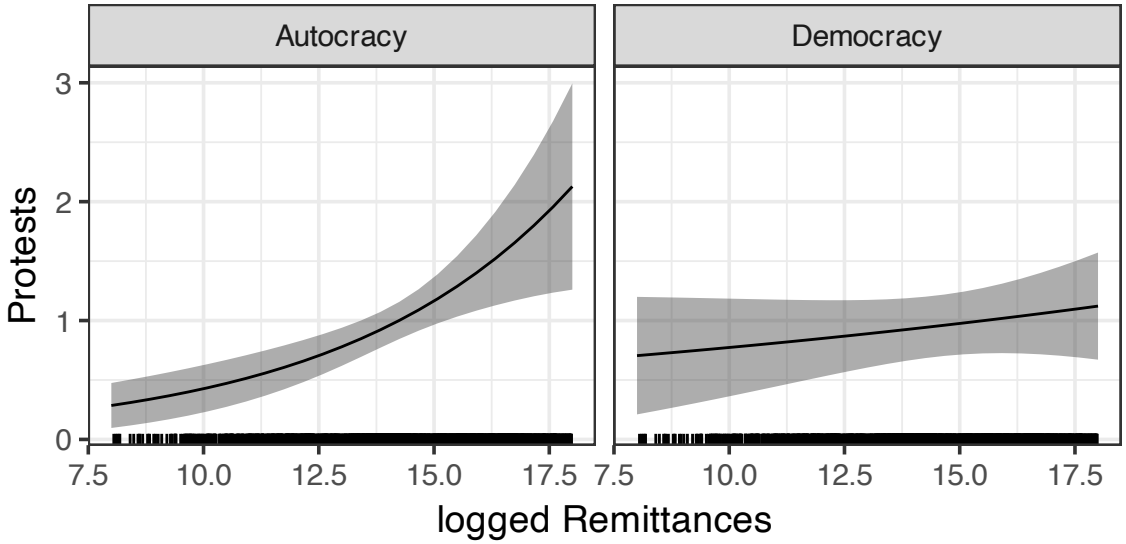



```
quantile(protests$remit, probs=c(0,0.025,.05, .95,.975,1))
```

```
##        0%      2.5%        5%       95%     97.5%      100%
##  4.277961  8.997328 10.184141 17.752887 18.140990 20.025698
```

```
## Create an "average" profile
## whatever that actually means
## I've never been to averagastan, have you?
Xstar <- t(replicate(50, colMeans(poi$x))) ##means for most things
Xstar[,"elec3"] <- median(poi$x) ## one binary covariate we'll set to the median

Xstar[,"dict"] <- c(rep(0,25), rep(1,25)) ## setting regime type to change
Xstar[,"remit"] <- seq(8, 18, length=25) ## setting remittances to vary
Xstar[,"remit:dict"] <- Xstar[,"dict"] *Xstar[,"remit"]

ystar <- drop(exp(Xstar %*% poi$coefficients))

## delta method
DyDb <- Xstar* ystar
V.ystar <- DyDb %*% Vp %*% t(DyDb)
se.ystar <- sqrt(diag(V.ystar))


## create a data frame for plotting
plot.df <- data.frame(Remittances=Xstar[,"remit"],
                      Regime=rep(c("Democracy", "Autocracy"), each=25),
```

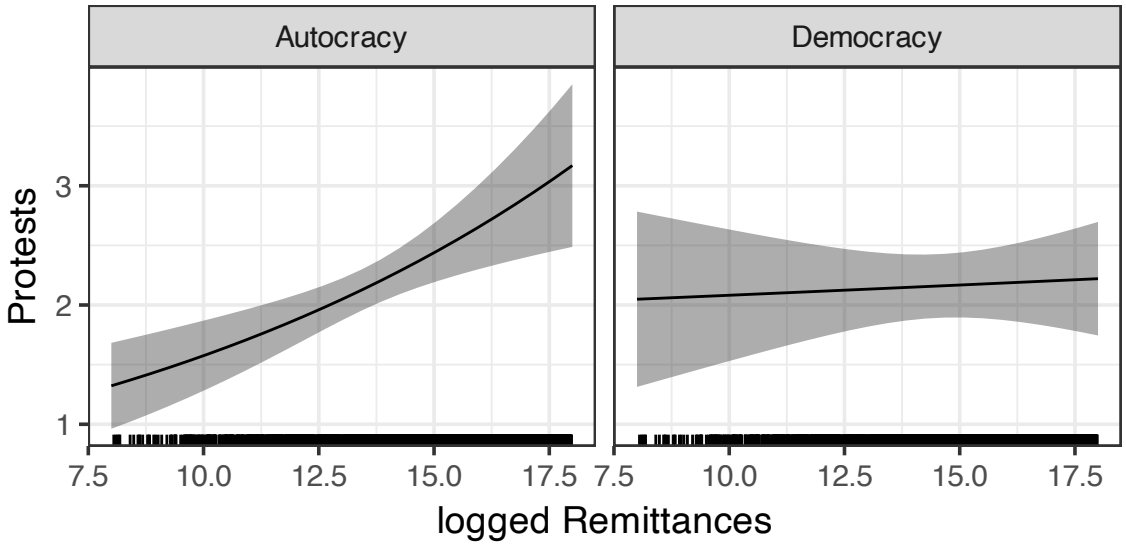
```
                      Protests=ystar,
                      lo=ystar-1.96*se.ystar,
                      hi=ystar+1.96*se.ystar)

## plot
ggplot(plot.df)+
  geom_ribbon(aes(x=Remittances, ymin=lo, ymax=hi), alpha=.4)+ ## CIs
  geom_line(aes(x=Remittances, Protests))+
  theme_bw(14)+
  facet_grid(~Regime)+
  xlab("logged Remittances")+
  geom_rug(aes(x=remit),data=protests%>%
             filter(remit>=8 & remit <= 18))
```



Or for the log-normal

```
ystar <- drop(exp(Xstar %*% log.normal1$coefficients+ log.normal1.s2.hat/2))

## delta method
DyDb <- Xstar* ystar
V.ystar <- DyDb %*% Vln %*% t(DyDb)
se.ystar <- sqrt(diag(V.ystar))


## create a data frame for plotting
```

```
plot.df <- data.frame(Remittances=Xstar[,"remit"],
                      Regime=rep(c("Democracy", "Autocracy"), each=25),
                      Protests=ystar,
                      lo=ystar-1.96*se.ystar,
                      hi=ystar+1.96*se.ystar)

## plot
ggplot(plot.df)+
  geom_ribbon(aes(x=Remittances, ymin=lo, ymax=hi), alpha=.4)+ ## CIs
  geom_line(aes(x=Remittances, Protests))+
  theme_bw(14)+
  facet_grid(~Regime)+
  xlab("logged Remittances")+
  geom_rug(aes(x=remit),data=protests%>%
             filter(remit>=8 & remit <= 18))
```



Finally, let's consider some model fit.

1. Poisson versus negative binomial (Nested models, so likelihood ratio, why?)
2. Correlations among the fitted values of these various approaches with the observed data

```
lrtest(nbreg, poi) ## from lmtest
```

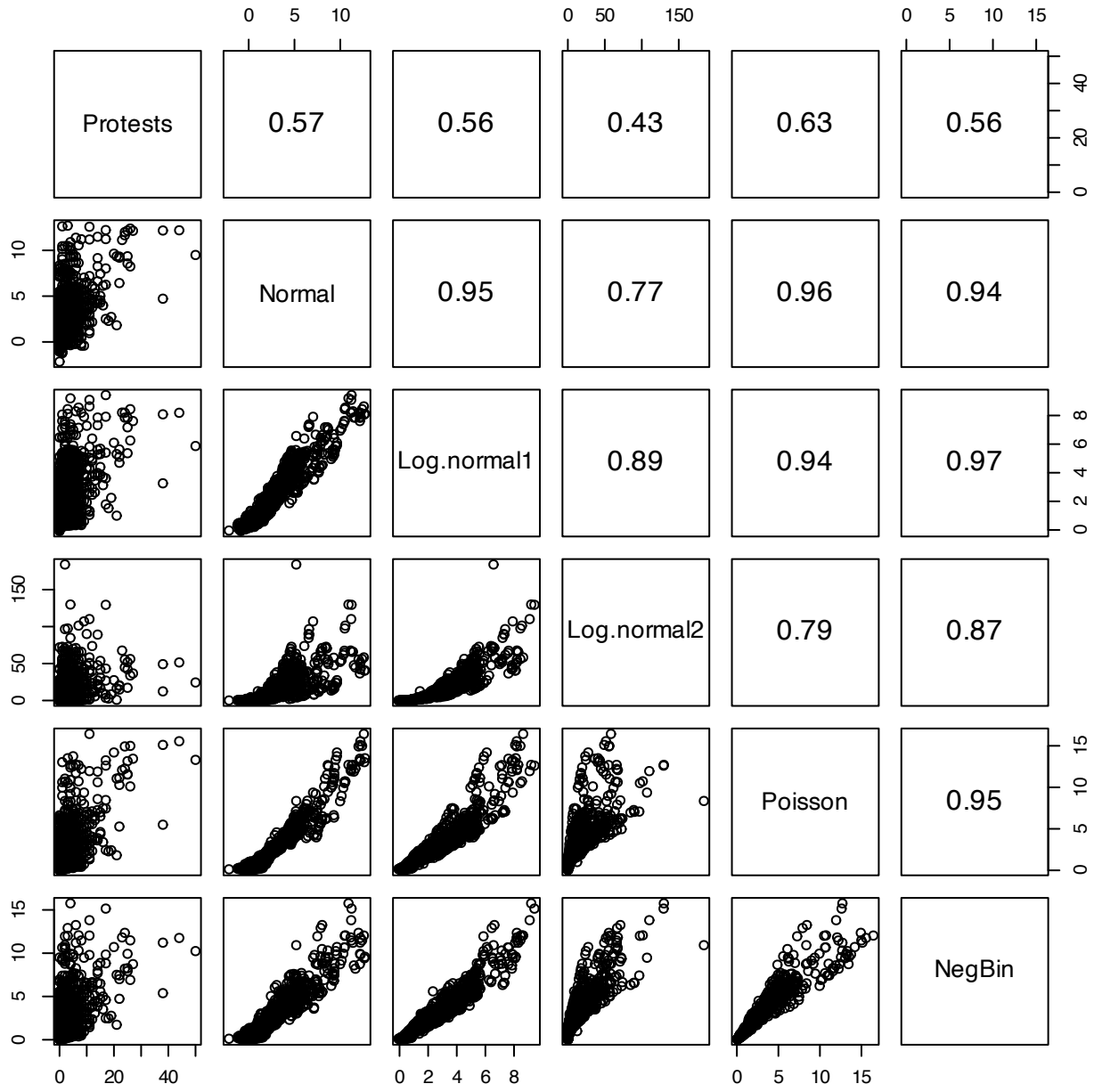
```
## Warning in modelUpdate(objects[[i - 1]], objects[[i]]): original model was of
## class "negbin", updated model is of class "glm"

## Likelihood ratio test
```

```
##
## Model 1: banks_protest_count ~ remit * dict + l1gdp + l1pop + l1nbr5 +
##     l12gr + l1migr + elec3 + factor(period) + factor(cowcode) -
##     1
## Model 2: banks_protest_count ~ remit * dict + l1gdp + l1pop + l1nbr5 +
##     l12gr + l1migr + elec3 + factor(period) + factor(cowcode) -
##     1
##   #Df  LogLik Df  Chisq Pr(>Chisq)
## 1 103 -3431.1
## 2 102 -4265.0 -1 1667.9  < 2.2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
##let's double check this one
LRstat <- 2*(logLik(nbreg) -logLik(poi))
LRpval <- pchisq(LRstat, df=1, lower.tail = FALSE)
c(LRstat, LRpval)
```

```
## [1] 1667.861    0.000
```

```
## Here we do find some evidence that negative binomial may be preferred
## However, remember that the Poisson is robust to more types of misspecification
## so it comes down to what our goal(s) are.



####
pairs(data.frame(Protests= protests$banks_protest_count,
                 Normal=normal$fitted.values,
                 Log.normal1=  exp(log.normal1$fitted.values+log.normal1.s2.hat/2)-1,
                 Log.normal2=  exp(log.normal2$fitted.values+log.normal2.s2.hat/2),
                 Poisson=  poi$fitted.values,
                 NegBin=  nbreg$fitted.values),
      upper.panel = \(x,y){par(usr = c(0, 1, 0, 1)); text(.5,.5,round(cor(x,y),2), cex=1
      cex.labels =1.285)
```



So while we do see some evidence that the negative binomial provides better "fit" in terms of the likelihood ratio test, this is by no means conclusive. Likewise, remember that this fit improvement is based on including that extra variance parameter, which the clustered standard errors render unnecessary.

What if we wanted to compare the normal to the log-normal or the Poisson? These are questions that would take us back to the AIC, BIC, and Vuong.

```
#### comparing normal to Poisson ####
## start with AIC/BIC
c(AIC(normal), AIC(poi))
```

```
## [1] 11567.592  8734.053
```

```
c(BIC(normal), BIC(poi))
```

```
## [1] 12157.437  9318.172
```

```
## which is better on these metrics?




###### Vuong test #####
## we need the likelihood ratio at each point
normal.ll <- function(theta, y,X){
  s2 <- theta[length(theta)]
  beta <- theta[-length(theta)]
  mu <- X %*%  beta
  return(-log(2*pi*s2)/2-(y-mu)^2 / (2*s2))
}
poisson.ll <- function(beta, y,X){
  mu <- X %*% beta
  lambda <- exp(X %*% beta)
  return(y*(mu)-lambda -lgamma(y+1))
}
s2.hat <-mean(normal$residuals^2) ##MLE for s2
LR1 <- normal.ll(c(normal$coef,s2.hat), y=normal$y, X=normal$x)
LR2 <- poisson.ll(poi$coef, y=poi$y,X=poi$x)
N <- length(normal$residuals)
LR <- LR1-LR2
omega2 <- mean(LR^2) - mean(LR)^2
LR <- sum(LR)
V <- LR/(sqrt(N*omega2))
V
```

```
## [1] -13.15356
```

```
pnorm(V) ## reject the null of equal fit in favor of model 2 (Poisson)
```

```
## [1] 8.117132e-40
```

```
vuongtest(normal, poi) ## from nonnest2 package (no penalty)
```

```
## Warning in imhof(n * omega.hat.2, lamstar^2): Note that Qq + abserr is
## positive.
```

```
##
## Model 1
##  Class: lm
##  Call: lm(formula = f1, data = protests, x = TRUE, y = TRUE)
##
## Model 2
##  Class: glm
##  Call: glm(formula = f1, family = poisson, data = protests, x = TRUE, ...
##
## Variance test
##   H0: Model 1 and Model 2 are indistinguishable
##   H1: Model 1 and Model 2 are distinguishable
##     w2 = 5.108,   p = <2e-16
##
## Non-nested likelihood ratio test
##   H0: Model fits are equal for the focal population
##   H1A: Model 1 fits better than Model 2
##     z = -13.154,   p = 1
##   H1B: Model 2 fits better than Model 1
##     z = -13.154,   p = < 2.2e-16
```

```
## Here we conclude that Poisson is a better model for y than the normal
```

3.5 Additional topics in Poisson GLMs: Instrumental variables and panel data

Given the advantages of the Poisson over the log-normal (e.g., it can handle 0 values) and the fact that are no real drawbacks to it in this context. We may want to know if we can expand it to other common settings like instrumental variables and panels The short answer is yes!

3.5.1 Panel Poisson

Above we briefly discussed that there were no real issues with including dummy variables in the Poisson setting. The one potential issue are the all-0 units. These do not affect estimates of β, but do affect the marginal effects. One thing we might want to know is how can we assess potential sensitivity here?

One way to proceed is to consider what's known as a Mundlak-specification this takes the form:

$$\mathrm{E}[y_{it}|X] = \lambda_{it} = \mathrm{E}_\alpha[\exp(x'_{it}\beta + \bar{x}'_i\gamma + \alpha_i)|X],$$

where $\bar{x}_i$ represents the within-unit mean of each X variable and α_i is a unit-level error term (often called a random effect or random intercept). Here the parameters of interest β are identified by their differences from these within-unit means which is analogous to the linear case.

Regarding α_i we can treat it in one of the following ways:

1. Let $\alpha_i \sim N(0, \sigma^2_\alpha)$ normally distributed (R default)
2. Let $\alpha_i \sim \Gamma(1, 1/\theta)$. Why? Makes the math easier (Stata default).
3. Ignore α_i (set it to 0) and fit the above with ordinary Poisson regression as a pseudo-likelihood estimator.

With either 1 or 2 our log-likelihood changes to

$$L(\theta|y) = \sum_{i=1}^{N} \log \left(\int_{-\infty}^{\infty} \left[\prod_{t=1}^{T_i} f(y_{it}|\lambda) g(\alpha_i) d\alpha_i \right] \right).$$

The random error inside the mean function means that we need to integrate over all possible values of α_i in order to evaluate the log-likelihood. For option 1, this requires us to introduce numeric integration methods (e.g., Gauss-Hermite or Monte Carlo integration). For option 2, the integral as a closed for solution, which helps.

For maximum challenge (and learning), let's briefly consider option 1. We will turn to the Gauss-Hermite method for evaluating integrals. Briefly Gaussian methods here is a way to evaluate difficult integrals by using polynomial approximations. There are difference types of Gauss methods for different types of problems. For us we will look at the Gauss-Hermite rule which approximates integrals

$$\int_{-\infty}^{\infty} \exp(-x^2) h(x) dx \approx \sum_{i=1}^{N} w_i h(x_i).$$

Here w_i are weights and x_i are a set of notes to evaluate the function at. This strategy is often the go-to for integrating over normal random variables because it's often easy to get into this form.

$$\begin{aligned} L(\theta|y) &= \sum_{i=1}^{N} \log\left(\int_{-\infty}^{\infty}\left[\prod_{t=1}^{T_i} f(y_{it}|\lambda) g(\alpha_i) d\alpha_i\right]\right) \\ &= \sum_{i=1}^{N} \log\left(\int_{-\infty}^{\infty} g(\alpha_i)\left[\prod_{t=1}^{T_i} f(y_{it}|\lambda)\right] d\alpha_i\right) \\ &= \sum_{i=1}^{N} \log\left(\int_{-\infty}^{\infty} \frac{\exp(-\alpha^2/2\sigma_\alpha^2)}{\sqrt{2\pi}\sigma_\alpha}\left[\prod_{t=1}^{T_i} f(y_{it}|\lambda)\right] d\alpha_i\right) \end{aligned}$$

Let $\alpha_i^* = \alpha_i/\sqrt{2}\sigma_\alpha$, then we can do a change of variables

$$\begin{aligned} &= \sum_{i=1}^{N} \log\left(\int_{-\infty}^{\infty} (\sqrt{2}\sigma_\alpha) \frac{\exp(-(\sqrt{2}\sigma_\alpha\alpha_i^*)^2/2\sigma_\alpha^2)}{\sqrt{2\pi}\sigma_\alpha}\left[\prod_{t=1}^{T_i} f(y_{it}|\lambda^*)\right] d\alpha_i^*\right) \\ &= \sum_{i=1}^{N} \log\left(\int_{-\infty}^{\infty} \frac{\exp(-\alpha_i^{*2})}{\sqrt{\pi}}\left[\prod_{t=1}^{T_i} f(y_{it}|\lambda^*)\right] d\alpha_i^*\right) \\ &\approx \sum_{i=1}^{N} \log\left(\frac{1}{\sqrt{\pi}}\left(\sum_{r=1}^{R} w_r\left[\prod_{t=1}^{T_i} f(y_{it}|\lambda_r^*)\right]\right)\right). \\ \lambda_r^* &= \exp(x_{it}'\beta + \bar{x}_i'\gamma + \alpha_{ir}^*\sqrt{2}\sigma_\alpha) \end{aligned}$$

From here we just evaluate it by choosing an R, higher is better, but costlier, and then evaluating it at the values of w_r and α_{ir}^* associated with that value. Finding out what these values are is a little involved, but thankfully it's been done for us.

```
library(fastGHQuad)
```

```
## Loading required package: Rcpp
```

```
gaussHermiteData(5)
```

```
## $x
## [1] -2.020183e+00 -9.585725e-01  1.023716e-16  9.585725e-01  2.020183e+00
##
## $w
## [1] 0.01995324 0.39361932 0.94530872 0.39361932 0.01995324
```

```
## returns values for evaluating the integral
## x is our alpha*_ir and w is w.
```

However, option 3 is very attractive as it means we only need to include the extra covariates to get what we want, which is an alternative to including the N dummy variables that still captures the idea of fixed effects. We can get away with this here, because we are assuming that α_i is independent of the observables, so omitting it is not a problem from a bias perspective.

3.5.2 IV Poisson

We start with the model of interest

$$\mathrm{E}[y_i|d, X, W] = \exp(x_i'\beta + d_i'\tau + u_i).$$

In this model, x_i is length-k, observed, and exogenous; d_i is length-M, observed, but only exogenous conditional on X and u; but u is unobserved. In other words, u are a set of omitted variables that make d_i, our variables of interest, endogenous.

Just like you did last semester, with 2SLS we will start by imposing a linear reduced-form on the endogenous variables with

$$d_i = z_i'\Gamma + v_i.$$

Note that because d_i may contain more than one variable, Γ is an ℓ by M matrix where M is the length of d_i and ℓ is the length of z_i, with $\ell - k \geq M$. Here v_i contains what we might think of as the endogenous portion of d_i, that is once we regress d_i on the exogenous factors and instruments, v_i is whatever is leftover. This will include the part of d_i that correlates with u_i.

We will assume that (u, v) are independent of Z (i.e., the instruments are in fact exogenous with respect to both outcomes and treatment), and that the unobserved variables can be decomposed as

$$u_i = v_i'\rho + \varepsilon_i,$$

with v_i being independent of ε_i. Note that this decomposition breaks u_i into the part that relates to d_i and everything else. For ease we will also assume that $\mathrm{E}[\exp(\varepsilon_i)] = 1$. With all this, we can rewrite the model as

$$\begin{aligned}
\mathrm{E}[y_i|d, X, W] &= \mathrm{E}_\varepsilon[\exp(x_i'\beta + d_i'\tau + v_i'\rho + \varepsilon_i)] \\
&= \mathrm{E}_\varepsilon[\exp(x_i'\beta + d_i'\tau + v_i'\rho)\exp(\varepsilon_i)] && \text{properties of exponents} \\
&= \mathrm{E}_\varepsilon[\exp(x_i'\beta + d_i'\tau + v_i'\rho)\,\mathrm{E}_\varepsilon[\exp(\varepsilon_i)] && \text{by our independence assumptions wrt to } \varepsilon_i \\
&= \exp(x_i'\beta + d_i'\tau + v_i'\rho)
\end{aligned}$$

This suggests a two-step procedure

1. Regress d_i on z_i using (multivariate) normal GLM, OLS, or seemingly unrelated regression (SUR). Estimate $\hat{v}_i = d_i - z_i'\hat{\Gamma}$.
2. Using Poisson regression, regress y_i on x_i, d_i and $\hat{v}_i$.

Because MLE is consistent, the resulting procedure will be consistent under our usual assumptions. There's just wrinkle here. We have now included an estimated value as a regressor. This violates our typical way of thinking. We often condition on observed regressors. However, the regressor v_i is estimated not observed and so this induces issues in how we think about the uncertainty of our estimates. Specifically, we will need to adjust our standard errors to reflect the fact that we now have an additional layer of uncertainty.

There are two ways to do this:

1. Use a non-parametric bootstrap that resamples y_i, d_i, z_i, and x_i. Refit both steps 1 and 2 above, generating new $\hat{v}$ each time (more on this later).
2. Find the analytic standard errors. This is not as hard as it sounds, in the sense that others have done it.

3.5.3 Two-step ML estimators

The basic intuition for finding the variance of a two-step estimator is that we want to know how the the two different likelihoods change with respect to the other step. Let θ_1 be the first-step parameters of length ℓ and θ_2 be the second-step parameters length k, and let $\theta = (\theta_1, \theta_2)$. From earlier this semester, we know that for MLEs the variance takes the form

$$\text{avar}(\hat{\theta}) = [H^{-1}]S[H^{-1}]',$$

where H is based on second derivatives of our log-likelihood function (Hessian), and S uses the variance/covariance of the scores. Under the information inequality, $S = H$ and this expression simplified. With two sequential log-likelihoods to consider, we expand this a bit:

$$H = \text{E}\begin{bmatrix} D_{\theta_1\theta_1'}L_1(\theta_1|d) & D_{\theta_1\theta_2'}L_1(\theta_1|d) \\ D_{\theta_2\theta_1'}L_2(\theta_2|y,\theta_1) & D_{\theta_2\theta_2'}L_2(\theta_2|y,\theta_1) \end{bmatrix} = \begin{bmatrix} D_{\theta_1\theta_1'}L_1(\theta_1|d) & 0 \\ D_{\theta_2\theta_1'}L_2(\theta_2|y,\theta_1) & D_{\theta_2\theta_2'}L_2(\theta_2|y,\theta_1) \end{bmatrix}$$

$$S = \text{E}\begin{bmatrix} D_{\theta_1}L_1(\theta_1|d)D_{\theta_1}L_1(\theta_1|d)' & D_{\theta_1}L_1(\theta_1|d)D_{\theta_2}L_2(\theta_2|y,\theta_1)' \\ D_{\theta_2}L_2(\theta_2|y,\theta_1)D_{\theta_1}L_1(\theta_1|d)' & D_{\theta_2}L_2(\theta_2|y,\theta_1)D_{\theta_2}L_2(\theta_2|y,\theta_1)' \end{bmatrix}.$$

Let's make this a little nicer to look at:

$$\begin{aligned}
H &= \begin{bmatrix} -V_{1,H}^{-1} & 0 \\ -C_H & -V_{2,H}^{-1} \end{bmatrix} \\
S &= \begin{bmatrix} -V_{1,OPG}^{-1} & R' \\ R & -V_{2,OPG}^{-1} \end{bmatrix} \\
H^{-1} &= \begin{bmatrix} -V_{1,H} & 0 \\ V_{2,H}C_H V_{1,H} & -V_{2,H} \end{bmatrix} \\
C_H &= -\operatorname{E}\left[D_{\theta_2\theta_1'} L_2(\theta_2|y,\theta_1)\right] \\
R &= \operatorname{E}\left[D_{\theta_2} L_2(\theta_2|y,\theta_1) D_{\theta_1} L_1(\theta_1|d)'\right].
\end{aligned}$$

Here, $V_{1,H}$ denotes the Hessian based estimators for the variance of θ_1 and likewise for θ_2. The OPG subscript denotes the OPG estimator for this variance. Under ordinary cases, we can mix and match the OPG and Hessians due to the information inequality. In these cases we get that the combined variance of the second step estimates $\hat{\theta}_2$ becomes

$$\operatorname{Var}(\hat{\theta}_2) = V_2 + V_2\left[CV_1C' - RV_1C' - CV_1R'\right]V_2.$$

Here, V_1 and V_2 are the variance matrices of the 1st and 2nd step, respectively. Under the information equality we can estimate these using either the Hessian or the OPG. The C matrix here can be estimated using C_H from above or an OPG estimator

$$C_{OPG} = \operatorname{E}\left[D_{\theta_2} L_2(\theta_2|y,\theta_1) D_{\theta_1} L_2(\theta_2|y,\theta_1)'\right].$$

Note that if the two steps are fit to different samples (including a sub-sample) than R doesn't exist. In these cases, the formula simplifies to

$$\operatorname{Var}(\hat{\theta}_2) = V_2 + V_2\left[C'V_2C\right]V_2.$$

However, the more general setup tells us what we can do if we don't want to assume the information equality holds. We can tweak the the Murphy-Topel result to get to robust or clustered standard errors. Specifically, we now have

$$\begin{aligned}
V_{1,R} &= V_{1,H}V_{1,OPG}^{-1}V_{1,H} \\
V_{2,R} &= V_{2,H}V_{2,OPG}^{-1}V_{2,H} \\
\operatorname{avar}(\hat{\theta}) &= V_{2,R} + V_{2,H}\left(C_H'V_{1,R}C_H - R'V_{1,H}C_H - C_H'V_{1,H}R\right)V_{2,H}.
\end{aligned}$$

This will give us robust standard errors, while replacing the OPGs with the within-OPGs (find the OPG for each unit and sum over those), will give us clustered standard errors. As before if the two samples differ at all, $R = 0$

3.5.4 Application

We return to the above example to consider first Mundlak pseudo-likelihood Poisson and then instrumental variable Poisson.

```
## basics
library(readstata13)
library(dplyr)
## econometric
library(lmtest)
library(sandwich)
library(MASS)
library(car)
## tables and figures
library(xtable)
library(ggplot2)

rm(list=ls())



protests <- read.dta13("Code/R/datasets/remittances_and_protests1.dta")
protests <- subset(protests, sample==1)

f1 <- banks_protest_count ~ remit*dict +
  l1gdp + #GDP pc capita
  l1pop+ #population
  l1nbr5 + #neighboring protests
  l12gr+ #Economic growth
  l1migr+ #Net migration
  elec3+ #election
  factor(cowcode)-1

## Poisson: ML-Dummy variable
```

```r
poi.mldv <- glm(f1, data=protests,family=poisson, y=TRUE, x=TRUE)

## Mundlak setup
protests <- protests %>%
  mutate(remit.dict = remit*dict) %>%
  group_by(cowcode)%>%
  mutate(across(c(names(poi.mldv$coef)[grep("factor|:",names(poi.mldv$coef),
                                            invert = TRUE)],
                  "remit.dict"),
                .fns=mean,
                .names="{.col}.bar")) %>%
  ungroup()

## Mundlak formula
f2 <- update(f1, .~.-factor(cowcode)+remit.bar + dict.bar +
               l1gdp.bar + l1pop.bar + l1nbr5.bar + l12gr.bar +
               l1migr.bar + elec3.bar + remit.dict.bar+1)
## fit the Mundlak
poi.mundlak <- glm(f2,  data=protests,family=poisson, y=TRUE, x=TRUE)

cbind(poi.mldv$coef[grep("factor",names(poi.mldv$coef),
                         invert = TRUE)],
      poi.mundlak$coef[grep("bar|Intercept",names(poi.mundlak$coef),
                            invert = TRUE)])
```

```
##                    [,1]         [,2]
## remit       -0.01183775 -0.004263146
## dict        -2.26481622 -2.077636493
## l1gdp        0.16865782  0.221618899
## l1pop       -0.63588909 -0.650920813
## l1nbr5      -0.08671863 -0.089896147
## l12gr       -0.02944027 -0.027957284
## l1migr       0.12500050  0.146602274
## elec3        0.15995501  0.175847769
## remit:dict   0.16881091  0.158730468
```

```r
## comparing marginal effects
X0 <- X1 <- poi.mldv$x
X0[,"dict"] <- X0[,"remit:dict"] <- 0
X1[,"dict"] <- 1
X1[, "remit:dict"] <- X1[,"remit"]

AME.mldv <- c(poi.mldv$coef["remit"] * mean(exp(X0 %*% poi.mldv$coef)),
              (poi.mldv$coef["remit"]+poi.mldv$coef["remit:dict"]) *
                mean(exp(X1 %*% poi.mldv$coef)))

## drop the all 0s
protests <- protests %>%
  mutate(sum.y = sum(banks_protest_count), .by=cowcode)

table(protests$sum.y==0)
```

```
##
## FALSE  TRUE
##  2268   161
```

```r
X0 <- X1 <- poi.mldv$x[protests$sum.y>0,]
X0[,"dict"] <- X0[,"remit:dict"] <- 0
X1[,"dict"] <- 1
X1[, "remit:dict"] <- X1[,"remit"]

AME.mldv2 <- c(poi.mldv$coef["remit"] * mean(exp(X0 %*% poi.mldv$coef)),
               (poi.mldv$coef["remit"]+poi.mldv$coef["remit:dict"]) *
                 mean(exp(X1 %*% poi.mldv$coef)))



## mundlak version
X0 <- X1 <- poi.mundlak$x
X0[,"dict"] <- X0[,"remit:dict"] <- 0
X1[,"dict"] <- 1
X1[, "remit:dict"] <- X1[,"remit"]
```

```r
AME.mundlak <- c(poi.mundlak$coef["remit"] * mean(exp(X0 %*% poi.mundlak$coef)),
                 (poi.mundlak$coef["remit"]+poi.mundlak$coef["remit:dict"]) *
                   mean(exp(X1 %*% poi.mundlak$coef)))

compMat <- rbind(AME.mldv,AME.mldv2,AME.mundlak)
colnames(compMat) <- c("Demo.", "Auto.")
compMat
```

```
##                      Demo.     Auto.
## AME.mldv    -0.018099147 0.3300514
## AME.mldv2   -0.019383963 0.3534809
## AME.mundlak -0.006394012 0.3306567
```

We see some differences here, particularly with respect to the effect within democracy.

Turning to the IV methods, in this paper, the authors consider an instrument for remittances based on two factors:

1. The average value of remittances received in high-income OECD countries over the last 2 years (millions of USD)
2. The average distance the receiving country is from a coast.

They argue that remittances to OECD countries are generally apolitical and come from other OECD countries. They only affect protests through by broadly raising the amount of money available to then remit to other countries. The distance measure introduce cross-section variance and reflects the ease of migration to this country.

We can apply the two-step logic from Murphy and Topel to build the two-step standard errors for the Poisson under the following assumptions:

1. Fully correct. Here we assume that we made all of the right assumptions and the information inequality holds.
2. Robust. iid observations but maybe some other misspecifications
3. Cluster robust. iid units but within units (countries here) there may be some iid violations.

let $\mathbf{X} = [X, d, \hat{v}]$, from our previous analyses we have the following already:

$$\begin{aligned}
V_{1,H} &= \hat{\sigma}^2 (Z'Z)^{-1} \\
V_{2,H} &= (\mathbf{X}'(I \exp(\mathbf{X}\theta_2))\mathbf{X})^{-1} \\
V_{1,OPG} &= \left(\left[(I\hat{v})Z/\hat{\sigma}^2 \right]' \left[(I\hat{v})Z/\hat{\sigma}^2 \right] \right)^{-1} \\
V_{2,OPG} &= \left(\left[(I(y - \hat{\lambda}))\mathbf{X} \right]' \left[(I(y - \hat{\lambda}))\mathbf{X} \right] \right)^{-1} \\
V^c_{1,OPG} &= \left[\sum_{i=1}^{N} \left[\left(\sum_{t=1}^{T_i} \hat{v}_{it} z_{it} / \hat{\sigma}^2 \right) \left(\sum_{t=1}^{T_i} \hat{v}_{it} z_{it} / \hat{\sigma}^2 \right)' \right] \right]^{-1} \\
V^c_{2,OPG} &= \left[\sum_{i=1}^{N} \left[\left(\sum_{t=1}^{T_i} (y_{it} - \hat{\lambda}_{it}) \mathbf{x}_{it} \right) \left(\sum_{t=1}^{T_i} (y_{it} - \hat{\lambda}_{it}) \mathbf{x}_{it} \right)' \right] \right]^{-1},
\end{aligned}$$

where c denotes the "clustered" estimator.

We also know R from is the OPG across models. For the unclustered estimator, this is

$$R = \left[(I\hat{v})Z/\hat{\sigma}^2 \right]' \left[(I(y - \hat{\lambda}))\mathbf{X} \right],$$

and for the clustered estimator it is

$$R^c = \sum_{i=1}^{N} \left[\left(\sum_{t=1}^{T_i} \hat{v}_{it} z_{it} / \hat{\sigma}^2 \right) \left(\sum_{t=1}^{T_i} (y_{it} - \hat{\lambda}_{it}) \mathbf{x}_{it} \right)' \right].$$

Writing it out like this perhaps makes it more clear why $R = 0$ if the two steps are fit to different samples, because using this formula becomes impossible.

Finally, we just need to know C. In cases where we have the information equality, we can use an OPG where we need the derivative of L_2 with respect to γ.

$$C = \left[\rho I(y - \hat{\lambda})Z \right]' \left[(I(y - \hat{\lambda}))\mathbf{X} \right].$$

In cases where we want robust or clustered standard errors, we need that cross-partial derivative C_H. Let's turn to `sympy` to see this one through. Our application below involves two endogenous variables so we'll work with that.

```
## Load the package
from sympy import *
```

```
##optional but makes the printed math nicer
init_printing(forecolor="White")

## define parameters. Do 2 endogenous variables to match example
g1, g2,b,t, r1, r2 =symbols("gamma_1, gamma_2 beta, tau, rho_1, rho_2", real=True)
s = symbols("sigma^2", positive=True)
i, N= symbols("i, N", positive=True, integer=True)

## define data
y = IndexedBase('y', real=True, positive=True)
x = IndexedBase('x', real=True)
z = IndexedBase('z', real=True)
d = IndexedBase('d', real=True)

## some shortcuts
v1 = d[i]-z[i]*g1
v2 = d[i]-z[i]*g2
xb = x[i]*b + d[i]*t+ v1*r1+v2*r2

## likelihoods
L1a = -N*log(2*pi)/2 -  N*log(s)/2 - Sum((d[i]-z[i]*g1)**2, (i,1,N))/ (2*s)
L1b = -N*log(2*pi)/2 -  N*log(s)/2 - Sum((d[i]-z[i]*g2)**2, (i,1,N))/ (2*s)
L2 = Sum(y[i]*xb -exp(xb),(i,1,N) )


## For H2, note that we are fixing the new inputs so the derive wrt to beta
## will give us the pattern we need for coding the full H2
H2 = L2.diff(b).diff(b)
## -X' (Iexp(XB)) X like the notes

## more important for us is C

## C_OPG needs the Jacobian wrt to gamma
DL2_Dg = L2.diff(g1)
## Z' rho * (lambda-y)
```

```
## C_H uses cross derivatives
C_bg = -L2.diff(b).diff(g1)
C_tg = -L2.diff(t).diff(g1)
C_r1g = -L2.diff(r1).diff(g1)
C_r2g = -L2.diff(r2).diff(g1)



C_bg
```

```
##   N
##  ___
##  \  `
##   \           beta*x[i] + rho_1*(-gamma_1*z[i] + d[i]) + rho_2*(-gamma_2*z[i] + d[i])
## - /    rho_1*e
##  /__,
##  i = 1
```

```
## rho * (X' (Iexp(XB)) Z)
```

```
C_r1g
```

```
##   N
##  ___
##  \  `
##   \    /                                beta*x[i] + rho_1*(-gamma_1*z[i] + d[i]) + rho_
## - /    \rho_1*(-gamma_1*z[i] + d[i])*e
##  /__,
##  i = 1
##
## >
## >
## >
## > ]) + tau*d[i]                  \
## >              *z[i] - y[i]*z[i]/
## >
## >
```

```
## rho * (v' *lambda) +(lambda - y)'Z
```

```
C_r2g
```

```
##   N
##  ___
##  \  `
##   \                                    beta*x[i] + rho_1*(-gamma_1*z[i] + d[i]) + rho_2
## - /    rho_1*(-gamma_2*z[i] + d[i])*e
##  /__,
##  i = 1
```

```
## rho * (v' *lambda)
```

With one endogenous variable ($M = 1$) this simplifies to

$$C_H = -\begin{bmatrix} \rho(X'I(\hat{\lambda})Z) \\ \rho(d'I(\hat{\lambda})Z) \\ \rho(\hat{v}'I(\hat{\lambda})Z) + (\hat{\lambda} - y)'Z \end{bmatrix}_{(k+2\times\ell)}$$

or with two (like in the example below)

$$C_H = -\begin{bmatrix} \rho_1(X'I(\hat{\lambda})Z) & \rho_2(X'I(\hat{\lambda})Z) \\ \rho_1(d'I(\hat{\lambda})Z) & \rho_2(d'I(\hat{\lambda})Z) \\ \rho_1(\hat{v}_1'I(\hat{\lambda})Z) + (\hat{\lambda} - y)'Z & \rho_2(\hat{v}_1'I(\hat{\lambda})Z) \\ \rho_1(\hat{v}_2'I(\hat{\lambda})Z) & \rho_2(\hat{v}_2'I(\hat{\lambda})Z) + (\hat{\lambda} - y)'Z \end{bmatrix}_{k+2M\times\ell M}$$

```
library(ivreg) ##compare to 2sls
protests <- protests %>%
  mutate(dist =(log(1+(1/(dist_coast)))) ,
         distwremit = log(1+((richremit/1000000)*(dist))))

tsls <- ivreg(log(banks_protest_count+1) ~ remit*dict +
                l1gdp + #GDP pc capita
                l1pop+ #population
                l1nbr5 + #neighboring protests
                l12gr+ #Economic growth
                l1migr+ #Net migration
```

```
                elec3+#election
                factor(cowcode)-1|
                distwremit*dict +
                l1gdp + #GDP pc capita
                l1pop+ #population
                l1nbr5 + #neighboring protests
                l12gr+ #Economic growth
                l1migr+ #Net migration
                elec3+#election
                factor(cowcode)-1,
              data=protests)
coeftest(tsls, vcovCL(tsls, protests$cowcode))[grep("factor",
                                                    names(tsls$coefficients),
                                                    invert=TRUE),]
```

```
##                Estimate  Std. Error    t value   Pr(>|t|)
## remit      -0.176410396 0.085091635 -2.0731814 0.03826564
## dict       -5.926834336 2.481378449 -2.3885250 0.01699574
## l1gdp       0.387264488 0.226433867  1.7102763 0.08734860
## l1pop      -0.326524477 0.295783383 -1.1039311 0.26973761
## l1nbr5      0.069913593 0.158544202  0.4409722 0.65927421
## l12gr      -0.018200932 0.008262172 -2.2029233 0.02769823
## l1migr      0.021365752 0.096342442  0.2217688 0.82451337
## elec3       0.007559129 0.055078799  0.1372421 0.89085137
## remit:dict  0.417373388 0.172766813  2.4158192 0.01577671
```

```
## first stages
m1 <- lm(remit~ distwremit*dict +
           l1gdp + #GDP pc capita
           l1pop+ #population
           l1nbr5 + #neighboring protests
           l12gr+ #Economic growth
           l1migr+ #Net migration
           elec3+ #election
          factor(cowcode)-1,
         x=TRUE,y=TRUE,
         data=protests)
```

```r
m2 <- lm(I(remit*dict)~ distwremit*dict +
           l1gdp + #GDP pc capita
           l1pop+ #population
           l1nbr5 + #neighboring protests
           l12gr+ #Economic growth
           l1migr+ #Net migration
         elec3+ #election
          factor(cowcode)-1,
         x=TRUE,y=TRUE,
         data=protests)


protests$v1 <- m1$residuals
protests$v2 <- m2$residuals

poi.iv <- glm(banks_protest_count ~ remit*dict +
                l1gdp + #GDP pc capita
                l1pop+ #population
                l1nbr5 + #neighboring protests
                l12gr+ #Economic growth
                l1migr+ #Net migration
                elec3+ #election
                v1 + v2+
                factor(cowcode)-1,
              data=protests,
              family=poisson,
              x=TRUE,
              y=TRUE)

coeftest(poi.iv, vcovCL(poi.iv, protests$cowcode))[grep("factor",
                                                        names(poi.iv$coefficients),
                                                        invert=TRUE),]
```

```
##               Estimate Std. Error    z value    Pr(>|z|)
## remit      -0.29957869 0.10953096 -2.7351051 0.006236036
## dict       -8.61614589 5.11479449 -1.6845537 0.092074714
```

```
## l1gdp       0.58789158 0.30178346  1.9480577 0.051408065
## l1pop      -0.50313866 0.68253934 -0.7371570 0.461026875
## l1nbr5     -0.29690288 0.33456476 -0.8874302 0.374847319
## l12gr      -0.03024825 0.01836988 -1.6466221 0.099635749
## l1migr      0.02379525 0.11013171  0.2160618 0.828939592
## elec3       0.08586825 0.12109819  0.7090796 0.478275096
## v1          0.36049067 0.10984961  3.2816746 0.001031926
## v2         -0.50997441 0.35839968 -1.4229209 0.154759077
## remit:dict  0.60901360 0.35754222  1.7033334 0.088505701
```

```
ell <- length(m1$coef) ## size of the instrument matrix

## First stage covariance matrix is block diagonal
## [V1,1    0 ]
## [0     V1,2]
## We fit these independently so there is no cross correlation.
## We could have tried to fit these as a system with
V1 <- rbind( cbind(vcov(m1),           matrix(0,ell,ell)),
             cbind(matrix(0,ell,ell),  vcov(m2)))
Vcl1 <- rbind( cbind(vcovCL(m1,~cowcode), matrix(0,ell,ell)),
              cbind(matrix(0,ell,ell),   vcovCL(m2,~cowcode)))



## second stage matricies
Vcl2 <- vcovCL(poi.iv, ~cowcode)
V2 <- vcov(poi.iv)

## C is the cross deriv D
lambda.hat <- poi.iv$fitted.values
bonus.term <- c( t(lambda.hat-poi.iv$y ) %*% m1$x)
C0 <- t(poi.iv$x)%*% diag(lambda.hat) %*% m1$x
C1 <- poi.iv$coefficients["v1"] * C0
C2 <- poi.iv$coefficients["v2"] * C0
C1["v1",] <-C1["v1",]  +bonus.term
C2["v2",] <-C2["v2",]  +bonus.term
C <- -cbind(C1, C2)
```

```r
## R is the OPG between steps.
## Since we're clustering we want to sum within units first
## create function to split our big matricies into smaller matricies
## by units
split.matrix <- function(x, f){
  lapply(split(x = seq_len(nrow(x)), f = f),
         function(ind) x[ind, , drop = FALSE])

}

## Poisson Jacobian split by cowcode
Jpoi <- split.matrix(poi.iv$x*(poi.iv$y-poi.iv$fitted), protests$cowcode)

## Two Normal Jacobians split by cowcode
Jnormal <- cbind(m1$x*(m1$residuals)/mean(m1$residuals^2),
                 m2$x*(m2$residuals)/mean(m2$residuals^2))
J1 <-  split.matrix(Jnormal, protests$cowcode)

## apply over cowcodes, return the outer produce of the within-sums
## Then add these up
R  <- Reduce(`+`,
             lapply(1:length(Jpoi),
                    \(x){
                      return(colSums(Jpoi[[x]]) %*% t(colSums(J1[[x]])))
                    }
             )
)


V2step <- Vcl2 + V2 %*% (C %*% Vcl1 %*% t(C)-
                         R %*% V1 %*% t(C) -
                          C %*% V1 %*% t(R) )%*% V2
```

```
coeftest(poi.iv, V2step)[grep("factor",
                              names(poi.iv$coefficients),
                              invert=TRUE),]
```

```
##                Estimate Std. Error    z value   Pr(>|z|)
## remit      -0.29957869 0.16364590 -1.8306520 0.06715250
## dict       -8.61614589 5.93825804 -1.4509551 0.14679236
## l1gdp       0.58789158 0.47058068  1.2492897 0.21155914
## l1pop      -0.50313866 0.80883721 -0.6220518 0.53390779
## l1nbr5     -0.29690288 0.35865844 -0.8278151 0.40777522
## l12gr      -0.03024825 0.02105581 -1.4365753 0.15083870
## l1migr      0.02379525 0.14513906  0.1639480 0.86977210
## elec3       0.08586825 0.13444938  0.6386660 0.52304023
## v1          0.36049067 0.16939471  2.1281105 0.03332792
## v2         -0.50997441 0.42035618 -1.2131959 0.22505492
## remit:dict  0.60901360 0.41628320  1.4629790 0.14347312
```

4 Simulation methods

Before going into the next type of GLM, let's take a few minutes to round out the kind of computational tools you'll want to have as an applied analyst.

4.1 Monte Carlo simulation

Monte Carlo simulation/methods cover a wide range of applications. The overarching theme is that we use simulations of a problem/model/statistic to say something about it's true (analytic) values. We use Monte Carlo methods in cases where writing out a full expression for something is too hard/impossible. In these cases we use randomness to cover a range of possible cases and evaluate those to understand the problem. If you write methods papers, Monte Carlo simulation is often the main part of these papers.

For this course, we'll use Monte Carlo simulations for two main purposes:

1. Analyzing the statistical properties of estimators and test statistics to say something about how they perform under in different circumstances.
2. Verifying that the code we write does what we think it does.

The former is typically what you see in methods papers where authors describe and illustrate

the properties of whatever they're proposing. For example, these can be to illustrate:

1. An estimator's bias
2. Efficiency compared to another estimator
3. How this bias/efficiency changes with different samples sizes or under different forms of misspecification
4. Coverage: Do 95% of estimated confidence intervals contain the true value?
5. Size/power: How often are we rejecting true nulls? How often are we rejecting untrue nulls?

This is useful for us, because while we derived some asymptotic results on MLEs, there's little we can say about their finite sample properties analytically. Likewise, when we want to access estimators under non-ideal conditions, proofs and analytic results often become harder, so simulation allows us to shed some light on that.

The other part of this, verifying our own code, is not something that makes it into papers usually, but when you're writing your own MLE or other estimation strategies you want to know that it actually works the way you think it should. Monte Carlo simulations are good for that.

For example, Reshi was telling me about a residualization method he is using for something. Because he's a smart and well-put-together chap, I didn't feel the need to tell him to do some Monte Carlo simulations to verify that it does what he thinks it does. I'm sure he already did (or will do) exactly that.

The basic workflow of a Monte Carlo simulation is relatively straightforward.

1. Generate simulated data according to a model
2. Use this data to compute the quantities you're interesting in analyzing. Save this value
3. Repeat steps 1–2 a large number of times
4. Analyze the output

Let's try an example. We know that we lose some properties of MLE under misspecification. Let's suppose that the true DGP is

$$y_i \overset{iid}{\sim} \log N(1 + x_i, 4),$$

where we'll let X be iid standard normal.

But we don't know what the true DGP is, and not thinking too much about it other than that we notice it is strictly positive, we try a Poisson. We recall that even though these aren't

integer outcomes, it should be consistent for strictly positive data. We have 5 questions:

1. Is the Poisson estimate of β_1 close enough to 1?
2. Is the true variance of this estimator better estimated using OPG, the inverse Hessian, or the robust sandwich estimator?
3. Is the Poisson estimate of the AME close to the true value?
4. Is the robust variance a good estimate for the true variance of the AME?
5. How much slower is the optimization if we do not include the gradient?

First, we need our Poisson likelihood, score, and Jacobian functions again

```
l <- function(beta, y,X){
  XB <- X%*%beta
  lambda <- exp(XB)
  LL <- y*XB - lambda
  return(-sum(LL))
}

r <- function(beta, y,X){
  XB <- drop(X%*%beta)
  lambda <- exp(XB)
  score <- X*(y-lambda)
  return(-colSums(score))
}

J <-function(beta, y,X){
  XB <- drop(X%*%beta)
  lambda <- exp(XB)
  score <- X*(y-lambda)
  return(score)
}
```

Then we can create a skeleton for what a simulation might look like

```
set.seed(1)
B <- 250 ## number of simulations
N <- 500 ## sample size
beta <- c(1,1) #parameters
```

```
## Create an empty matrix for storing results
output <- matrix(0,
                 nrow=B, # number of simulations
                 ncol=8) # number of things to save
for(b in 1:B){
  ## Generate data
  X <- cbind(1,rnorm(N))
  y <- rlnorm(N, X %*%beta, 2)

  #### DO WHAT WE NEED TO DO TO GET RESULTS####

  ## save output of simulation b
  output[b,] <- c(fit1$par[2], #Poisson est of beta1
                  diag(Vh)[2], #hessian var of beta1
                  diag(Vopg)[2], #OPG var of beta1
                  diag(Vrobust)[2], #Robust var of beta1
                  AME.true, #log normal AME
                  AME.est, # estimated AME
                  V.ame, #estimated variance of the AME
                  time2[3]/time1[3]) #time differences
}
```

Okay, now we can fill in some of this. We have code from above to fit the model with and without the score function, and we know how to find the various variance estimators.

```
set.seed(1)
B <- 250 ## number of simulations
N <- 500
beta <- c(1,1)

## Create an empty matrix for storing results
output <- matrix(0,
                 nrow=B, # number of simulations
                 ncol=8) # number of things to save
for(b in 1:B){
  X <- cbind(1,rnorm(N))
  y <- rlnorm(N, X %*%beta, 2)
```

```
  #Fit models and check times (Q1 and 5)
  time1 <- system.time({
    fit1 <- optim(rep(0,2),
                  fn=l, gr=r,
                  y=y, X=X,
                  method="BFGS",
                  hessian=TRUE)
  })
  time2 <- system.time({ #no score
    fit2 <- optim(rep(0,2),
                  fn=l,
                  y=y, X=X,
                  method="BFGS",
                  hessian=TRUE)
  })

  ## find the variances (Q2)
  Vh <- solve(fit1$hessian)
  OPG <- crossprod(J(fit1$par, y=y,X=X))
  Vopg <- solve(OPG)
  V.robust <- Vh %*% OPG %*% Vopg

  ## save output of simulation b
  output[b,] <- c(fit1$par[2], #Poisson est of beta1
                  diag(Vh)[2], #hessian var of beta1
                  diag(Vopg)[2], #OPG var of beta1
                  diag(Vrobust)[2], #Robust var of beta1
                  AME.est, # estimated AME
                  V.ame, #estimated variance of the AME
                  time2[3]/time1[3]) #time differences
}
```

Next we want the AMEs. Who remembers what those are for true log-normal and the Poisson

model we impose?

$$
\begin{aligned}
AME(X_1) &= \beta_1 \, \mathrm{E}[\exp(x_i'\beta + \sigma^2/2)] \\
&= \exp(7/2) \qquad \text{under the imposed DGP} \\
\widehat{AME(X_1)} &= \hat{\beta}_1 \frac{1}{N} \sum_{i=1}^{N} \exp(x_i'\hat{\beta}).
\end{aligned}
$$

```
set.seed(1)
B <- 250 ## number of simulations
N <- 500
beta <- c(1,1)
trueAME <- exp(7/2)

## Create an empty matrix for storing results
output <- matrix(0,
                 nrow=B, # number of simulations
                 ncol=8) # number of things to save
for(b in 1:B){
  X <- cbind(1,rnorm(N))
  y <- rlnorm(N, X %*%beta, 2)

  #Fit models and check times (Q1 and 5)
  time1 <- system.time({
    fit1 <- optim(rep(0,2),
                  fn=l, gr=r,
                  y=y, X=X,
                  method="BFGS",
                  hessian=TRUE)
  })
  time2 <- system.time({ #no score
    fit2 <- optim(rep(0,2),
                  fn=l,
                  y=y, X=X,
                  method="BFGS",
                  hessian=TRUE)
  })
```

```
  ## find the variances (Q2)
  Vh <- solve(fit1$hessian)
  OPG <- crossprod(J(fit1$par, y=y,X=X))
  Vopg <- solve(OPG)
  V.robust <- Vh %*% OPG %*% Vopg

  ## find the AME (Q3)
  AME.est <- fit1$par[2] * mean(exp(X %*% fit1$par))



  ## save output of simulation b
  output[b,] <- c(fit1$par[2], #Poisson est of beta1
                  diag(Vh)[2], #hessian var of beta1
                  diag(Vopg)[2], #OPG var of beta1
                  diag(Vrobust)[2], #Robust var of beta1
                  AME.est, # estimated AME
                  V.ame, #estimated variance of the AME
                  time2[3]/time1[3]) #time differences
}
```

Ok what's next? Now we construct the variance estimate for the AME, which is what?

$$\text{Var}(\widehat{AME(X_1)}) = [D_\beta AME(X_1)]' \, \text{avar}(\hat{beta})[D_\beta AME(X_1)].$$

For this we need the derivative the AME with respect to β

$$\begin{aligned} D_\beta AME(X_1) &= D_\beta \beta_1 \sum \exp(x_i'\beta)/N \\ &= \left[\sum \beta_1 \exp(x_i'\beta)/N \quad \sum (x_{1i}\beta_1 + 1)\exp(x_i'\beta)/N\right] \end{aligned}$$

```
set.seed(1)
B <- 250 ## number of simulations
N <- 500
beta <- c(1,1)
trueAME <- exp(7/2)
```

```
## Create an empty matrix for storing results
output <- matrix(0,
                 nrow=B, # number of simulations
                 ncol=7) # number of things to save
for(b in 1:B){
  X <- cbind(1,rnorm(N))
  y <- rlnorm(N, X %*%beta, 2)

  #Fit models and check times (Q1 and 5)
  time1 <- system.time({
    fit1 <- optim(rep(0,2),
                  fn=l, gr=r,
                  y=y, X=X,
                  method="BFGS",
                  hessian=TRUE)
  })
  time2 <- system.time({ #no score
    fit2 <- optim(rep(0,2),
                  fn=l,
                  y=y, X=X,
                  method="BFGS",
                  hessian=TRUE)
  })

  ## find the variances (Q2)
  Vh <- solve(fit1$hessian)
  OPG <- crossprod(J(fit1$par, y=y,X=X))
  Vopg <- solve(OPG)
  V.robust <- Vh %*% OPG %*% Vh

  ## find the AME (Q3)
  AME.est <- fit1$par[2] * mean(exp(X %*% fit1$par))

  ## Find the variance of the estimated AME (Q4)
  Xstar <- X
```

```
  Xstar[,2] <- Xstar[,2] + 1/fit2$par[2]
  D.ame.beta <- colMeans(Xstar*fit1$par[2]*exp(drop(X%*%fit1$par)))
  V.ame <- D.ame.beta %*% V.robust %*% D.ame.beta

  ## save output of simulation b
  output[b,] <- c(fit1$par[2], #Poisson est of beta1
                  diag(Vh)[2], #hessian var of beta1
                  diag(Vopg)[2], #OPG var of beta1
                  diag(V.robust)[2], #Robust var of beta1
                  AME.est, # estimated AME
                  V.ame, #estimated variance of the AME
                  time2[3]/time1[3]) #time differences
}
```

Okay so how do we analyze this?

1. Is the Poisson estimate of β_1 close enough to 1?

```
mean(output[,1]) ## A little bit less but not bad
```

```
## [1] 0.9132749
```

2. Is the true variance of this estimator better estimated using OPG, the inverse Hessian, or the robust sandwich estimator?

```
var(output[,1]) ## observed variance
```

```
## [1] 0.06516022
```

```
round(colMeans(output[,2:4]),3) ##variance estimates
```

```
## [1] 0.000 0.000 0.041
```

The first two where we incorrectly assume we got it right are way to small, The "robust" standard errors do much better here, but they still slightly underestimate the observed variance.

3. Is the Poisson estimate of the AME close to the true value?

```
trueAME
```

```
## [1] 33.11545
```

```r
mean(output[,5]) ## slightly under estimates on average
```

```
## [1] 29.87963
```

```r
## out of curiousity, do we do well on percentage
## change estimates?
c(mean(exp(output[,1])-1), exp(1)-1) #yes
```

```
## [1] 1.580981 1.718282
```

4. Is the robust variance a good estimate for the true variance of the AME?

```r
var(output[,5]) ## observed variance
```

```
## [1] 314.8254
```

```r
mean(output[,6]) ##variance estimates
```

```
## [1] 275.2221
```

```r
## under estimates our uncertainty
```

5. How much slower is the optimization if we do not include the gradient?

```r
mean(output[,7]) ##nearly 3 times slower with score on average
```

```
## [1] 1.325697
```

4.2 Bootstrapping

The very last thing we're going to discuss in this section is the use of simulation methods to estimate variance and confidence intervals when either

1. We can't find an analytic solution or it's too hard
2. We don't believe some of the assumptions that underpin our asymptotic results

In these cases we can turn to a set of simulation methods known as bootstraps. There are many different kinds of bootstrap methods. We'll introduce two of the easier and more common ones.

The first is called a *parametric bootstrap*. It relies on our assumption of the model/DGP, $f(\theta)$. Specifically, we treat our estimated MLEs as the true model parameters and then we generate new simulated datasets using $f(\hat{\theta})$. The steps here are listed in Algorithim **??**

Algorithm 5: Parametric bootstrap

Input: Number of simulations, B; Sample size N; Estimated parameters $\hat{\theta}$; A function that generates data from the assumed model `rmodel`; A function that fits the model using the new simulated data `fitmodel`; A function f to produce the length-K quantities of interest.

Output: Estimated variance matrix of $f(\hat{\theta})$

$output \leftarrow \texttt{matrix}(0, N, K)$
for $b \leftarrow 1$ *to* B **do**
  $y^* \leftarrow \texttt{rmodel}(N, \hat{\theta})$
  $\hat{\theta}^* \leftarrow \texttt{fitmodel}(y^*)$
  $output[b,] \leftarrow f(y^*, \hat{\theta})$
return `var`$(output)$

An alternative version of the parametric bootstrap relies on using sampling from the large sample distribution (i.e., the normal) of the MLEs. This approach is sometimes referred to as "Clarify" based on a Gary King paper from sometime ago.

Algorithm 6: "Clarify:" A type of parametric bootstrap

Input: Number of simulations, B; Estimated parameters, $\hat{\theta}$; Variance matrix $\hat{V}$ of $\hat{\theta}$; A function f to produce quantity of interest; Data y.

Output: Estimated variance matrix of $f(\hat{\theta})$

$\theta^* \leftarrow \texttt{mvrnorm}(B, \hat{\theta}, \hat{V})$
$output \leftarrow f(\theta^*, y)$
return `var`$(output)$

The advantages of the parametric bootstrap, is that it tends to reasonably easy and fast. The drawback, is that it is more model dependent than it's non-parametric cousin.

The non-parametric bootstrap uses only the observed data, but resamples from it to create simulated datasets.

Specifically, it relies on the *empirical distribution* to serve as an estimate the true distribution. Consider a sample $y_1, \ldots, y_N \overset{iid}{\sim} F(y)$ where F is an unknown CDF with some parameter of interest θ that is a function of the distribution $\theta = T(F(y))$. We can consider the empirical distribution function (EDF) F_N as an approximation of F.

$$F_N(y) = \frac{1}{N} \sum_{i=1}^{N} \mathbb{I}(y_i \leq y),$$

which is a discrete distribution that puts probability $1/N$ on each observation. We want to be able to use this CDF to say something about true CDF and more importantly for us usually, some function of it T.

For example, if T is the expected value as in

$$T(F(y)) = \mathrm{E}[y] = \int yf(y)dy = \mu_y$$

and substituting the EDF gives us

$$T(F_N(y)) = \sum_{i=1}^{N} y_i f_N(y_i) = \frac{1}{N}\sum_{i=1}^{N} y_i = \hat{\mu}_y,$$

which better known as the sample mean. If we wanted to know the variance of this estimate without knowing anything else about the true distribution, we would ask, well do we think the EDF is a good approximation of the CDF? So long we say yes to that, then we can use the EDF as if it were the CDF and draw "new" samples from it.

You may be familiar with how to sample from CDFs using built in commands, but what do we do when such a function doesn't exist? One useful trick for sampling from distributions, generally, is called inverse uniform sampling or inverse transformation sampling.

Algorithm 7: INVERSE TRANSFORMATION ALGORITHM

Input: Number of observations to draw, N; inverse CDF of F, $\mathtt{Q}$.
Output: A pseudo-random sample of size n from distribution F

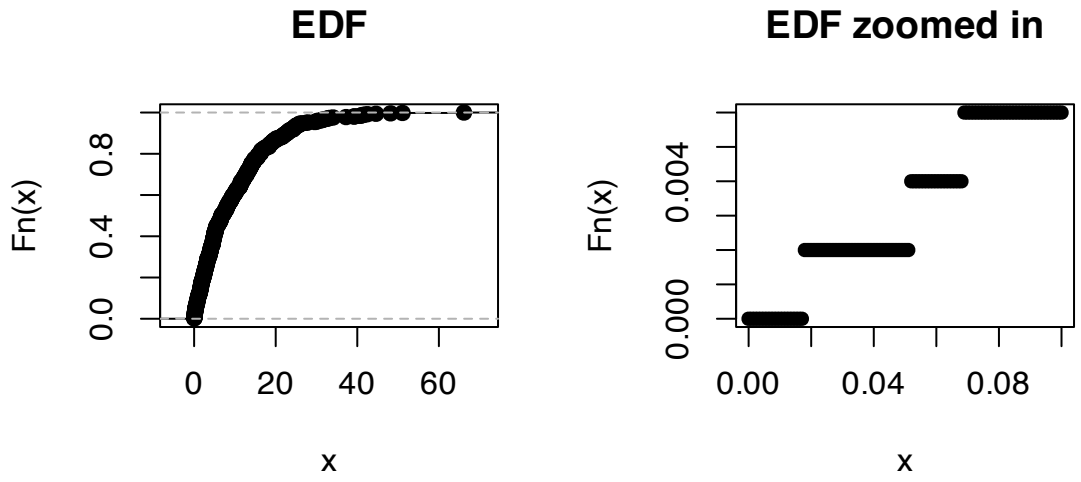
1 $U \leftarrow \mathtt{uniform}(N, 0, 1)$
2 $q \leftarrow \mathtt{Q}(U)$
3 **return** q

The intuition is that $U_n \sim U[0,1]$ gives us a random vector of probabilities to feed into the inverse CDF $Q : [0,1] \to y$.

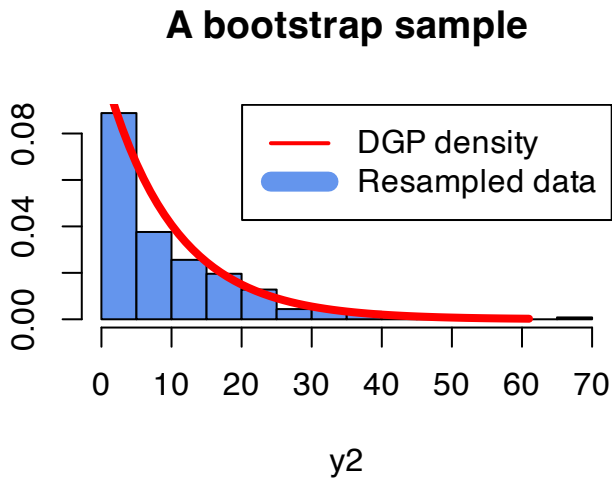
In the case of the EDF, we have a step function and so inversion is just the quantile function. Which is nice.

```
set.seed(10)
y <- rexp(500, 1/10)
par(mfrow=c(1,2))
plot(ecdf(y), main="EDF")
curve(ecdf(y)(x), from=0, to=0.1, type="p", pch=16,
```

```
ylab="Fn(x)",
main="EDF zoomed in")
```

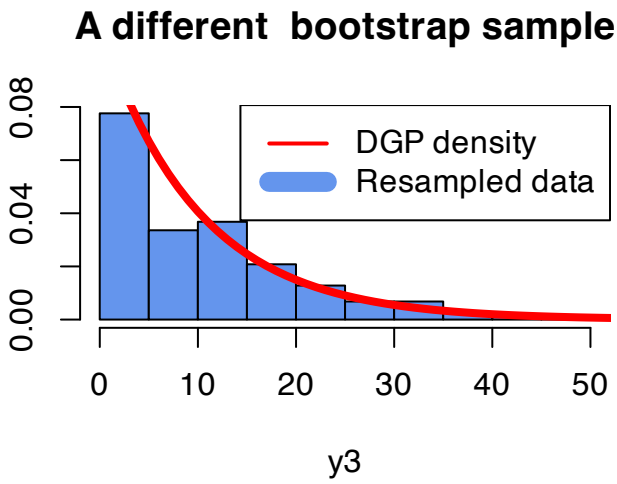



```
U <- runif(500)
y2 <- quantile(y, probs=U, type=1)
```



This looks a little convoluted for what it actually is. Drawing samples from the EDF, is just a fancy way to say resample the data with replacement!

```
y3 <- sample(y, size=500, replace=TRUE)
```



Algorithm 8: NON-PARAMETRIC BOOTSTRAP

Input: Number of simulations, B; A sample size N; Estimated parameters $\hat{\theta}$; A function f to produce the length-K quantities of interest; Data y.

Output: Estimated variance matrix of $f(\hat{\theta})$

$Q \leftarrow \texttt{matrix}(0, B, K)$

for $b \leftarrow 1$ *to* B **do**

  $y^* \leftarrow \texttt{sample}(y, \texttt{size} = N, \texttt{replace} = \texttt{TRUE})$

  $Q[b,] \leftarrow f(y^*, \hat{\theta})$

return $\texttt{var}(B)$

Or in words, for bootstrap iteration $b = 1, 2, \ldots, B$:

1. Resample N observations with replacement to create a new data set
2. Fit the model using this new data set
3. Compute and save any quantities of interest using the estimates $\hat{\theta}_b$

```
#### Bootstrapping examples ####
## basics
library(readstata13)
library(matrixStats)
library(MASS)
library(dplyr)
## econometric
library(lmtest)
library(sandwich)
## tables and figures
library(xtable)
```

```
rm(list=ls())


protests <- read.dta13("Code/R/datasets/remittances_and_protests1.dta")
protests <- subset(protests, sample==1)


f1 <- banks_protest_count ~ remit*dict +
l1gdp + #GDP pc capita
l1pop+ #population
l1nbr5 + #neighboring protests
l12gr+ #Economic growth
l1migr+ #Net migration
elec3+ #election
factor(cowcode)-1

## Poisson: ML-Dummy variable
poi <- glm(f1, data=protests, family=poisson, y=TRUE, x=TRUE)


##  parametric bootstrap
B <- 100 ##number of bootstraps. should be larger, but this is for an example
boot.out1 <- matrix(0, ncol=2, nrow=B)
X0 <- X1 <- poi$x
X0[,"dict"] <- X0[,"remit:dict"] <- 0
X1[,"dict"] <- 1
X1[, "remit:dict"] <- X1[,"remit"]
for(b in 1:B){
y.sim <- rpois(length(poi$y), lambda = poi$fitted.values)
boot.mod <-  glm(update(f1, y.sim~.),
             data=protests, family=poisson)
poissonAME <-  c(boot.mod$coefficients['remit']*
               mean(exp(X0 %*% boot.mod$coefficients)),
             (boot.mod$coefficients['remit'] +
                boot.mod$coefficients["remit:dict"])*
```

```
              mean(exp(X1 %*% boot.mod$coefficients)))

boot.out1[b,] <-poissonAME

}
se.parametric <- colSds(boot.out1)

## non-parametric bootstrap
boot.out2 <- matrix(0, ncol=4, nrow=B)

dim(protests)
```

```
## [1] 2429   43
```

```
dim(poi$x)
```

```
## [1] 2429  111
```

```
for(b in 1:B){
## ordinary bootstrap
id1 <-  sample(1:nrow(protests), replace=TRUE)
boot.mod <-  glm(f1,data=protests[id1,],
             family=poisson,x=TRUE)
X0 <- X1 <- boot.mod$x
X0[,"dict"] <- X0[,"remit:dict"] <- 0
X1[,"dict"] <- 1
X1[, "remit:dict"] <- X1[,"remit"]
poissonAME1 <-  c(boot.mod$coefficients['remit']*
                mean(exp(X0 %*% boot.mod$coefficients)),
              (boot.mod$coefficients['remit'] +
                 poi$coefficients["remit:dict"])*
                mean(exp(X1 %*% boot.mod$coefficients)))

## clustered
id2 <-  sample(unique(protests$cowcode), replace=TRUE)
boot.df <- sapply(id2, \(x){subset(protests, cowcode==x)},
              simplify=FALSE)
boot.df <- lapply(1:length(boot.df),
```

```
              \(x){
                boot.df[[x]]$cowcode <- x
                return(boot.df[[x]])
              })
boot.df <- do.call(rbind, boot.df)
boot.mod <-  glm(f1,data=boot.df,
             family=poisson,x=TRUE)
X0 <- X1 <- boot.mod$x
X0[,"dict"] <- X0[,"remit:dict"] <- 0
X1[,"dict"] <- 1
X1[, "remit:dict"] <- X1[,"remit"]
poissonAME2 <-  c(boot.mod$coefficients['remit']*
                mean(exp(X0 %*% boot.mod$coefficients)),
              (boot.mod$coefficients['remit'] +
                 boot.mod$coefficients["remit:dict"])*
                mean(exp(X1 %*% boot.mod$coefficients)))
poissonAME2

boot.out2[b,] <- c(poissonAME1,poissonAME2)

}
se.nonparametric <- colSds(boot.out2)

## Clarify: parametric bootstrap
## Here we'll refit the model to drop the all-zero cases because
## we're relying on the estimated variance.
## For the fixed effects on the all zero units
## these will also be excessively large (these parameters are unidentified)
protests2 <- protests %>%
mutate(sum.y= sum(banks_protest_count), .by=cowcode) %>%
filter(sum.y > 0)
poi <- update(poi, data=protests2)

Bmat1 <- mvrnorm(B, poi$coef,vcov(poi))
Bmat2 <- mvrnorm(B, poi$coef,vcovCL(poi, protests2$cowcode))
X0 <- X1 <- poi$x
```

```
X0[,"dict"] <- X0[,"remit:dict"] <- 0
X1[,"dict"] <- 1
X1[, "remit:dict"] <- X1[,"remit"]
poissonAME1 <-  cbind(Bmat1[,'remit']*
                  colMeans(exp(X0 %*%t(Bmat1))),
                (Bmat1[,'remit'] + Bmat1[,"remit:dict"])*
                  colMeans(exp(X1 %*%t(Bmat1))))
poissonAME2 <-  cbind(Bmat2[,'remit']*
                  colMeans(exp(X0 %*%t(Bmat2))),
                (Bmat2[,'remit'] + Bmat2[,"remit:dict"])*
                  colMeans(exp(X1 %*%t(Bmat2))))
se.clarify <- c(colSds(poissonAME1), colSds(poissonAME2))


compMat <- rbind(parametric=c(se.parametric, NA, NA),
           non.parametric=se.nonparametric,
           clarify=se.clarify)
colnames(compMat) <- c("Auto", "Demo", "Auto-cluster", "Demo-cluster")
print(compMat)
```

```
##                      Auto       Demo Auto-cluster Demo-cluster
## parametric     0.02942927 0.04646503           NA           NA
## non.parametric 0.05197231 0.07079552   0.07161865    0.1976711
## clarify        0.03126906 0.05064629   0.07829059    0.1260310
```

We can also go back to our IV-Poisson to demonstrate using the bootstrap in the two-step setting. We'll use this opportunity to demonstrate the idea of parallel processing. Monte Carlos and bootstrap simulations are sometimes referred to as "embarassingly parallel" operations. What this means is that each iteration is whole independent of the others and so there's no real reason why they have to be done sequentially. We don't use the results of one iteration to inform the next. In theory, with a enough computing power we could do 100 bootstrap iterations all at once side-by-side. This should decrease the time it takes to do these simulations down to just the time it takes to do one.

Often times, we're not in a world we have **that** much computing power, but we are in world where nearly every computer processor has multiple "cores" or "threads." Most R functions only use 1 of these cores at time leaving your processor free to look at cat memes while you

wait for this code to run. However with some small tweaks we can farm out some of our iterations to other cores so maybe we can run 2 or 3 or 30 side by side.

There are few factors that go into making the decision of how many cores you should use at once:

1. Memory: if your iterations involve a lot of data (big bootstrapping problems) then you may be limited. Each new core we recruit needs to use the same amount of memory.
2. Functions that already farm out work. While most R functions stick to a single process some do not, this is more likely for well made packages where real programmer tries to optimize their code. This is great for using those packages day-to-day but can become an issue if you're competing with the underlying code for cores
3. What else you want to do while it's running. If you want to watch Netflix versus just letting it run over night can sway you one way or another.

Generally speaking, if you notice your computer actually slowing or freezing it could be a sign that you're asking too much of your memory. Keeping an eye on your machine's system monitor can also be helpful in terms of knowing how much/little you're asking of it.

```
## some packages for parallel computing
library(doParallel) ## main work horse
library(doRNG) ## an update that allows us to set a seed in parallel

## number of available cores
detectCores()
```

```
## [1] 8
```

```
#let's use 3/4 of them
ncores <- floor(.75*detectCores())



cl <- makeCluster(ncores) # creates copies of R that can run side-by-side
registerDoParallel(cl) # connect our cores to the foreach loop we're about to run

## instead of for we now have foreach
## Differences:
######  1. save the output from the loop up top rather
######     than in pre-initalized matrix
```

```
######  2. b=1:B rather than b in 1:B
######  3. It has additional options for how to combine the output,
######     whether to export any packages to our other processes,
######     what to do with errors, and so on. See ?foreach for more
######  4. We give it a "do" option. %do% is the same as for (not parallel)
######     %dopar% is parallel and %dorng% is parallel but it respects the seed.

set.seed(1)
B <- 500
bootMat <- foreach(b=1:B, .combine = "rbind")%dorng%{
  id2 <-  sample(unique(protests$cowcode), replace=TRUE)
  boot.df <- sapply(id2, \(x){subset(protests, cowcode==x)},
                    simplify=FALSE)
  boot.df <- lapply(1:length(boot.df),
                    \(x){
                      boot.df[[x]]$cowcode <- x
                      return(boot.df[[x]])
                    })
  boot.df <- do.call(rbind, boot.df)

  m1 <- lm(remit~ distwremit*dict +
             l1gdp + #GDP pc capita
             l1pop+ #population
             l1nbr5 + #neighboring protests
             l12gr+ #Economic growth
             l1migr+ #Net migration
             elec3+ #election
             factor(cowcode)-1,
           x=TRUE,y=TRUE,
           data=boot.df)
  m2 <- lm(I(remit*dict)~ distwremit*dict +
             l1gdp + #GDP pc capita
             l1pop+ #population
             l1nbr5 + #neighboring protests
             l12gr+ #Economic growth
             l1migr+ #Net migration
```

```
              elec3+ #election
              factor(cowcode)-1,
            x=TRUE,y=TRUE,
            data=boot.df)

  boot.df$v1 <- m1$residuals
  boot.df$v2 <- m2$residuals

  step2 <- glm(banks_protest_count ~ remit*dict +
                 l1gdp + #GDP pc capita
                 l1pop+ #population
                 l1nbr5 + #neighboring protests
                 l12gr+ #Economic growth
                 l1migr+ #Net migration
                 elec3+ #election
                 v1 + v2+
                 factor(cowcode)-1,
               data=boot.df,
               family=poisson,
               x=TRUE,
               y=TRUE)
  out <- step2$coefficients[c(1:10,length(step2$coef))]
  if(any(abs(out)>100)){
    ##sometimes weird things happen. Poisson coefficients should not
    ## be more than 100. definite sign of numeric issue
    out <- rep(NA, length(out))
  }
  out
}

## close those other copies so they're not just
## creeping in the background
stopCluster(cl)

dim(bootMat)
```

```
## [1] 500  11
```

```
dim(na.omit(bootMat))
```

```
## [1] 498  11
```

```
cbind(Est=poi.iv$coefficients[c(1:10,length(poi.iv$coef))],
      AnalyticSE=sqrt(diag(V2step)[c(1:10,length(poi.iv$coef))]),
      BootstrapSE=sqrt(diag(var(bootMat,na.rm=TRUE))))
```

```
##                    Est AnalyticSE BootstrapSE
## remit      -0.29957869 0.16364590  0.26756546
## dict       -8.61614589 5.93825804  9.28243175
## l1gdp       0.58789158 0.47058068  0.76698370
## l1pop      -0.50313866 0.80883721  1.03397790
## l1nbr5     -0.29690288 0.35865844  0.45442102
## l12gr      -0.03024825 0.02105581  0.02449027
## l1migr      0.02379525 0.14513906  0.19727059
## elec3       0.08586825 0.13444938  0.15350849
## v1          0.36049067 0.16939471  0.27172060
## v2         -0.50997441 0.42035618  0.66293359
## remit:dict  0.60901360 0.41628320  0.65282188
```

5 GLMs: Binary, ordered, and multinomial outcomes

We now change our focus to look at categorical outcomes. These come in many forms, but their commonality is that we consider some finite set of outcomes $y_i \in \{0, 1, \ldots, J\}$.

5.1 Binomial/Bernoulli GLM

We will start with the simplest case where y_i is binary, making it a Bernoulli random variable. This basics are

$$f(y|p) = p^y(1-p)^{1-y}, \quad y \in \{0, 1\}$$
$$\mathrm{E}[y] = p$$
$$\mathrm{Var}(y) = p(1-p)$$

At this point, we have two different options. One is to return to the linear model and impose a link-function of

$$\mathrm{E}[y_i \mid X] = x_i'\beta.$$